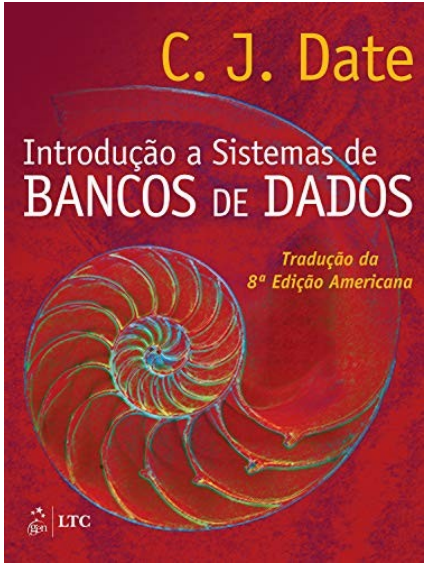
Banco de Dados

Apresentação (Ementa)

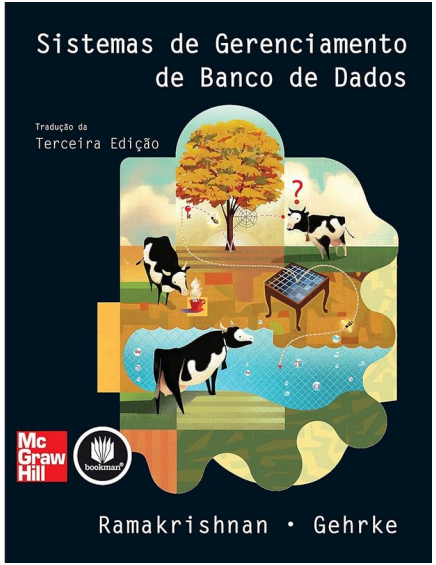
BANCO DE DADOS

Ementa: Conceitos básicos de um SGBD. Estrutura de um SGBD. Linguagem de definição de dados e Linguagem de manipulação de dados. O modelo relacional. Aspectos de implementação dos SGBDs relacionais: indexação, métodos de acesso e "bufferização". Estrutura de arquivos e armazenamento. Store procedures. Replicação de dados. Acesso remoto. Padrão ODBC. Aspectos de integridade, segurança e privacidade. Elaboração e implementação de um projeto de Sistemas de Banco de Dados. Aspectos de segurança, integridade e privacidade: autorizações, visões, bloqueios, triggers, asserções. Novas aplicações para Banco de Dados..

Bibliografia



Christopher J. Date





https://pearson.unirv.edu.br/

Metodologia de Avaliação

- **Avaliação 1**
  - Avaliação (Peso **7**)
  - Atividades (Peso **3**)
- **Avaliação 2**
  - Avaliação (Peso **7**)
  - Atividades (Peso **3**)
- **Avaliação 3**
  - Avaliação (Peso **4**)
  - Atividades (Peso **6**)

Introdução à Banco de Dados

Introdução

- **O que são dados ?**

Introdução

➔ O que são dados ?

- Tudo que **podemos inferir** ou **coletar** sobre uma situação específica.
- São valores **brutos** e **isolados** que, por si só, não possuem significado específico.
- Um dado considerado isoladamente não representa uma informação útil

  Exemplo de dados:
  - quantidade de carteiras
  - cor da parede
  - dimensões da sala
  - entre outros.

➔ Domínio de um dado

- O domínio de um dado é o conjunto dos valores que um dado pode armazenar:

- Funciona como uma regra ou limite que define quais valores são permitidos para aquele dado.

- Está ligado ao tipo de dado (inteiro, texto, data etc.) e, às vezes, a restrições adicionais (intervalos, listas de valores permitidos, formato).

➔ Domínio de um dado

Tipo de dado	**Domínio**
Define a forma do dado	Define o **conjunto exato** de valores válidos
Ex.: INT	Ex.: 0 a 120
Ex.: CHAR(2)	Ex.: "SP", "RJ", "MG"

Introdução

Os **dados úteis** são chamados de **informação**!

Introdução

→ **O que é Informação?**

- É o dado associado à sua semântica (metadado);
- Quando os dados são organizados e contextualizados, eles se transformam em **informação**.
- A informação têm significado intrínseco

- É o resultado do **processamento de dados** para dar-lhes significado.

- A informação é um recurso valioso que permite gerar e difundir conhecimento.

➔ O que é Informação?

- Exemplo:

Paulo: aluno de informática;
16/06/2001: primeira aula de sistemas distribuídos;
R$ 120,00: valor pago de telefone pelo acesso Internet.

Introdução

Por convenção, na **área de banco de dados**, os termos "**informação**" e "**dado**" *significam a mesma coisa*.

- Pois devemos **armazenar** apenas aquilo que é útil para a nossa aplicação.
- Daqui para frente, os dois termos serão usados como sinônimos.

Introdução

➔ **Base de Dados**

- É uma coleção **organizada de dados**, que pode ser facilmente acessada, gerenciada e atualizada.
- Logicamente coerente com significado intrínseco;
- Agrupados e definidos com um objetivo bem determinado;
- Representa alguns aspectos do mundo real.

Introdução

➔ **Base de Dados**

- Pode ser um **conjunto de tabelas**, **arquivos** ou qualquer outra forma estruturada de armazenar dados.

- Uma base de dados terá um **nome**, e este nome deverá representar o que aquela base armazena.

  Exemplo de base de dados:
  - Uma **planilha do Excel** contendo os nomes, idades e endereços de clientes.

Introdução

➔ **Metadados**

- Dados sobre os dados”
- São os dados referentes à estrutura dos dados em um banco de dados armazenado no dicionário de dados;
- Os metadados são usados para descrever tabelas, colunas, restrições, índices e assim por diante.

Introdução

➔ **Banco de Dados**

- É a Base de Dados + Metadados:
- É um repositório que armazena **bases de dados** de forma estruturada.
- Pode **suportar múltiplas bases de dados**.
- **Esquema:** projeto geral do DB; descrição da estrutura; definição dos dados.
- **Instância:** dados contidos em um determinado instante.

Introdução

➔ **Banco de Dados**

- É gerenciado por um **Sistema de Gerenciamento de Banco de Dados (SGBD)**.

- Dentro de um SGBD você pode criar diversos banco de dados.

O Que É um Sistema de Banco de Dados (SBD) ?

- Computacionalmente, um **Sistema de Banco de Dados** é uma **ferramenta** que será utilizada para *armazenar informações*.

- Responsável pela criação (definição e construção) e pela manipulação das bases de dados:

O Que É um Sistema de Banco de Dados (SBD) ?

- Definição: especificação dos tipos de dados que serão armazenados juntamente com a descrição detalhada de cada tipo;
- Construção: processo de armazenamento dos dados em um meio físico controlado pelo SGBD;
- Manipulação: recuperação e atualização de dados específicos contidos na base de dados com a finalidade de refletir mudanças da realidade representada.

Introdução

➔ **Essa ferramenta possui três principais características:**

- Armazenar os dados.
- Relacionar os dados armazenados.
- Recuperar os dados rapidamente.

O Que É um Sistema Gerenciador de Banco de Dados (**SGBD**)?

- É uma ferramenta muito mais completa que um Sistema de Banco de Dados.
- Um SGBD disponibiliza uma série de funcionalidades que permitem controlar e acompanhar melhor os dados armazenados.
- Exemplos: MySQL, Oracle Database, Microsoft SQL Server, entre outros.

Introdução

➔ **As principais características de um SGBD são:**

- Permitir o **acesso concorrente** às bases de dados;
- Realizar o **gerenciamento de transações**;
- Permitir criar e aplicar **regras de segurança** às bases de dados;
- Permitir criar **regras** que garantam a **integridade** da base de dados.

Introdução

➔ **As principais características de um SGBD são:**

- Permitir o **acesso concorrente** às bases

> ❑ Duas ou mais pessoas **acessem** uma mesma base de dados ao **mesmo tempo**.

- Realizar o **gerenciamento de transações**;
- Permitir criar e aplicar **regras de segurança** às bases de dados;
- Permitir criar **regras** que garantam a **integridade** da base de dados.

Introdução

➔ **As principais características de um SGBD são:**

- Permitir o **acesso concorrente** às bases
- Realizar o **gerenciamento de transaçõ**
- Permitir criar e aplicar **regras de segurança** às bases de dados;
- Permitir criar **regras** que garantam a **integridade** da base de dados.

❑ Uma **transação** possui várias **operações**.

❑ Falha durante uma transação?

- Antes e Depois

Introdução

➔ **As principais características de um SGBD são:**

- Permitir o **acesso concorrente** às bases
- Realizar o **gerenciamento de transac**
- Permitir criar e aplicar **regras de segurança** às bases de dados;
- Permitir criar **regras** que garantam a **integridade** da base de dados.

- ❑ Definição de login e senha.
- ❑ Acesso ao SGBD
- ❑ Acesso aos dados armazenados.

Introdução

➔ **As principais características de um SGBD são:**

- Permitir o **acesso concorrente** às bases
- Realizar o **gerenciamento de transaçõ**
- Permitir criar e aplicar **regras de segurança** ases de dados;
- Permitir criar **regras** que garantam a **integridade** da base de dados.

❑ Uma **regra** para **toda** a **base de dados.**

❑ As aplicações podem utilizar.

❑ Ex: estados, parcelamentos

Relação entre os termos

1. **Dados** são os elementos mais básicos.

2. **Informação** é o que obtemos ao processar e contextualizar os dados.

3. Uma **base de dados** organiza os dados de forma estruturada.

4. Um **banco de dados** é um repositório que pode conter uma ou mais bases de dados.

5. Um **sistema de banco de dados** é uma ferramenta que será utilizada para armazenar informações.

6. Um **SGBD** é o sistema que gerencia os bancos de dados, permitindo a manipulação e acesso aos dados de forma eficiente.

Quem Usa um Banco de Dados?

1. **Administrador do banco de dados** (DBA)

    i. Monitora e gerencia todas as bases de dados criadas no SGBD.
    ii. Controla as permissões dos usuários.
    iii. Realiza backups.
    iv. recupera os dados em caso de falhas.
    v. Garante o melhor desempenho para o banco de dados.
    vi. Um DBA é responsável por um SGBD específico.

Quem Usa um Banco de Dados?

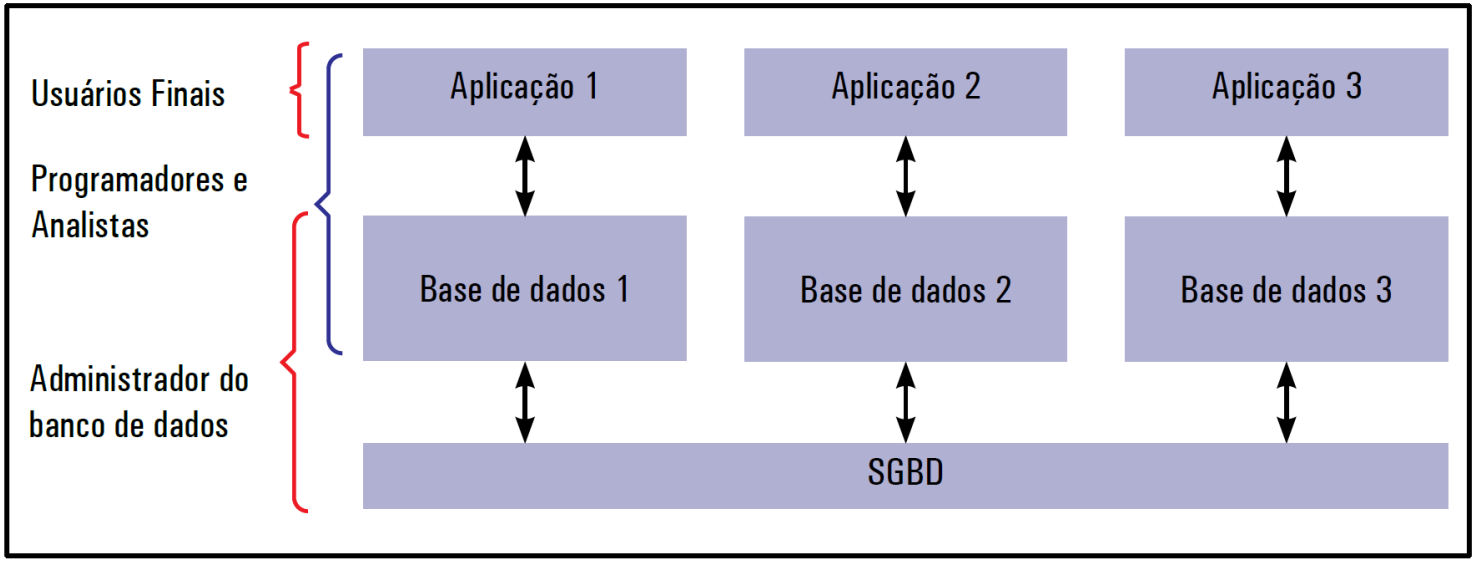
1. **Analistas de sistemas e programadores de aplicações**

    i. Modelar a base de dados e implementá-la no SGBD escolhido.
    ii. Desenvolver a aplicação (Java, PHP, C++, C#, etc)
    iii. Conectar essa aplicação à base de dados do sistema.
    iv. Esse usuário precisa:
        1. conhecer a área de banco de dados
        2. saber modelar uma base de dados
        3. Conhecer a linguagem SQL

Quem Usa um Banco de Dados?

1. **Usuários finais**

    i. Pessoas que vão trabalhar diariamente com as aplicações desenvolvidas.
    ii. Responsáveis pela entrada de dados no banco de dados
    iii. Alterações
    iv. Não precisa ter conhecimento de Banco de Dados ou saber qual SGBD.



Acesso dos usuários ao banco de dados

Desenvolvimento de um Banco de Dados

O que é um Modelo?

- É a representação abstrata de aspectos específicos sobre uma determinada realidade que permite compreender um conceito, idéia ou objeto antes de sua existência real;

- Um modelo deve ser construído com objetivos bem definidos que determinem os aspectos importantes a serem representados;

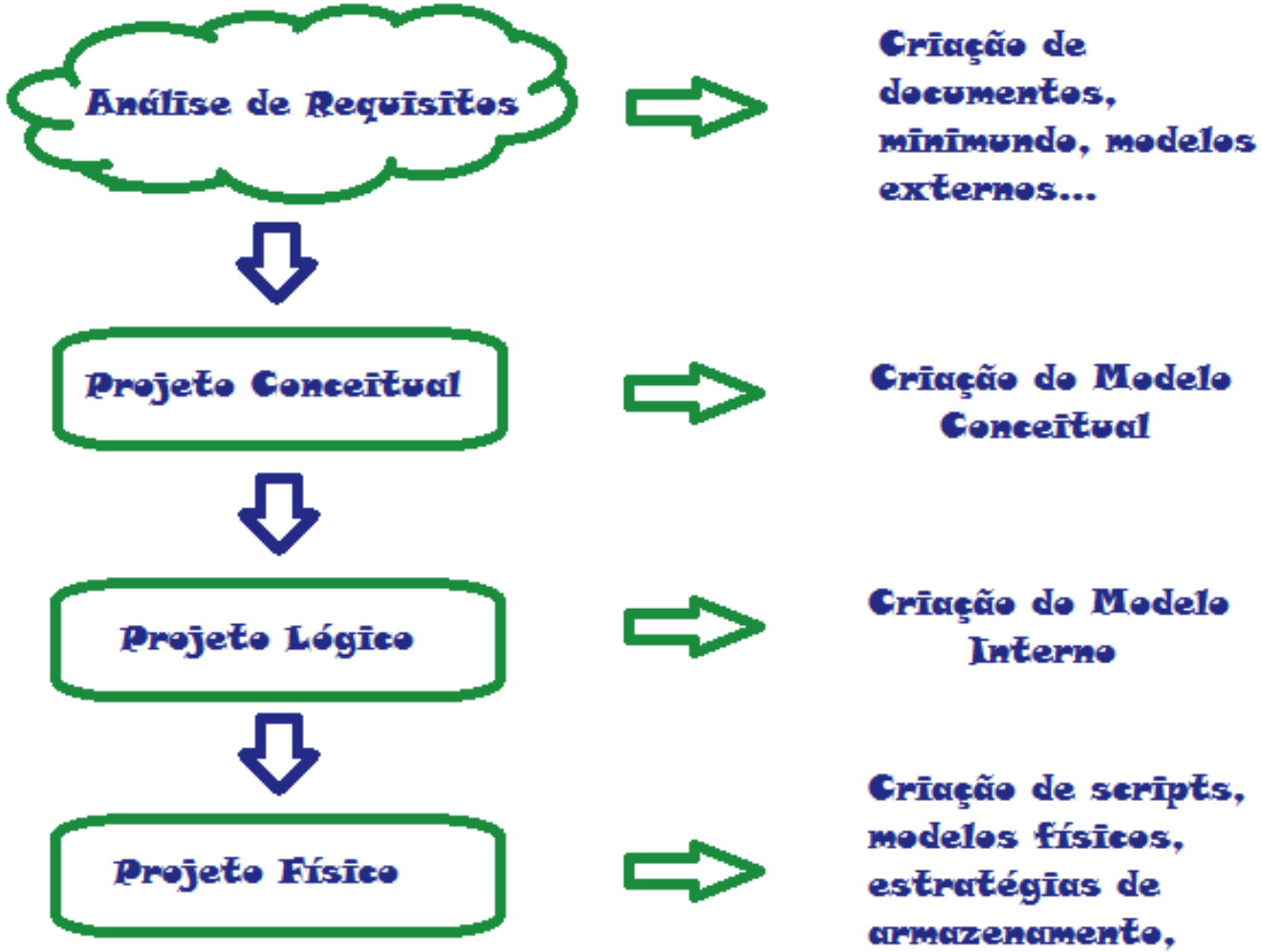
- Exemplo:

Maquete de um prédio, mapa geográfico, DFD, planta baixa, etc

Modelagem de dados

- Processo no qual busca-se a representação das estruturas de dados que nos levem ao nosso projeto de banco de dados

- Deve-se descreve:
  - → os dados,
  - → relacionamentos entre os dados
  - → semântica e restrições de integridade sobre os dados;


Análise de Requisitos
Criação de documentos, minimundo, modelos externos...
Projeto Conceitual
Criação do Modelo Conceitual
Projeto Lógico
Criação do Modelo Interno
Projeto Físico
Criação de scripts, modelos físicos, estratégias de armazenamento, backup...

Fases no Desenvolvimento de um Projeto de Banco de Dados

- **Modelagem Conceitual -** modelo inicial da base de dados que reflita as necessidades do usuário
  - Quais dados serão armazenados.
  - Quais dados se relacionam
  - Como obter os dados?
- Faz a representação fiel ao ambiente observado, independentemente de limitações quaisquer impostas por tecnologias, técnicas de implementação ou dispositivos físicos.

Fases no Desenvolvimento de um Projeto de Banco de Dados

- **Modelagem Lógica -** Descreve **como** os dados são **organizados** e **relacionados**,
  - têm a representação de acordo com as regras de implementação e limitantes impostos por algum tipo de tecnologia.
  - sem se preocupar com a forma exata de armazenamento no hardware.

  - Modelo Relacional

Fases no Desenvolvimento de um Projeto de Banco de Dados

- **Implementação do Modelo Lógico (modelo físico)**
  - Implementar ou criar a base de dados no SGBD escolhido
  - O desenvolvedor precisa conhecer de SQL e o SGBD selecionado.
  - Cada diferente SGBD poderá definir um diferente modo de implementação física das características e recursos necessários para o armazenamento e manipulação das

Desenvolvimento de um B.D

- A modelagem é fundamental.
- É desanimador e trabalhoso perceber a necessidade de alterar o modelo de dados.
- Gera um retrabalho.
- A base terá que ser refeita
- Os diagramas.
- A aplicação

- **Modelo de Dados**
  - ➔ Também conhecido como ***Esquema da base de dados***.
  - ➔ Não deve mudar com frequência
  - ➔ Uma vez alterado....
  - ➔ **Independência dos dados** em relação às aplicações (**Desejável/Difícil**)
  - ➔ Se precisar dividir duas tabelas.

- **Modelo de Dados**

  - ➔ Até dá para alterar um pouco sem muito impacto...

Cuidado!

Às vezes, mesmo mudanças simples, como a do exemplo acima, provocam grandes mudanças na aplicação. Por exemplo, o *bug* do milênio foi causado pelos tipos de dados utilizados. Em alguns casos mesmo só mudando os tipos de dados será necessário rever toda a aplicação.

https://g1.globo.com/tecnologia/noticia/2024/07/20/bug-do-milenio-atrasado-entenda-medo-global-na-virada-dos-anos-2000.ghtml

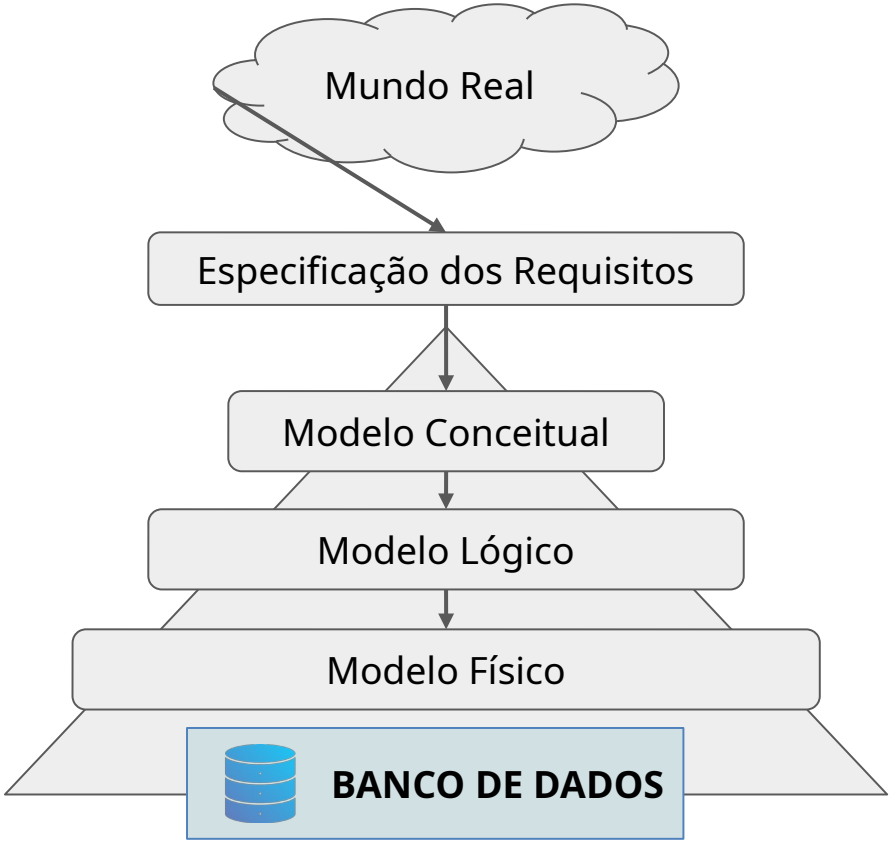
❖ Modelo de Dados

Modelo de dados **conceitual** x Modelo de dados **lógico**



É importante destacar que algumas literaturas tratam o modelo resultante do Modelo Lógico, que é dependente de um SGBD específico, de Modelo Físico.

Desenvolvimento de um B.D



Introdução ao Modelo de Entidade e Relacionamento

Introdução

➔ **Modelo de Entidade e Relacionamento (ER)**

- É um modelo conceitual.
- Está mais próximo da visão que o usuário tem dos dados.
- Descreve quais dados devem ser armazenados pela aplicação.
- Como esses dados se relacionam.

Introdução

➔ **História do MER**

- Em março de 1976, Peter P. Chen publicou um trabalho intitulado “The Entity-Relationship Model: Toward the unified view of data”, no qual definia uma possível abordagem para o processo de modelagem dos dados.
- O modelo entidade-relacionamento tem seu embasamento teórico na “Lei do Mundo”.
- A Lei do Mundo determina que “o mundo está cheio de coisas que possuem características próprias e que se relacionam entre si”.
- Portanto, deve-se sempre identificar: entidades, relacionamentos e atributos.

Introdução

➔ **História do MER**

O modelo entidade-relacionamento tem por base a percepção de que o mundo real é formado por conjunto de objetos chamados *entidades* e pelo conjunto dos *relacionamentos* entre esses objetos.

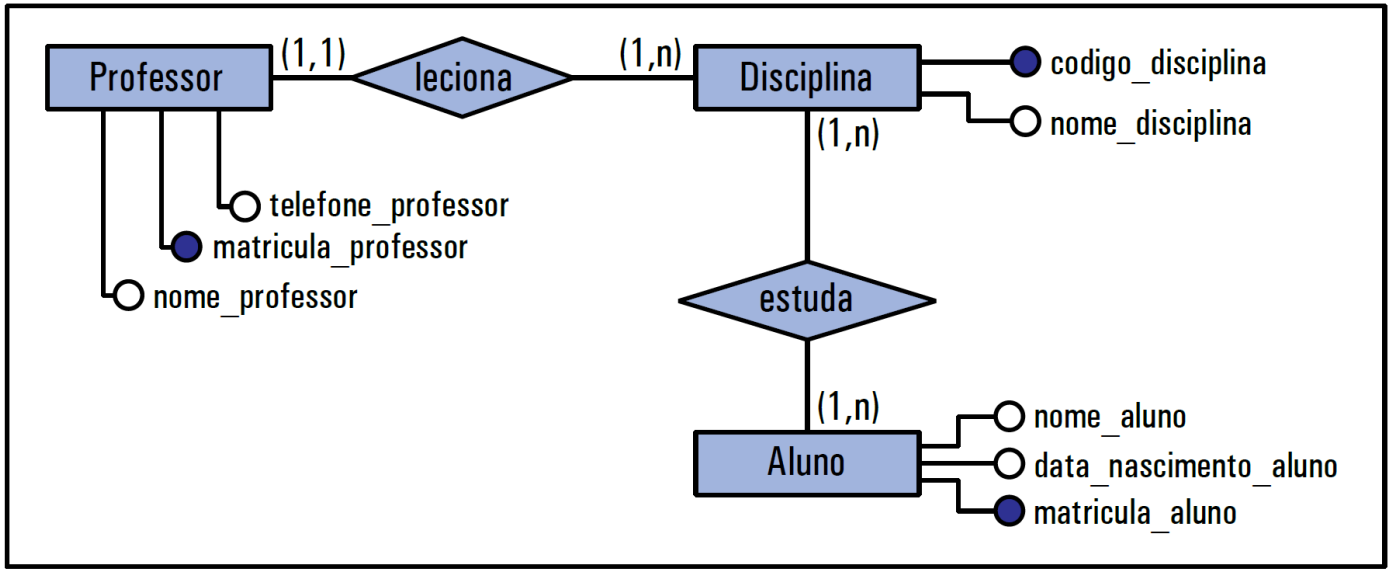
Introdução

➔ **Modelo de Entidade e Relacionamento (ER)**

- Retângulos, que representam os conjuntos de entidades;
- Elipses, que representam os atributos;
- Losangos, que representam os relacionamentos entre os conjuntos de entidades;
- Linhas, que unem os atributos aos conjuntos de entidades e o conjunto de entidades aos seus relacionamentos.

Introdução

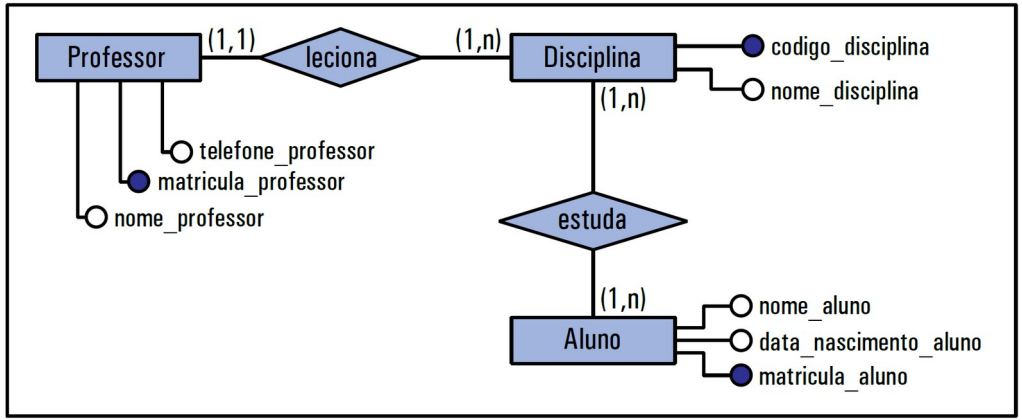
- Uma escola precisa armazenar as informações sobre seus alunos, professores, e disciplinas.



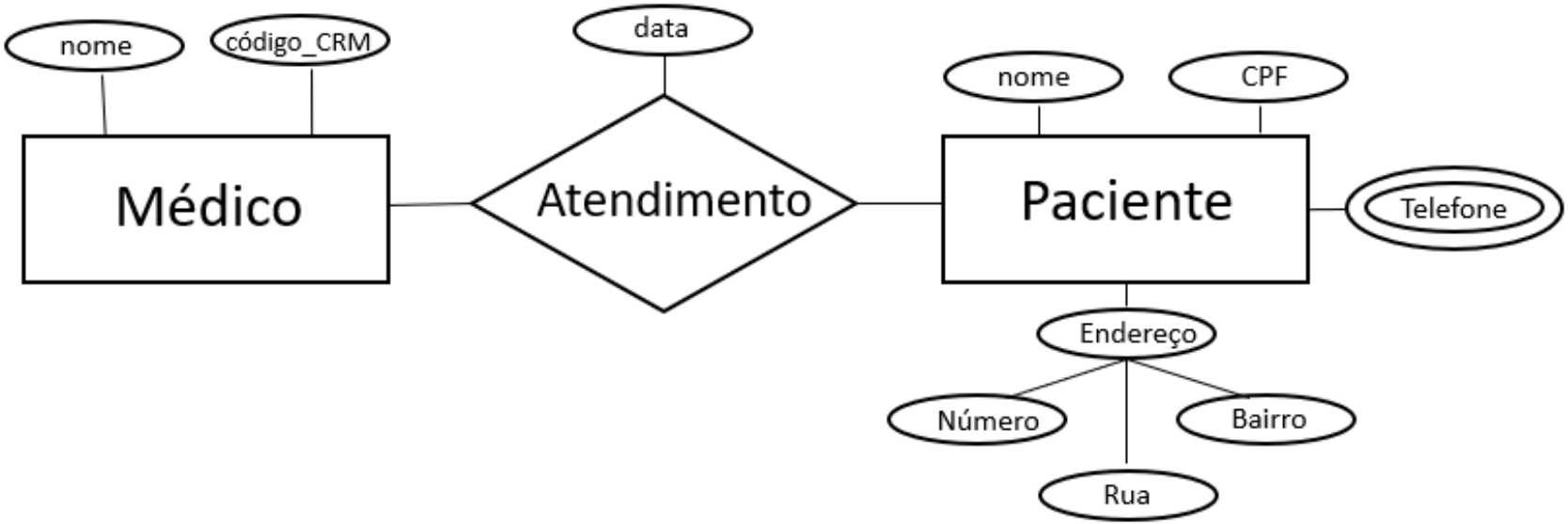
Exemplo de modelo conceitual

Introdução

- Não há preocupação em saber como os dados serão armazenados.
- Entender o que precisa ser armazenado.
- **Diagrama de Entidade e Relacionamento (DER).**



Introdução


nome
código_CRM
data
nome
CPF
Médico
Atendimento
Paciente
Telefone
Endereço
Número
Bairro
Rua

Entidades e Atributos

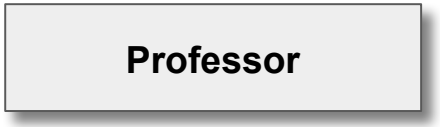
Introdução

- Uma **entidade** representa um conjunto de **objetos** do mesmo tipo do mundo real e sobre os quais se pretende armazenar dados.
- Ex: Sistema para uma escola:
  - professores, alunos, disciplinas, turmas e cursos.



Introdução

- Uma **entidade** representa um conjunto de **objetos** do mesmo tipo do mundo real e sobre os quais se pretende armazenar dados.
- Ex: Sistema para uma escola:
  - professores, alunos, disciplinas, turmas e cursos.



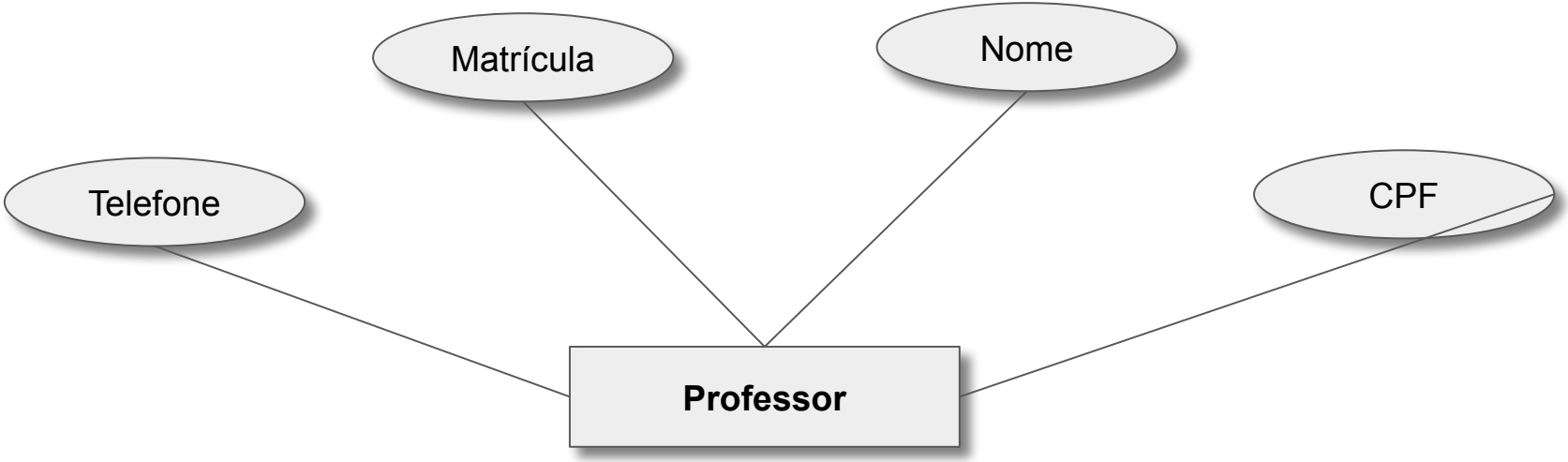
- Representada graficamente por um **retângulo** com o nome da **entidade** dentro do retângulo.

Introdução



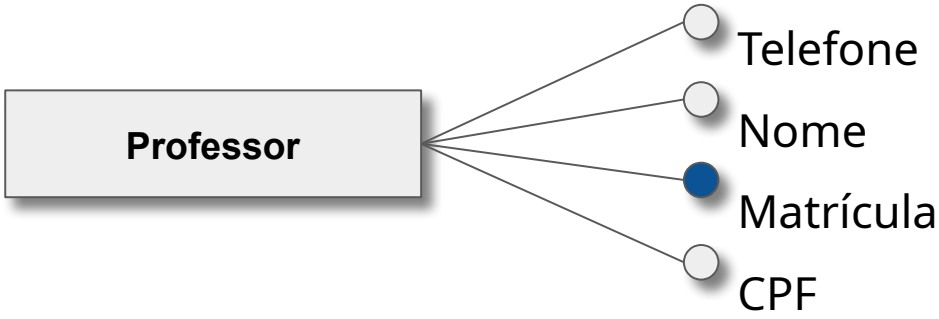
- Um conjunto de **objetos** do mesmo **tipo.**
- Um conjunto de propriedades que a caracterize e a descreve
- **Atributos**

Introdução



- Um atributo pode ser representado graficamente por uma elipse

Introdução

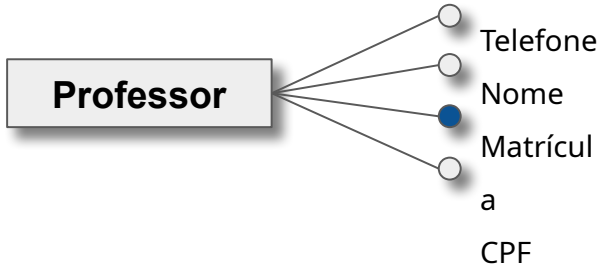


- Uma bolinha ligada à entidade e com o nome do atributo ao lado.

Introdução

- Uma entidade deve ter **ao menos dois atributos**.

- Todo atributo possui um **tipo de dado.**
  - os valores permitidos para aquele atributo**.**



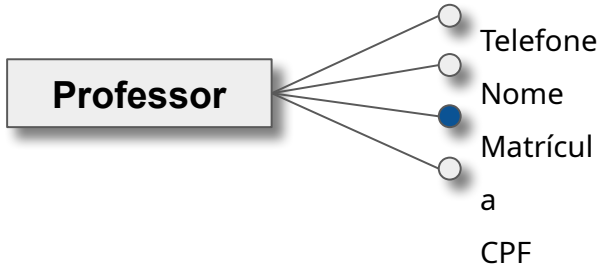
Introdução

- Uma entidade deve ter **ao menos dois atributos**.

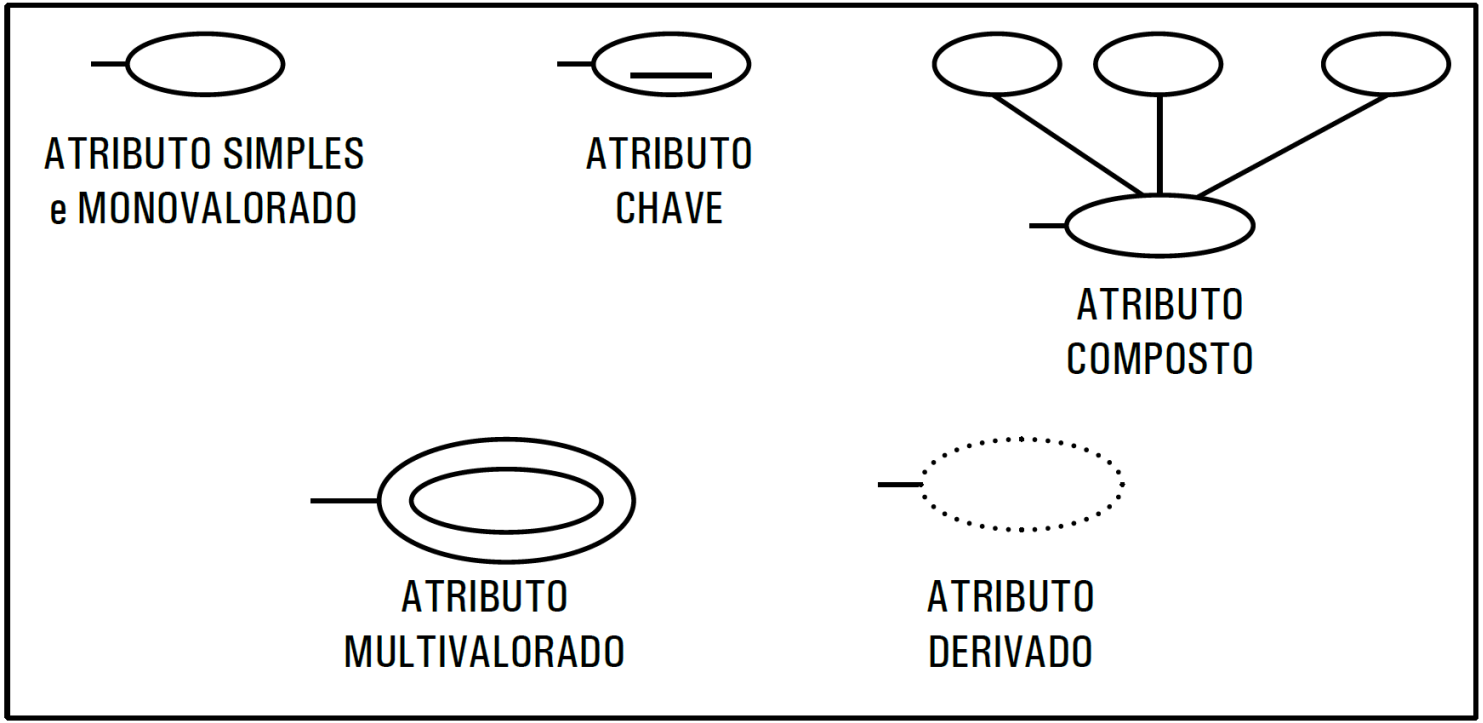
- Todo atributo possui um **tipo de dado.**
  - os valores permitidos para aquele atributo**.**

- **inteiro, caracter, real (ou float), data e hora.**

- Tamanho



Atributos (6 Categorias)

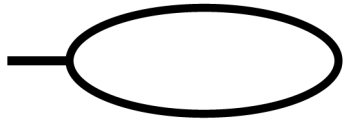


Representação gráfica dos atributos

Atributos (6 Categorias)

ATRIBUTO SIMPLES

- Indivisível - não pode ou não deve ser decomposto.
  - CPF
  - nº matrícula
  - RG
  - preço
  - etc.

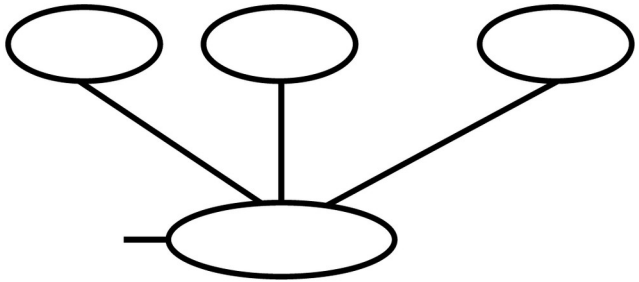


ATRIBUTO SIMPLES
e MONOVALORADO

Atributos (6 Categorias)

ATRIBUTO COMPOSTO

- pode ser decomposto.
  - Endereço
    - Nome da rua
    - número
    - complemento

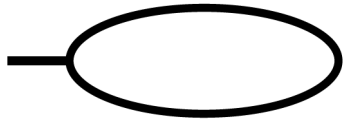


ATRIBUTO COMPOSTO

Atributos (6 Categorias)

ATRIBUTO MONOVALORADO

- Indivisível - permite apenas o armazenamento de um valor por vez.
  - CPF
  - Uma pessoa possui apenas um número de CPF.

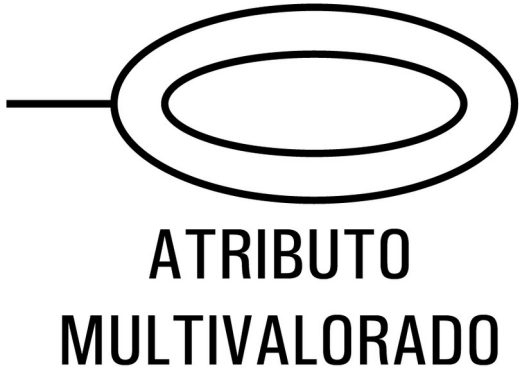


ATRIBUTO SIMPLES e MONOVALORADO

Atributos (6 Categorias)

ATRIBUTO MULTIVALORADO

- Armazena mais de um valor ao mesmo tempo no mesmo campo
  - E-mail
  - Uma pessoa pode ter vários e-mails.



Atributos (6 Categorias)

ATRIBUTO NULO

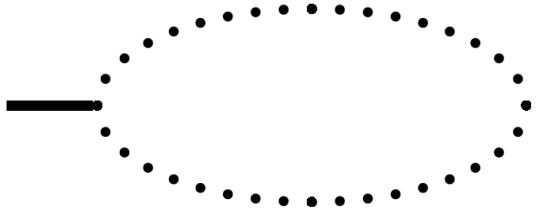
- Permite que seja inserido um valor nulo para ele.
- A inexistência de um valor.
- O usuário não precisa cadastrar um valor para o atributo e pode deixá-lo vazio.

OBS: O ideal é não ter o nulo e receber um valor padrão (*default*).

Atributos (6 Categorias)

ATRIBUTO DERIVADO

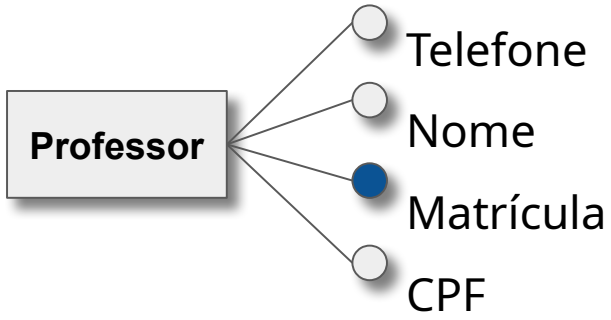
- O valor para ele deriva de outro(s) atributo(s).
- número da compra, data da compra, valor da compra, percentual de desconto e **valor da compra com o desconto**.
- “**valor da compra com o desconto**” é *calculado/derivado* de outros atributos.
- Aplicação ou SGBD



ATRIBUTO DERIVADO

Chave Primária

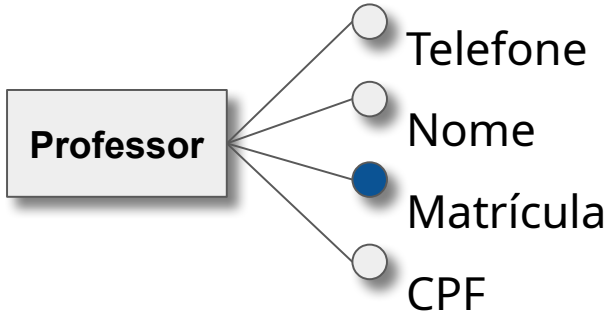
- **Primary Key** ou ainda **PK**
- É um atributo da entidade que **identifica apenas um objeto** dessa entidade.
- Esse valor não pode se repetir (**único**)
- Não pode receber nulo.
- Professor (Entidade)
  - CPF, Matrícula.





Chave Primária

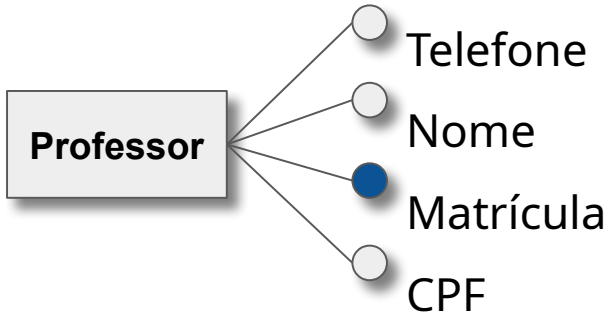
- **Qual deles eu devo definir como uma chave primária?**





Chave Primária

- **Qual deles eu devo definir como uma chave primária?**
  - Vai depender do **escopo** do problema.
  - Como serão realizadas as consultas.
  - A maioria dos SGBD vincula um índice à chave primária.





Tipos de Chave Primária

- **SIMPLES**
  - Formada por **apenas um** atributo
  - Exemplo:
    - matrícula
- **COMPOSTA**
  - Formada por **dois ou mais** atributos.
  - Exemplo:
    - Entidade (Localização)
      - nome da cidade e nome do estado

Tipos de Chave Primária

nome_cidade	nome_estado	nome_país
Curitiba	Paraná	Brasil
Maringá	Paraná	Brasil
Campo Grande	Mato Grosso do Sul	Brasil
Campo Grande	Rio de Janeiro	Brasil

Exemplos de dados armazenados na entidade “Localização”.

Tipos de Chave Primária

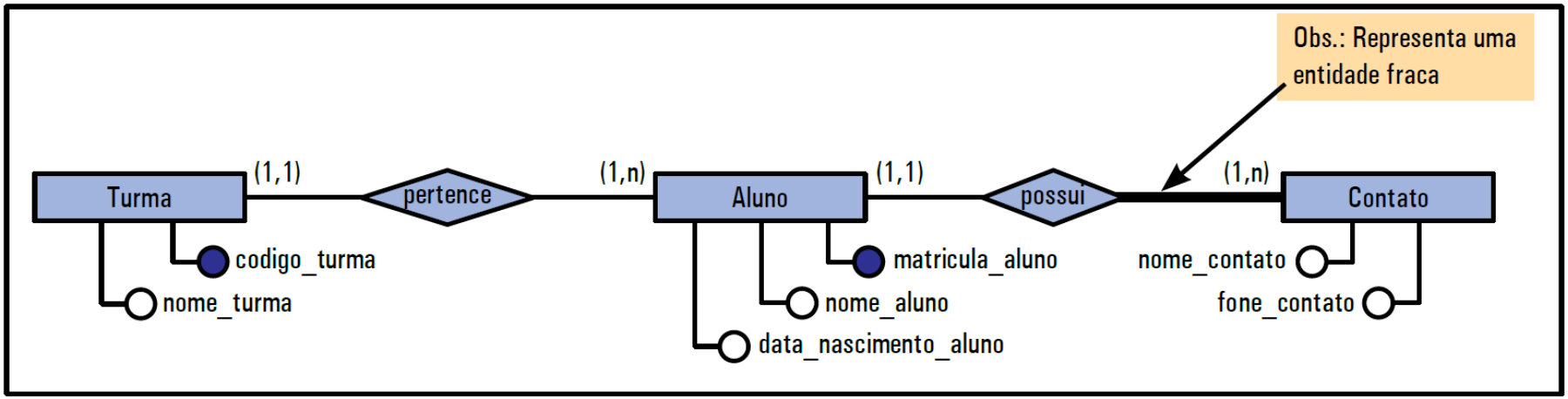
nome_cidade	nome_estado	nome_país
Curitiba	Paraná	Brasil
Maringá	Paraná	Brasil
Campo Grande	Mato Grosso do Sul	Brasil
Campo Grande	Rio de Janeiro	Brasil

É importante destacar que **não existe mais de uma chave primária** por entidade. Essa chave primária poderá ser **simples** ou **composta**. Mesmo composta, **é uma única chave primária** composta de mais de um atributo.

Entidade Fraca

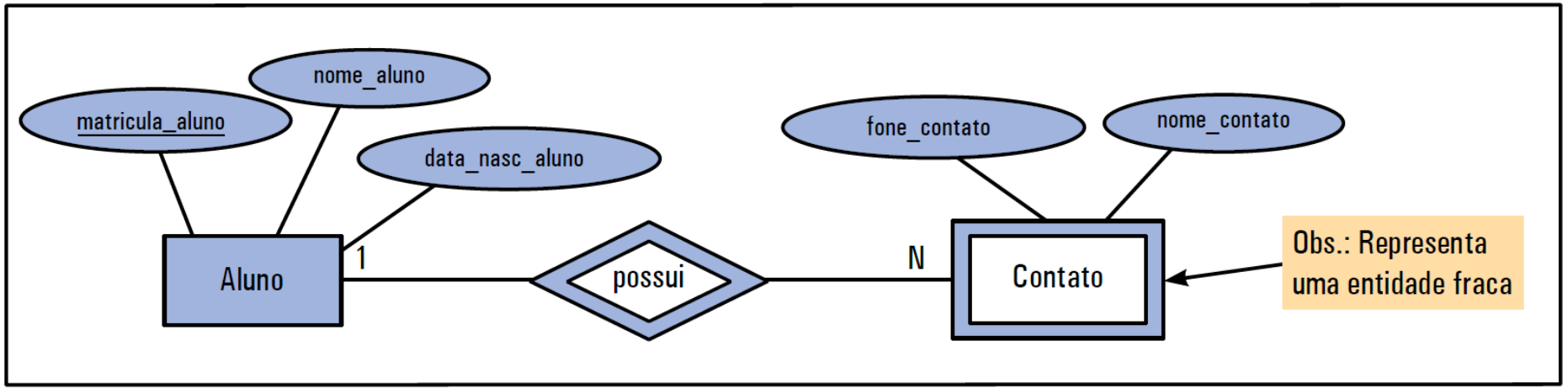
- É um tipo de entidade que **não possui atributo chave primária** por si só.
- **Não** é possível definir uma chave primária, **nem simples** e **nem composta.**
- **Depende de uma outra entidade.**
- O relacionamento entre a entidade fraca e a outra entidade é normalmente **1:N**
  - **(Lê-se 1 para N)**
  - **Cardinalidade**.

Entidade Fraca



Exemplo de uma entidade fraca

Entidade Fraca

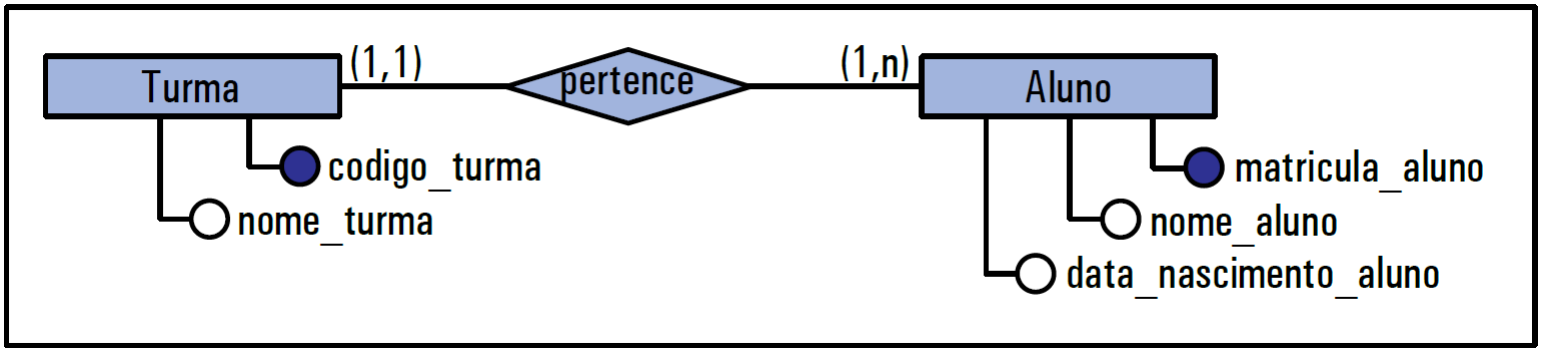


Exemplo de uma entidade fraca usando outra notação

Relacionamento, Especialização e Agregação

Relacionamento

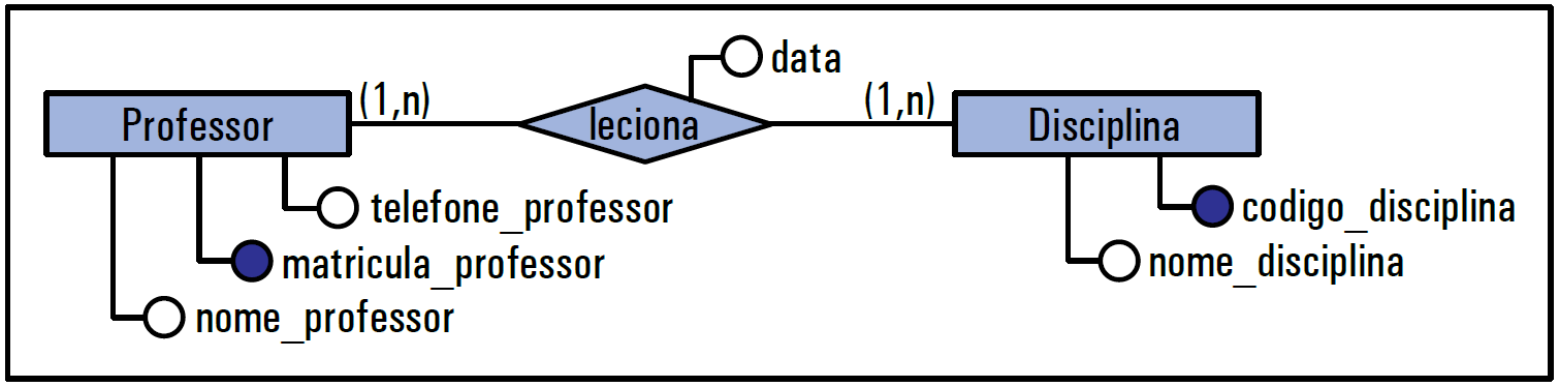
- Um **relacionamento** é uma associação entre as **entidades**.



- Representado por um **losango**.
- Pode ser um **verbo**: pertence, leciona, estuda, possui, etc.
- Composição dos nomes das entidades: Aluno_Turma.

Relacionamento

- Um **relacionamento** pode ter atributos.

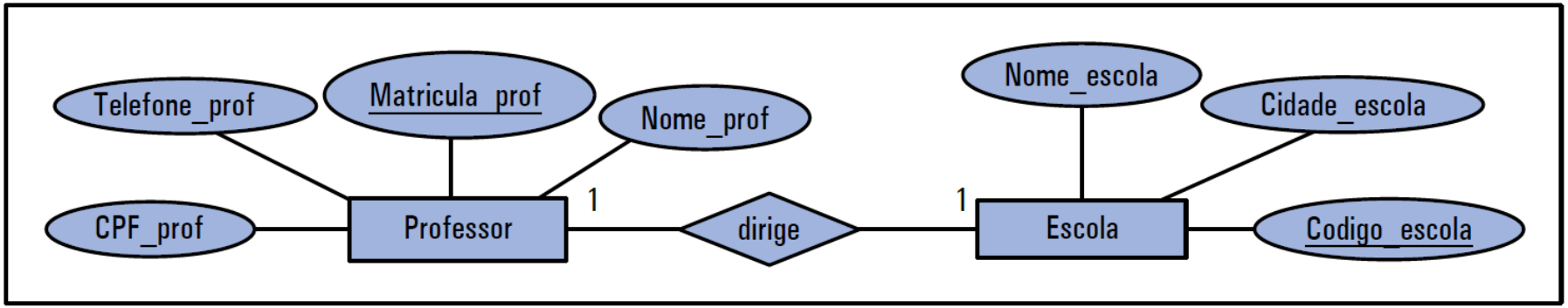


Relacionamento

- **Restrições de Mapeamento ou Cardinalidade**

- Cardinalidade Um para Um **(1:1)**

  - **um objeto de uma entidade pode se relacionar a apenas um objeto de outra entidade e vice-versa**

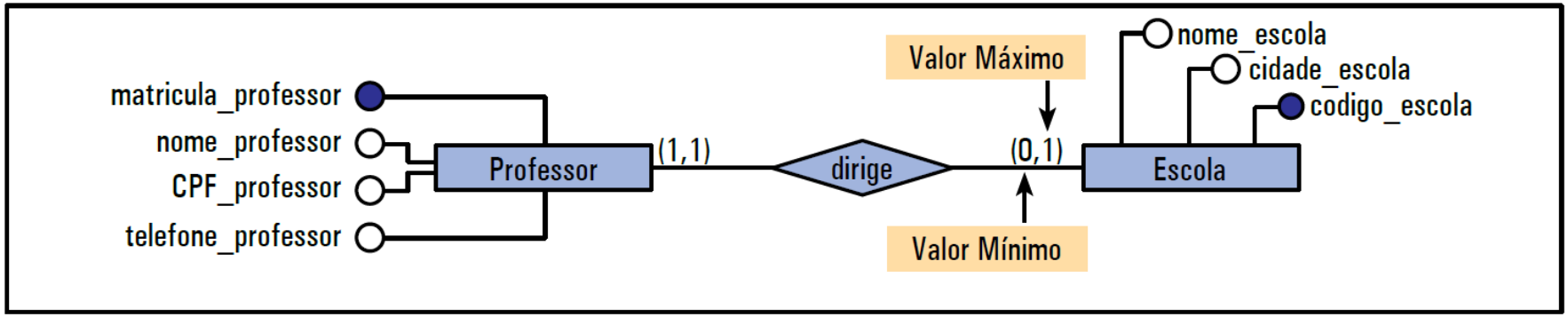


Relacionamento

- **Restrições de Mapeamento ou Cardinalidade**

brModelo

- Cardinalidade Um para Um **(1:1)**
  - **um objeto de uma entidade pode se relacionar a apenas um objeto de outra entidade e vice-versa**

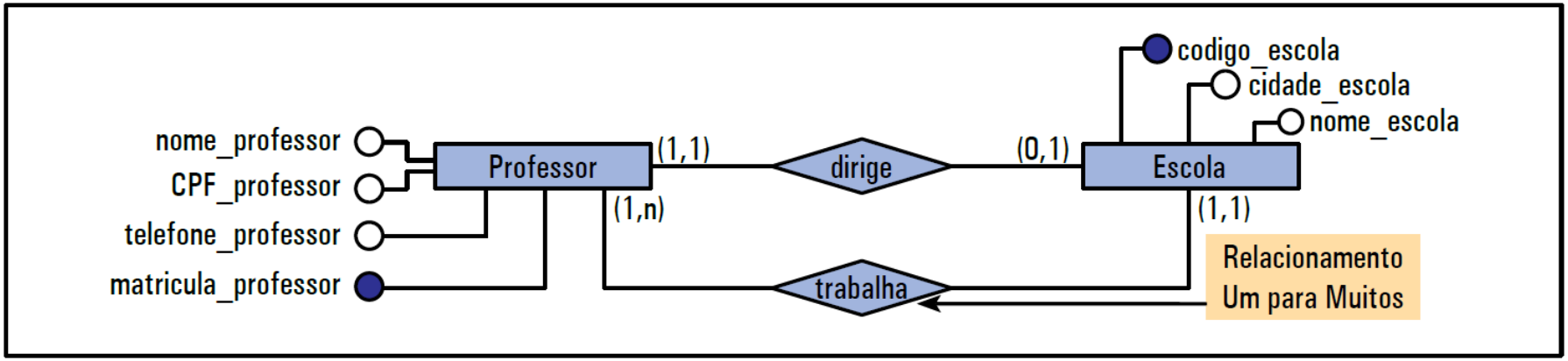


Relacionamento

- **Restrições de Mapeamento ou Cardinalidade**

- Cardinalidade Um para Muitos **(1:N)**

  - **um objeto de uma entidade pode se relacionar a vários objetos da outra entidade, mas o contrário não é verdadeiro**

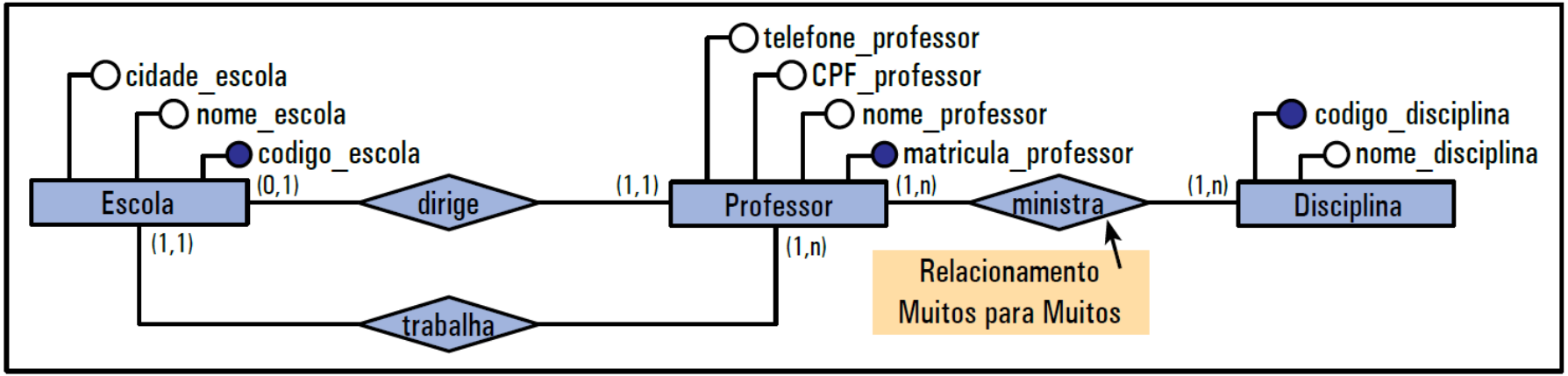


Relacionamento

- **Restrições de Mapeamento ou Cardinalidade**

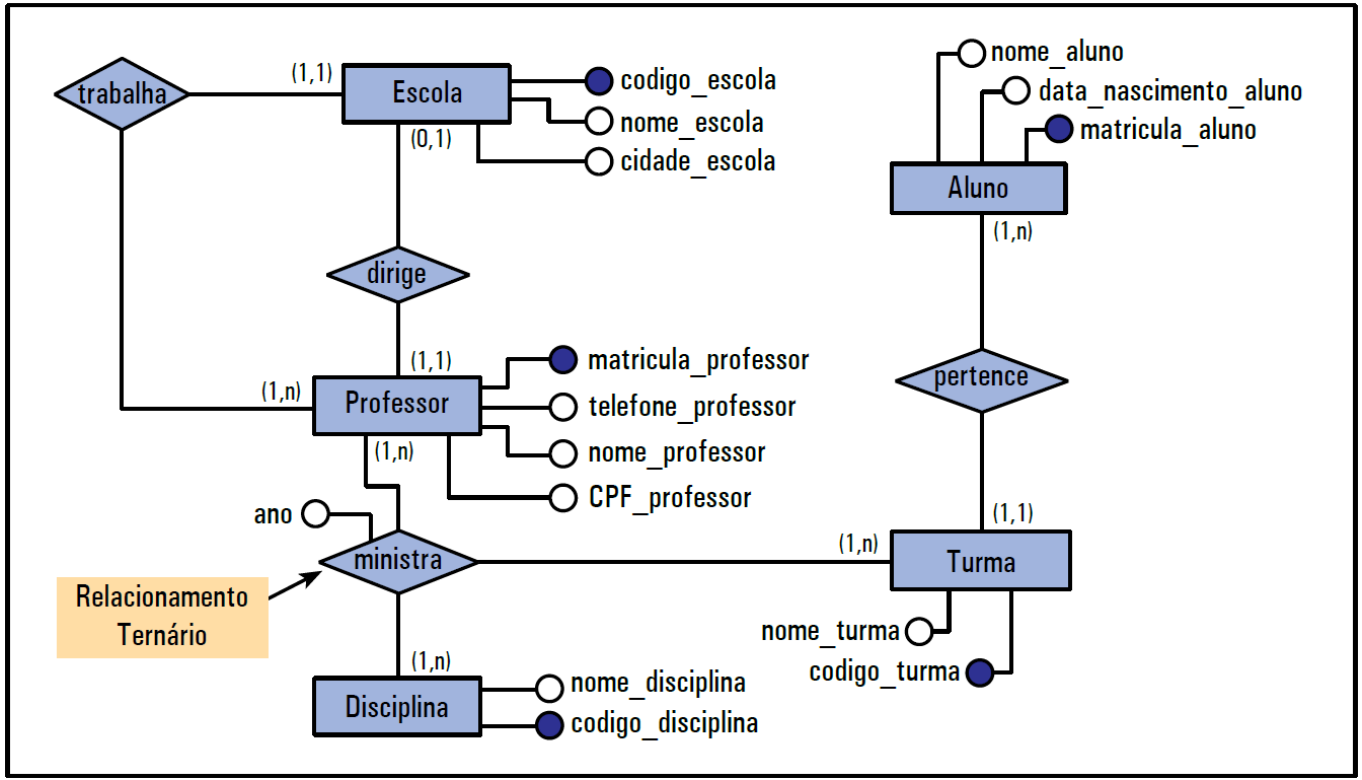
- Cardinalidade Muitos para Muitos (N:N)

  - **um objeto de uma entidade pode se relacionar a vários objetos da outra entidade e vice-versa.**



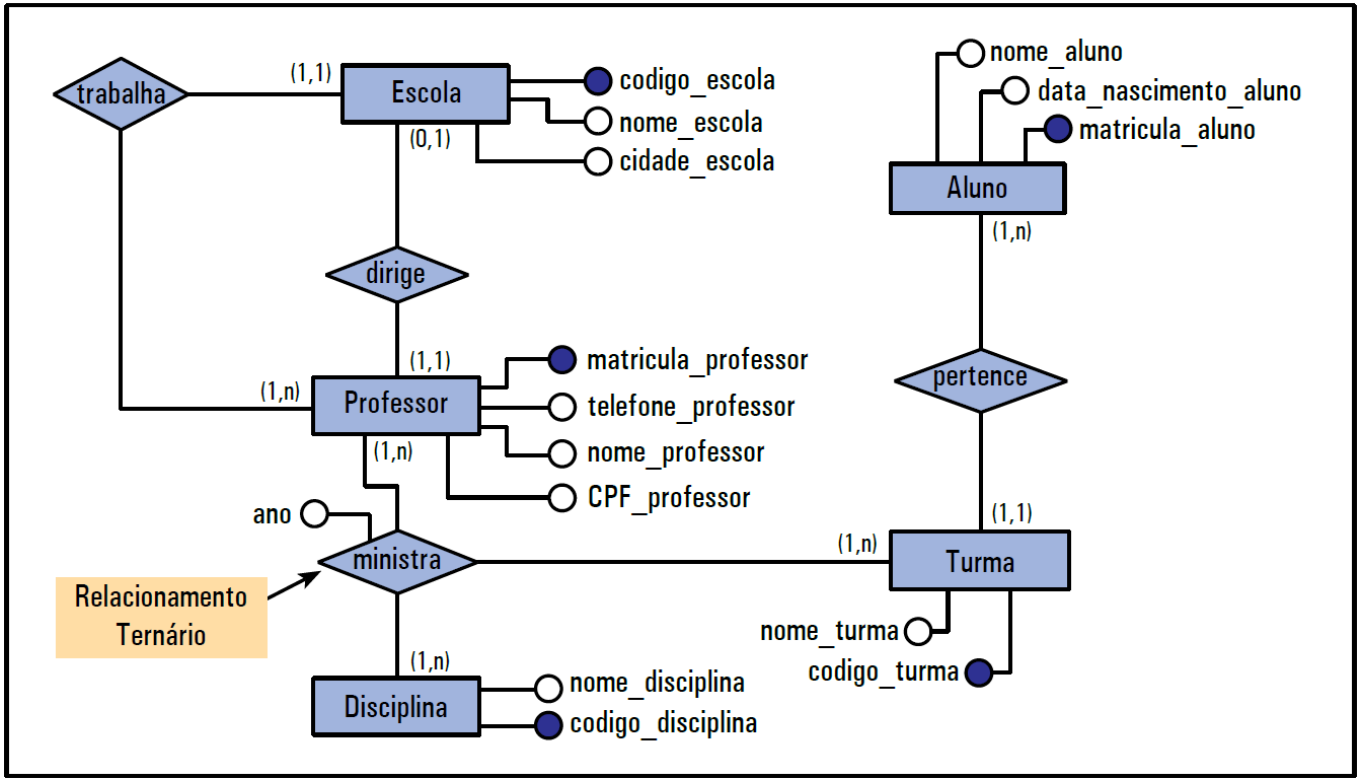
❑ Relacionamento Ternário ou Maiores

- Um relacionamento pode ocorrer também entre **três ou mais entidades**.



❑ Relacionamento Ternário ou Maiores

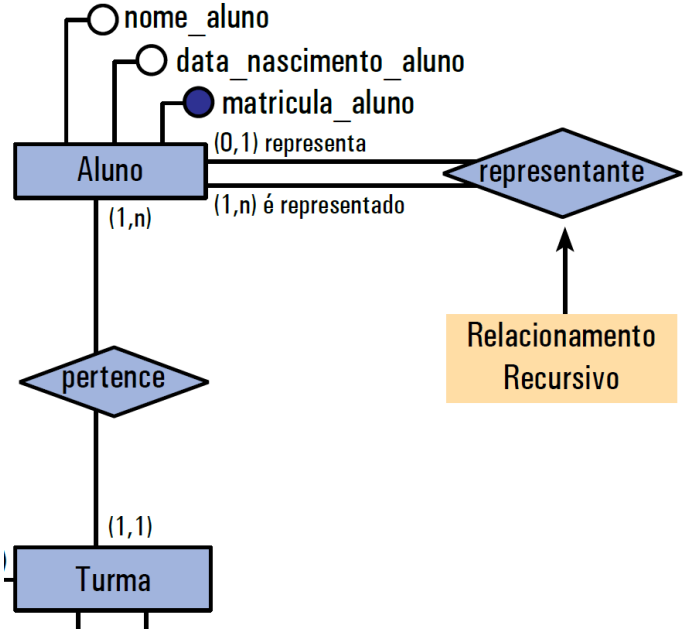
- Um relacionamento pode ocorrer também entre **três ou mais entidades**.



- Relacionamentos maiores que ternários devem ser evitados.
- São difíceis de serem compreendidos e implementados.
- A relação fica bastante complexa.

Relacionamento

- **Relacionamento Recursivo -** *Relaciona objetos de uma mesma entidade*.
  - Relacionamento recursivo ou Autorrelacionamento.

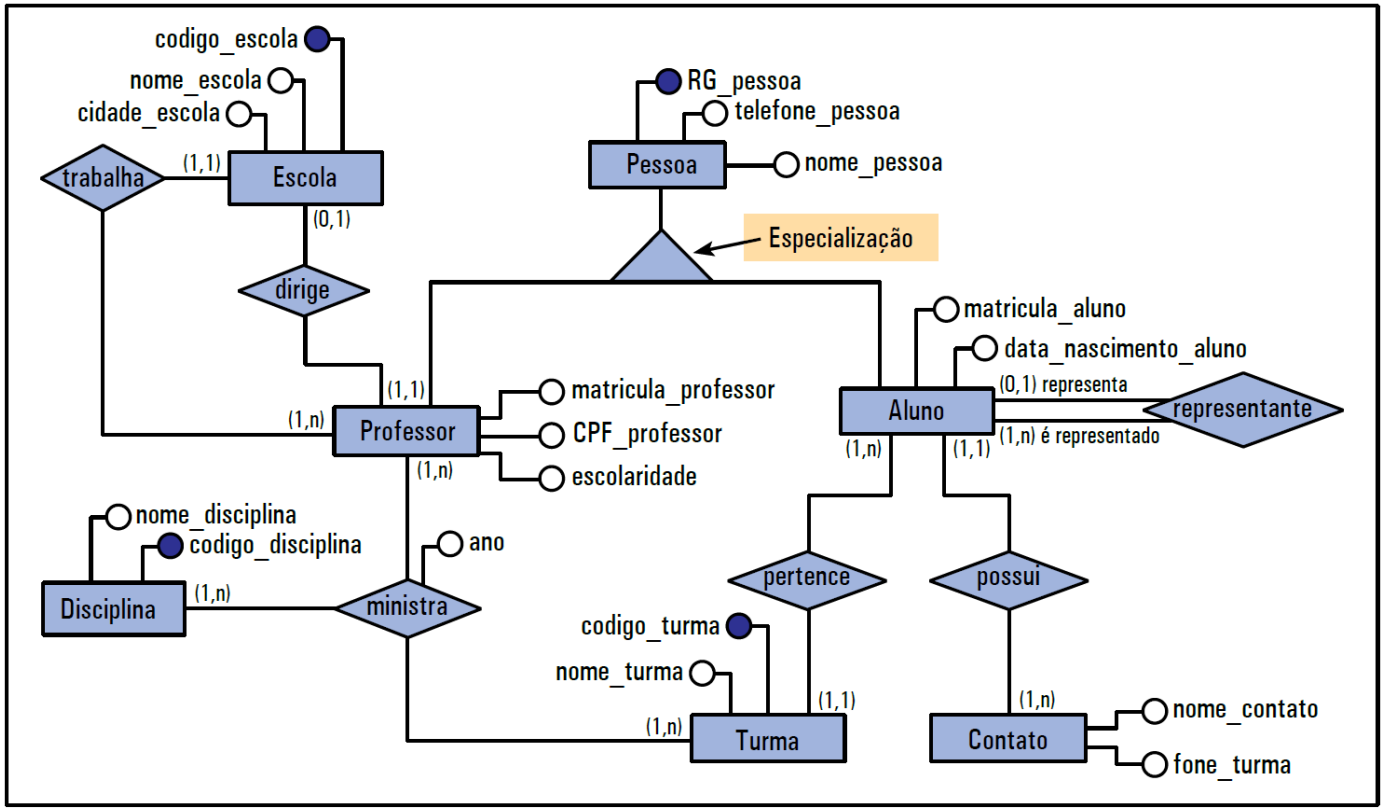


Especialização

Especialização

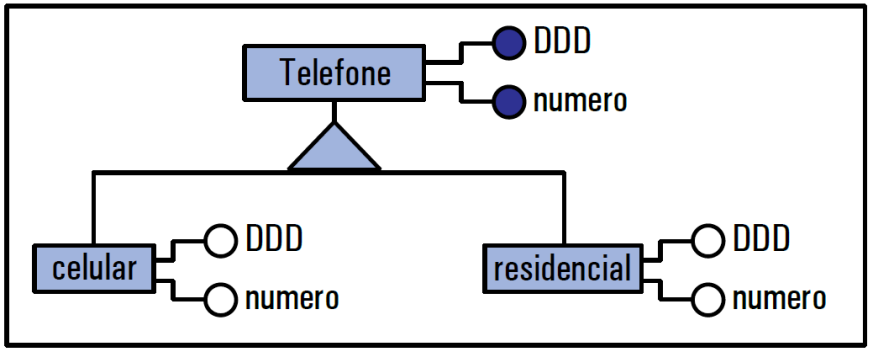
- Especialização consiste na **subdivisão** de uma entidade mais genérica.
- Entidade pai —> Entidades filhas (**entidades especializadas**).
- Um conjunto de entidades pode conter **subgrupos.**
- Cada subgrupo de entidades possui **atributos específicos.**

Especialização



Especialização

- uma especialização sem necessidade



Especialização

- A depender do padrão de MER que você está utilizando, pode haver variação em como a especialização é representada, e esta pode inclusive ser classificada em:

- **Disjoint (ou mutuamente exclusivas)**

Definição: Um registro pertence a no máximo uma das subclasses.

Exemplo:
Entidade Veículo → subclasses Carro e Moto.
Um mesmo veículo não pode ser, ao mesmo tempo, um carro e uma moto.

Representação no DER:Normalmente se usa um "d" dentro de um círculo ou anotação explícita indicando disjoint.

Especialização

- A depender do padrão de MER que você está utilizando, pode haver variação em como a especialização é representada, e esta pode inclusive ser classificada em:

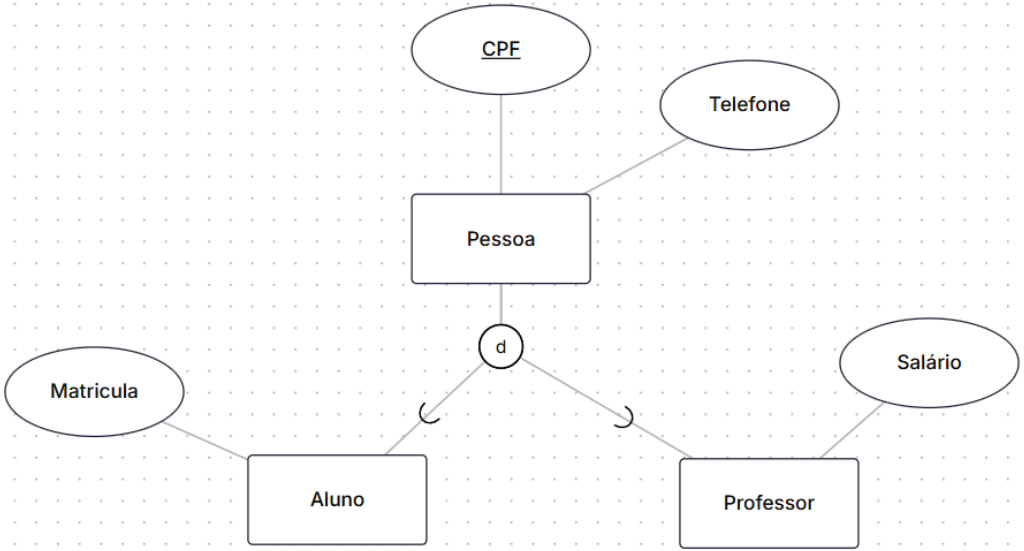
- **Overlapping (ou sobreposição)**

Definição: Um registro pode pertencer a mais de uma subclasse ao mesmo tempo.

Exemplo:
Entidade Pessoa → subclasses Professor e Pesquisador.
Uma mesma pessoa pode ser professor e pesquisador ao mesmo tempo.

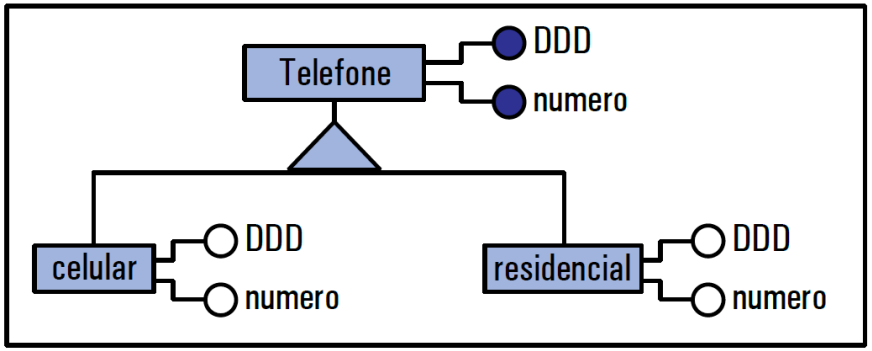
Representação no DER: Normalmente se usa um "o" no círculo ou anotação de overlapping.

Especialização

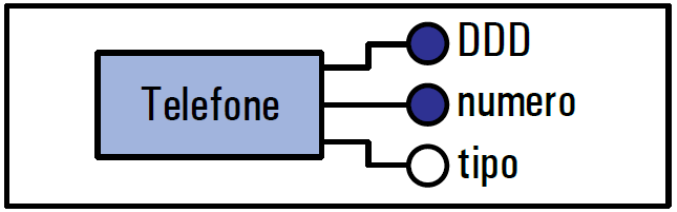


Especialização

- uma especialização sem necessidade



- Uma **generalização**

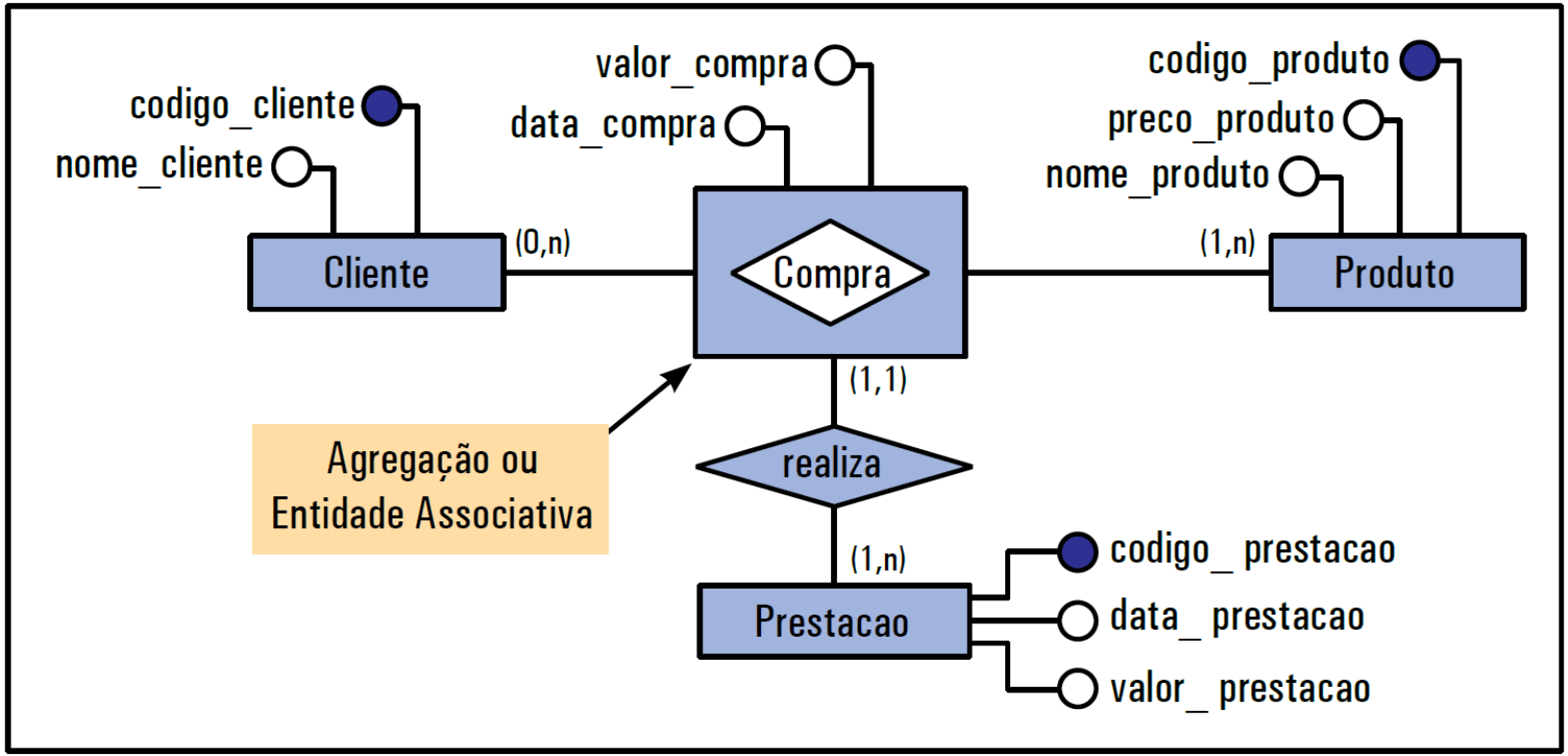


Agregação

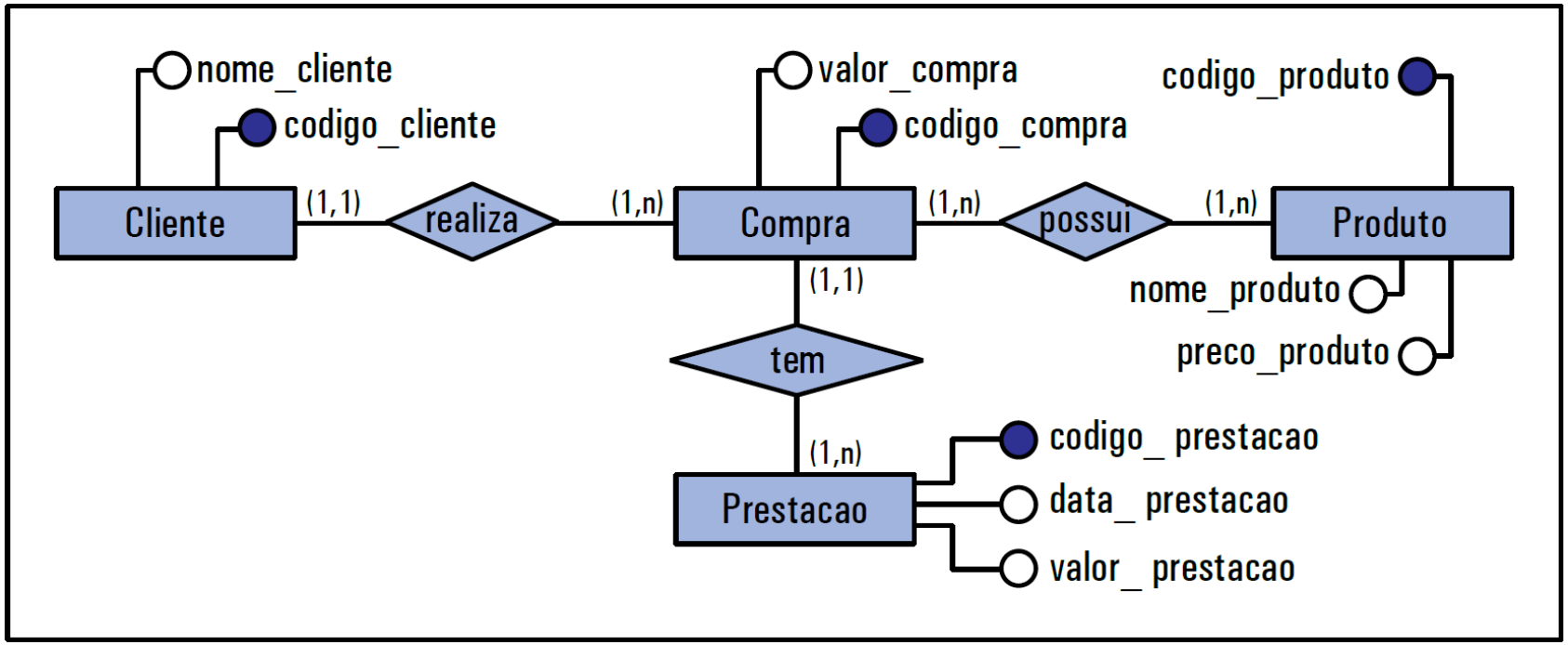
Agregação

- A **agregação**, ou entidade **associativa**, ocorre quando precisamos relacionar dois relacionamentos entre si.
- No Modelo Entidade-Relacionamento (MER), a agregação é um recurso usado quando um relacionamento inteiro precisa ser tratado como se fosse uma entidade para se relacionar com outra entidade.
- É como se você elevasse um relacionamento ao status de "entidade temporária", para poder criar novas associações a partir dele.

Agregação

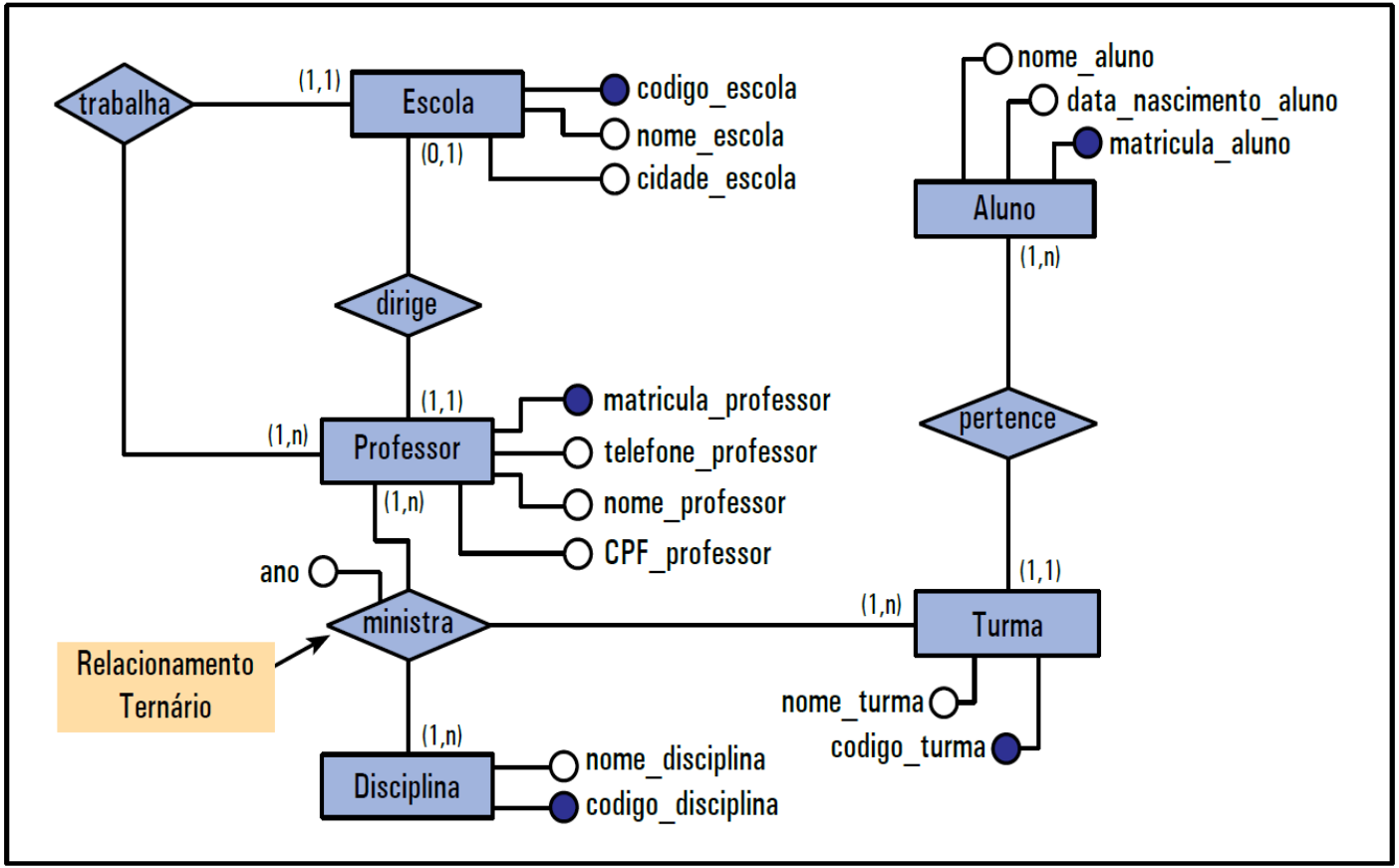

codigo_cliente
nome_cliente
Cliente
(0,n)
valor_compra
data_compra
Compra
(1,1)
realiza
(1,n)
Prestacao
codigo_produto
preco_produto
nome_produto
(1,n)
Produto
codigo_ prestacao
data_ prestacao
valor_ prestacao
Agregação ou
Entidade Associativa

Agregação



- Sem utilizar Agregação

EXERCÍCIO



- Pensando na dificuldade de implementação, modele novamente este diagrama para evitar o ternário.
- A forma mais comum é criando entidades intermediárias
- Me entregue na próxima aula

https://erdplus-v2.com/trial

https://www.brmodeloweb.com/lang/pt-br/index.html

Introdução ao Modelo Lógico/Relacional 1/2

Introdução

- **Terminado o projeto conceitual**, precisamos criar o projeto lógico do banco de dados;

- O projeto lógico descreve ***como será a estrutura do banco de dados***;

  - Mostrando as informações que serão armazenadas e como elas serão organizadas;

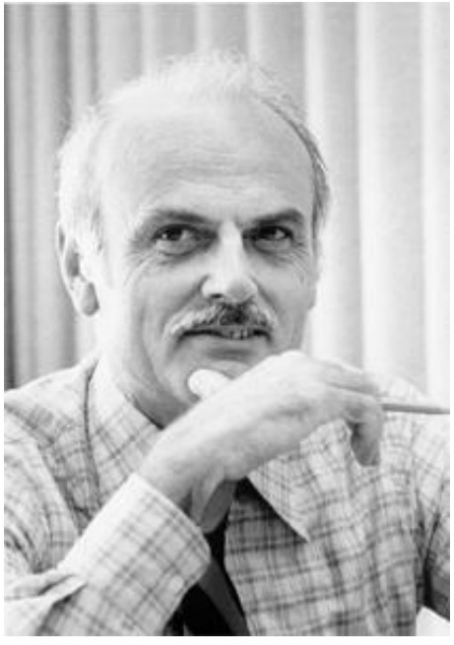
Introdução

- Diferentemente do projeto conceitual, que é voltado para a validação com o cliente, o **projeto lógico é voltado para a equipe de desenvolvimento;**

- O **administrador** do banco de dados usa o projeto lógico como guia para a implementação do banco de dados;

Introdução

- O **Modelo Relacional** foi introduzido por **Edgar Frank** Codd em 1970.
- Organiza os dados em uma estrutura de **tabelas** bidimensionais denominadas **relações**;
- Cada linha da tabela (**tupla)** representa um elemento do conjunto de dados;
- Cada coluna da tabela **(atributo)** contém valores de um conjunto definido **(domínio)**



Edgar Frank Codd
(1923-2003)

Conceitos Básicos

Tabelas

- Correspondem ao principal elemento do modelo relacional;
  - Todas as informações de um banco de dados relacional são armazenadas em **tabelas**;
  - O banco de dados é representado como uma coleção de relações (tabelas).

- Cada **tabela** representa uma **relação** e deve ter um nome distinto;

Tabelas

- Exemplo de uma tabela no modelo relacional:

Atributo

Nome	Matrícula	Cargo	Salário
João	111111	Diretor	4500,00
Maria	121212	Atendente	600,00
Sérgio	131313	Eletricista	550,00
Carlos	141414	Vigilante	400,00

Tupla?
registro completo que representa uma ocorrência específica de uma entidade.

Tabelas

- Cada tabela é um conjunto desordenado de **tuplas** exclusivas.

1. Por que desordenado?

- Em teoria de conjuntos (base do modelo relacional), conjuntos não têm ordem.
- Isso significa que, na lógica do banco de dados, a ordem física em que as linhas são armazenadas ou exibidas não importa — você define a ordem apenas no momento da consulta
- Por isso, tecnicamente, não existe "primeira" ou "última" linha a não ser que se imponha uma ordenação.

Tabelas

- Cada tabela é um conjunto desordenado de **tuplas** exclusivas.

2. Por que exclusivas?

- No modelo relacional, cada tupla (linha) deve ser única para evitar duplicidade de registros que representem exatamente o mesmo elemento do conjunto.

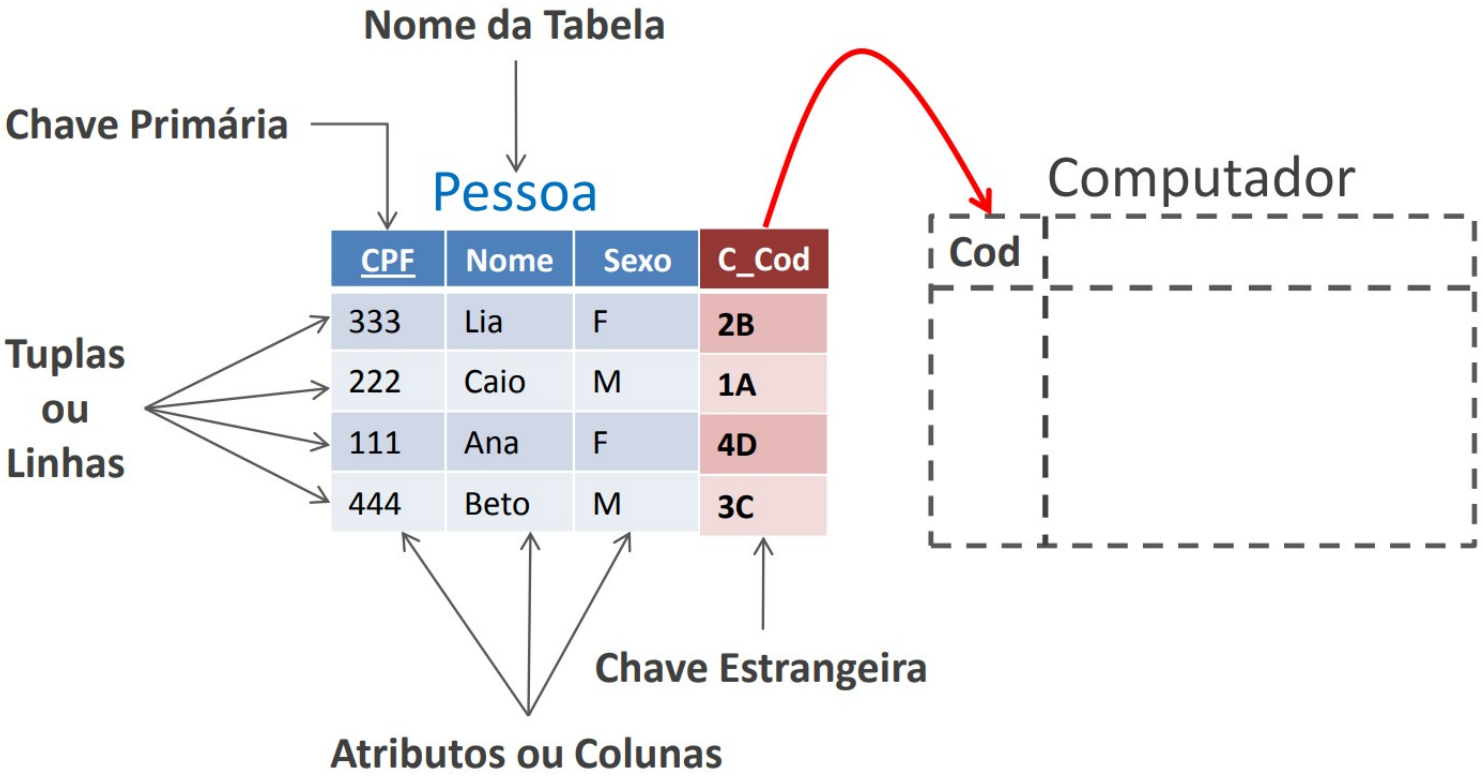
- A exclusividade das tuplas se dá por meio das **chaves primárias**

Tabelas

- Valor do campo deve ser **atômico**:
  - Simples (não é divisível); e
  - Monovalorado (só possui um valor).

- O relacionamento das tabelas se dá por meio de **Chaves estrangeiras**.

Tabelas


Nome da Tabela
Chave Primária
Pessoa
CPF
Nome
Sexo
C_Cod
333
Lia
F
2B
222
Caio
M
1A
111
Ana
F
4D
444
Beto
M
3C
Tuplas
ou
Linhas
Computador
Cod
Chave Estrangeira
Atributos ou Colunas

Chaves

- candidatas;
- primárias;
- alternativas ou alternadas; e
- estrangeiras.

Chave Primária

- Coluna ou combinação de colunas cujos valores distinguem uma linha das demais dentro de uma tabela.

- Propriedades:
  - Única;
  - Campos obrigatórios (not null); e
  - Irredutível → Um conjunto de atributos é irredutível quando não é possível remover nenhum atributo dele sem perder a capacidade de identificar unicamente uma tupla.

Chave Primária

Chave Primária Simples

Pessoa

CPF	Nome	Sexo
333	Lia	F
222	Caio	M
111	Ana	F
444	Beto	M

Chave Primária Composta

Estante

Corredor	Prateleira	Volume
A	1	10
A	2	20
B	1	20
B	2	15
C	1	10
C	2	30

Chave Primária

- Candidatas → “uma possível chave primária”
- Primárias; e
- Alternativas ou alternadas.

Pessoa

CPF	Matricula	Nome	Sexo
333	2B	Lia	F
222	1A	Caio	M
111	3C	Ana	F
444	5E	Beto	M

Uma tabela pode ter várias chaves candidatas. Entre elas, uma é escolhida como chave primária; as outras ficam como chaves alternativas.

Chave Primária

- Candidatas;
- Primárias; e
- Alternativas ou alternadas.

Pessoa

CPF	Matricula	Nome	Sexo
333	2B	Lia	F
222	1A	Caio	M
111	3C	Ana	F
444	5E	Beto	M

Possibilidade 1

- Chaves candidatas:
  - CPF e Matricula.
- Chave Primária:
  - CPF.
- Chave Alternativa ou alternada:
  - Matrícula.

Possibilidade 2

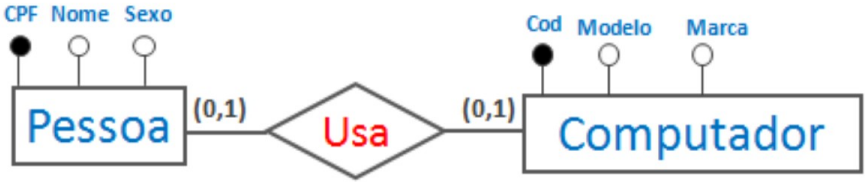
- Chaves candidatas:
  - CPF e Matricula.
- Chave Primária:
  - Matrícula.
- Chave Alternativa ou alternada:
  - CPF.

Chave Estrangeira

- Coluna ou combinação de colunas, cujos valores aparecem na chave primária (candidata) da tabela referenciada.

- Mecanismo que permite a implementação de relacionamentos em um banco de dados relacional.

Conceitual

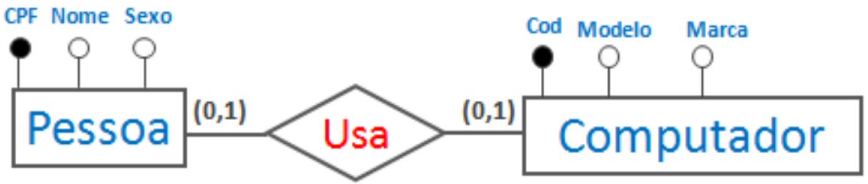


Chave Estrangeira

- Coluna ou combinação de colunas, cujos valores aparecem na chave primária (candidata) da tabela referenciada.

- Mecanismo que permite a implementação de relacionamentos em um banco de dados relacional.

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Computador

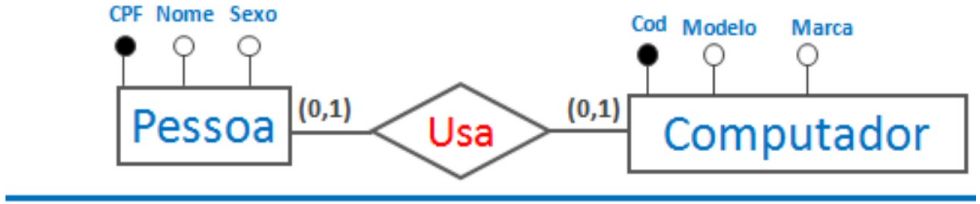
Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

Chave Estrangeira

- Coluna ou combinação de colunas, cujos valores aparecem na chave primária (candidata) da tabela referenciada.

- Mecanismo que permite a implementação de relacionamentos em um banco de dados relacional.

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo	C_Cod
111	Ana	F	1A
222	Beto	M	2B
333	Lia	F	3C
444	Caio	M	NULL

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

Restrições de Integridade

Restrições de Integridade

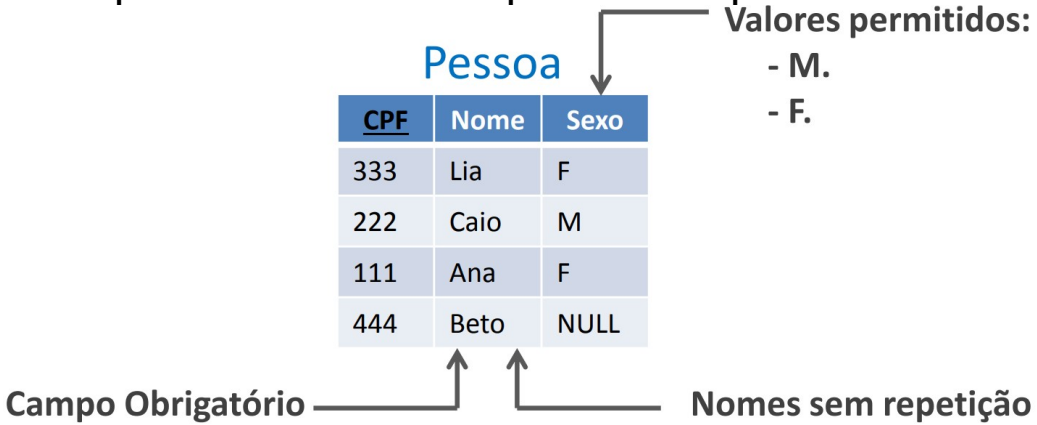
- É uma regra de consistência de dados que é garantida pelo próprio SGBD.

- Restrições de Integridade Básicas;
  - Integridade de Domínio;
  - Integridade de Vazio;
  - Integridade de Chave;
  - Integridade Referencial;
  - Integridade de Unicidade;
  - Integridade de Entidade.

- São garantidas automaticamente pelo SGBD Relacional.
- O programador não precisa implementá-las.

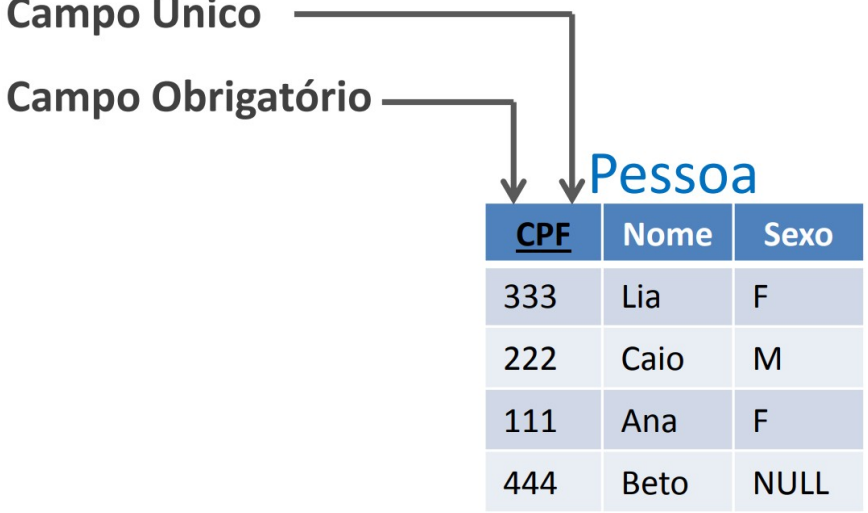
Restrições de Integridade

- **Integridade de Domínio:**
  - Define os valores que podem ser assumidos pelos campos de uma coluna.
- **Integridade de Vazio:**
  - Especifica se os campos de uma coluna podem ou não ser vazios.
- **Integridade de Unicidade:**
  - Define que o valor do campo ou campos são únicos.



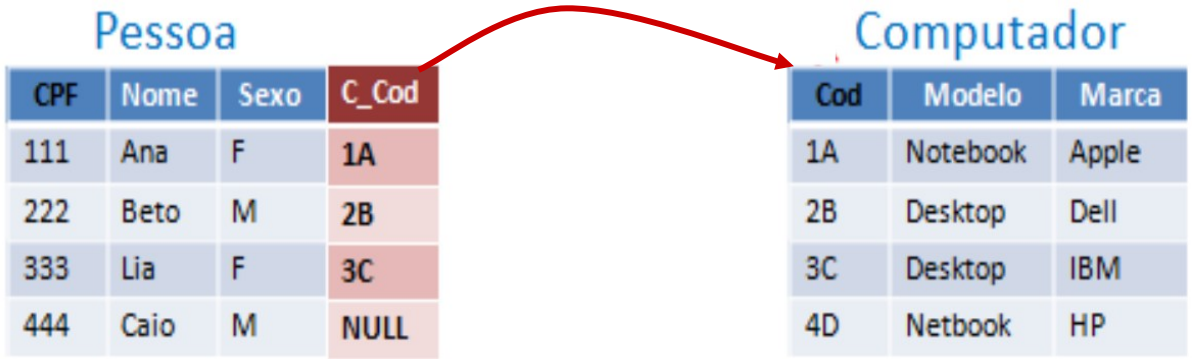
Restrições de Integridade

- **Integridade de Chave:**
  - Define que os valores da chave primária e alternativa devem ser únicos.
- **Integridade de Entidade:**
  - Define que nenhum valor da chave primária pode ser nulo (obrigató



Restrições de Integridade

- **Integridade Referencial:**
  - Define que os valores dos campos que aparecem numa chave estrangeira devem aparecer na chave primária (candidata) da tabela referenciada.



Pessoa

CPF	Nome	Sexo	C_Cod
111	Ana	F	1A
222	Beto	M	2B
333	Lia	F	3C
444	Caio	M	NULL

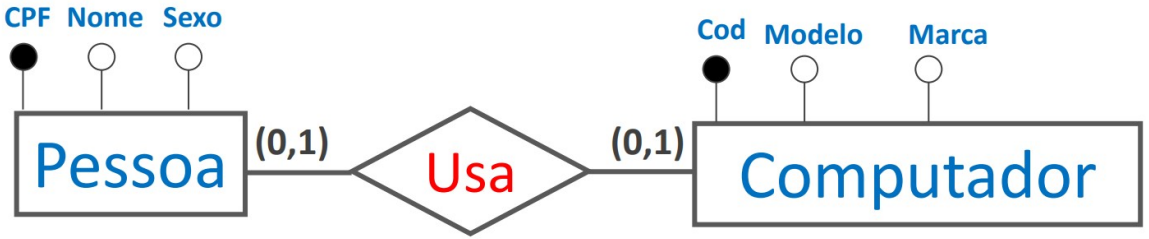
Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

Modelo Relacional
Cardinalidade 1:1

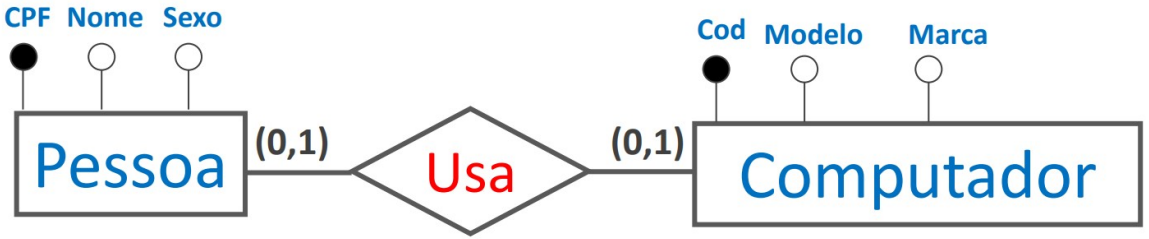
Cardinalidade 1:1

Conceitual



Cardinalidade 1:1

Conceitual



Lógico

Pessoa

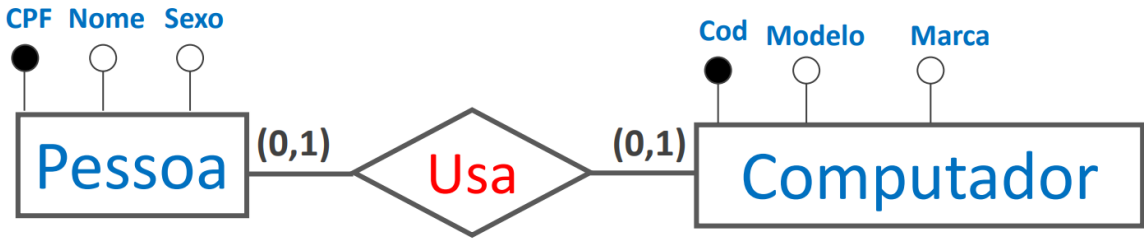
CPF	Nome	Sexo	C_Cod
111	Ana	F	**1A**
222	Beto	M	**2B**
333	Lia	F	**3C**
444	Caio	M	**NULL**

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

Cardinalidade 1:1

Conceitual



Lógico

Pessoa

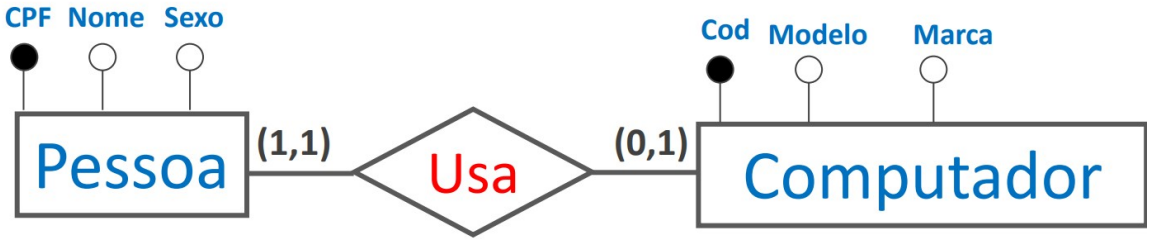
CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Computador

Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	**111**
2B	Desktop	Dell	**222**
3C	Desktop	IBM	**333**
4D	Netbook	HP	**NULL**

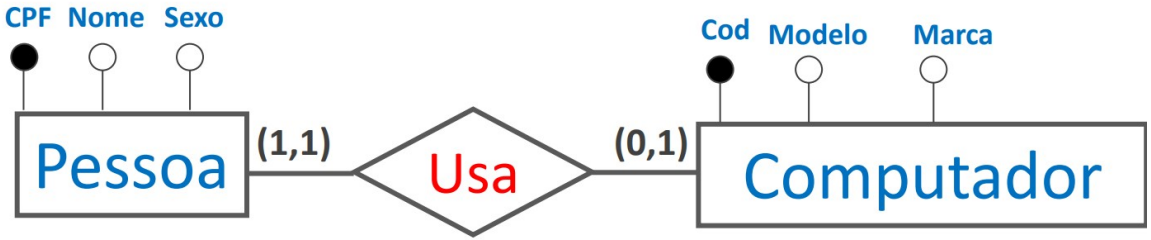
Cardinalidade 1:1

Conceitual



Cardinalidade 1:1

Conceitual



Lógico

Pessoa

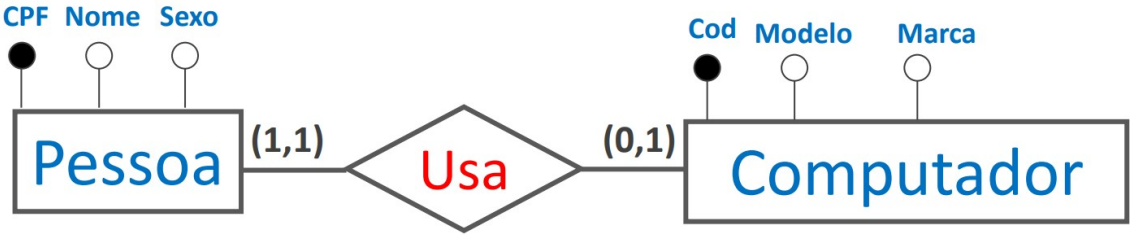
CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Computador

Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	111
2B	Desktop	Dell	222
3C	Desktop	IBM	333

Cardinalidade 1:1

Conceitual



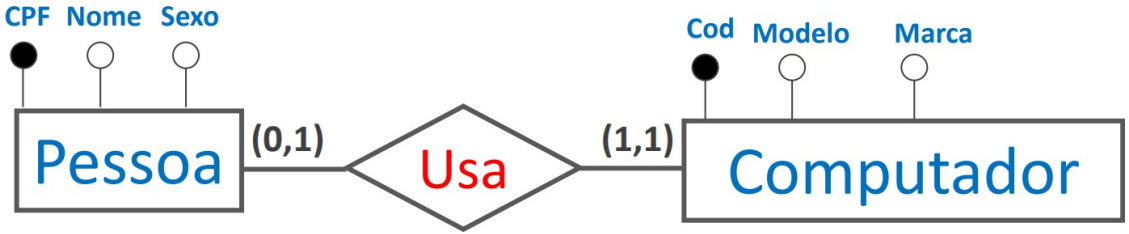
Lógico

Pessoa_Computador

CPF	Nome	Sexo	Cod	Modelo	Marca
111	Ana	F	1A	Notebook	Apple
222	Beto	M	2B	Desktop	Dell
333	Lia	F	3C	Desktop	IBM
444	Caio	M	NULL	NULL	NULL

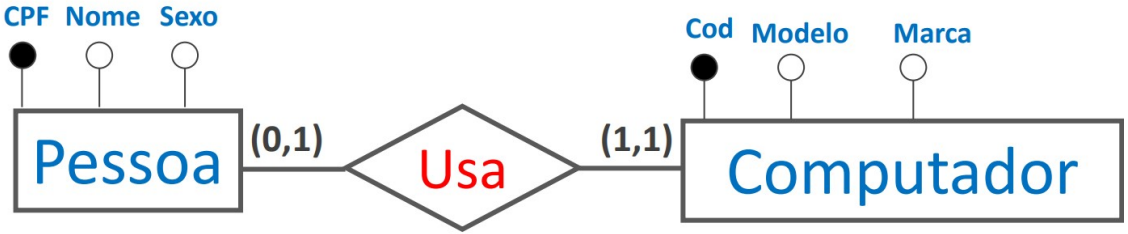
Cardinalidade 1:1

Conceitual



Cardinalidade 1:1

Conceitual



Lógico

Pessoa

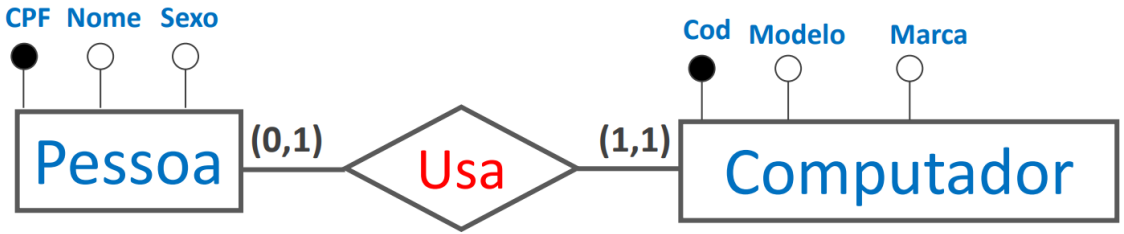
CPF	Nome	Sexo	C_Cod
111	Ana	F	**1A**
222	Beto	M	**2B**
333	Lia	F	**3C**

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

Cardinalidade 1:1

Conceitual



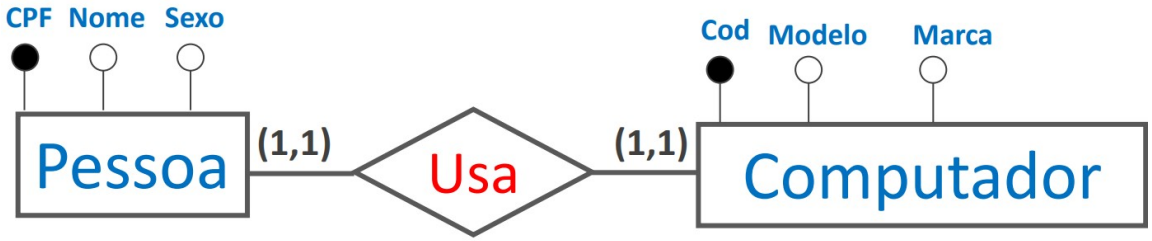
Lógico

Pessoa_Computador

Cod	Modelo	Marca	CPF	Nome	Sexo
1A	Note	Notebook	111	Ana	F
2B	Desk	Desktop	222	Beto	M
3C	Desk	Desktop	333	Lia	F
4D	Net	Netbook	NULL	NULL	NULL

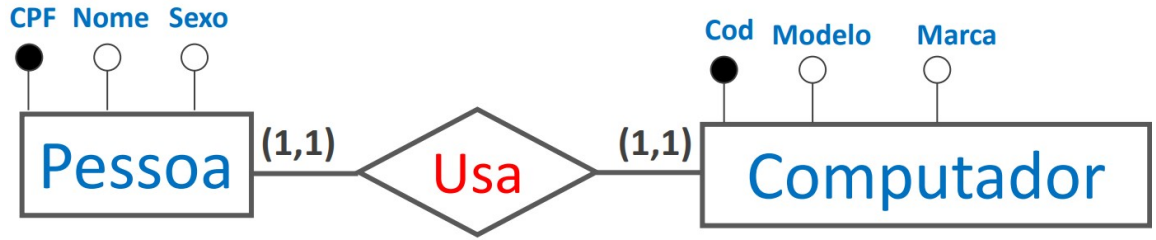
Cardinalidade 1:1

Conceitual



Cardinalidade 1:1

Conceitual



Lógico

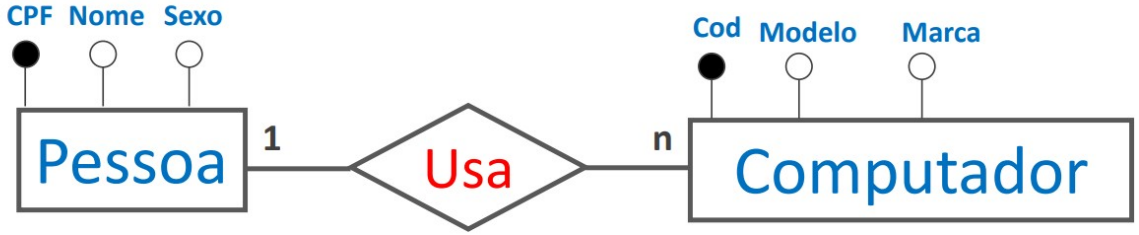
Pessoa_Computador

CPF	Nome	Sexo	Cod	Modelo	Marca
111	Ana	F	1A	Notebook	Apple
222	Beto	M	2B	Desktop	Dell
333	Lia	F	3C	Desktop	IBM
444	Caio	M	4D	Netbook	HP

Modelo Relacional
Cardinalidade 1:N

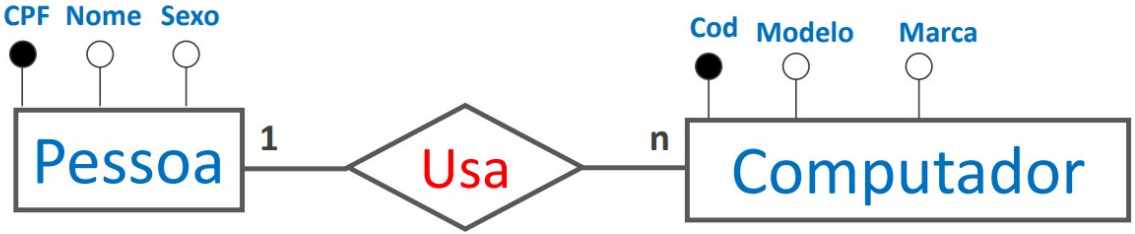
Cardinalidade 1:N

Conceitual



Cardinalidade 1:N

Conceitual



Lógico

Pessoa

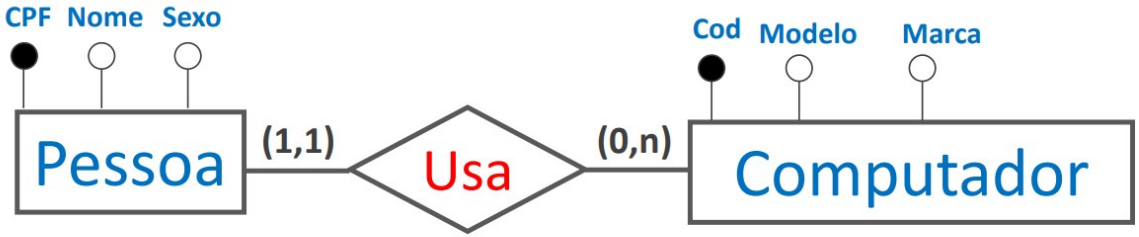
CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Computador

Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	111
2B	Desktop	Dell	111
3C	Desktop	IBM	333
4D	Netbook	HP	NULL

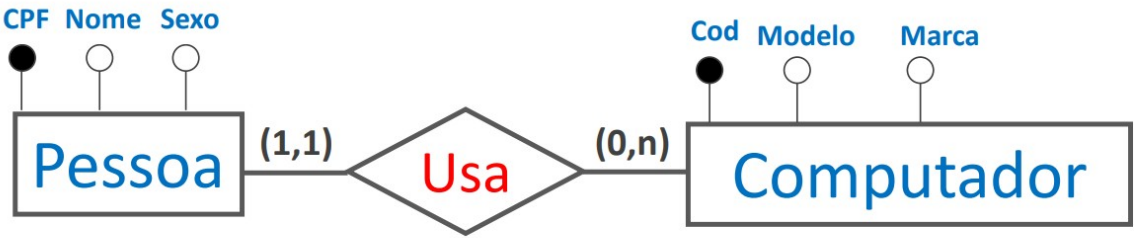
Cardinalidade 1:N

Conceitual



Cardinalidade 1:N

Conceitual



Lógico

Pessoa

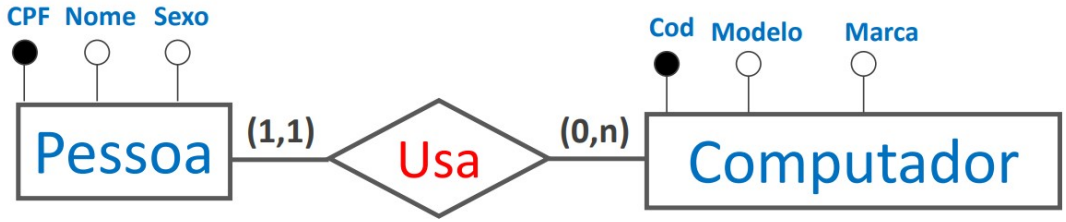
CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Computador

Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	111
2B	Desktop	Dell	111
3C	Desktop	IBM	333
4D	Netbook	HP	NULL

Cardinalidade 1:N

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Computador

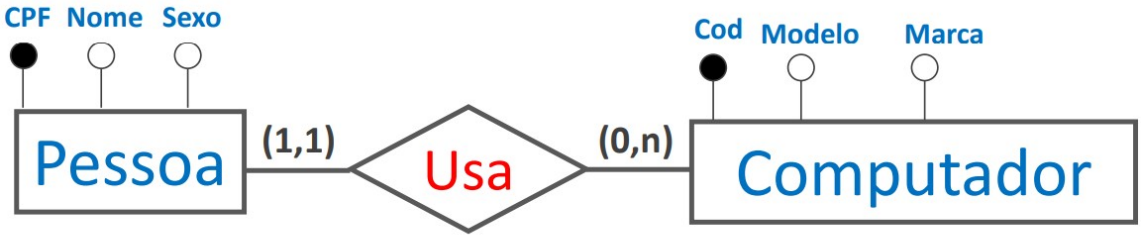
Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	111
2B	Desktop	Dell	111
3C	Desktop	IBM	333
4D	Netbook	HP	888

?

Essa imagem ilustra um exemplo de violação da integridade referencial no modelo lógico.

Cardinalidade 1:N

Conceitual



Lógico

Pessoa

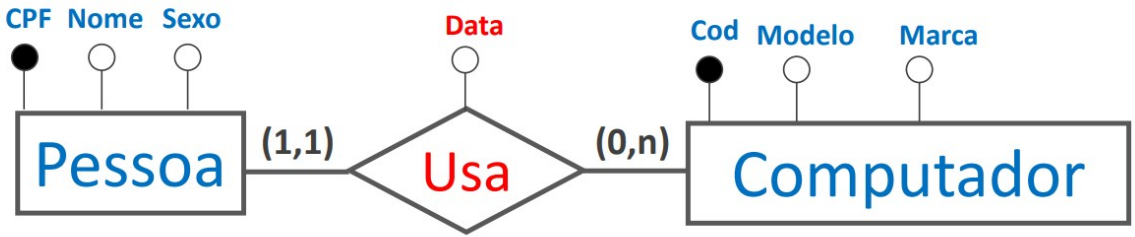
CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Computador

Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	111
2B	Desktop	Dell	111
3C	Desktop	IBM	333
4D	Netbook	HP	222

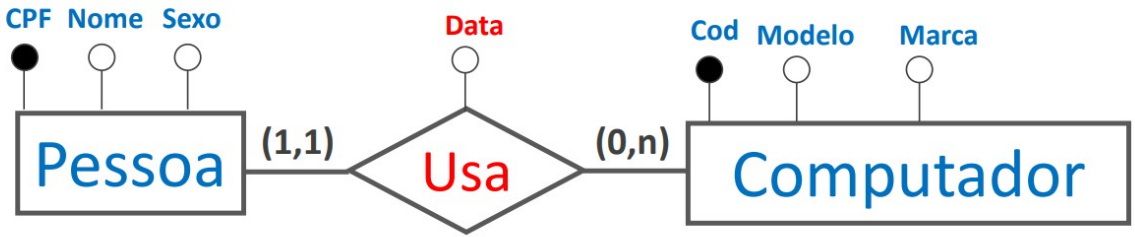
Cardinalidade 1:N

Conceitual



Cardinalidade 1:N

Conceitual



Lógico

Pessoa

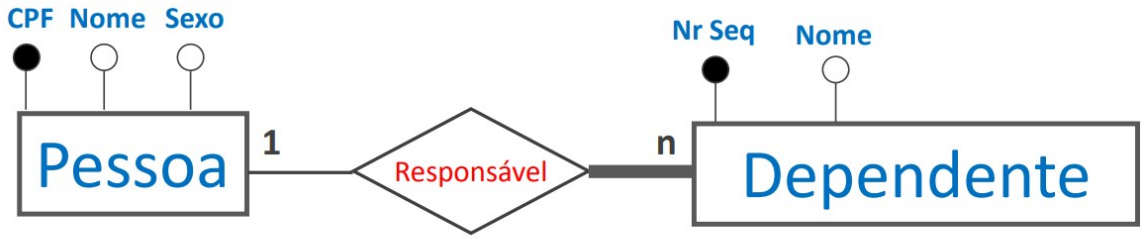
CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Computador

Cod	Modelo	Marca	P_CPF	Data
1A	Notebook	Apple	111	01-Jan-2011
2B	Desktop	Dell	111	02-Fev-2012
3C	Desktop	IBM	333	03-Mar-2013
4D	Netbook	HP	222	04-Abr-2014

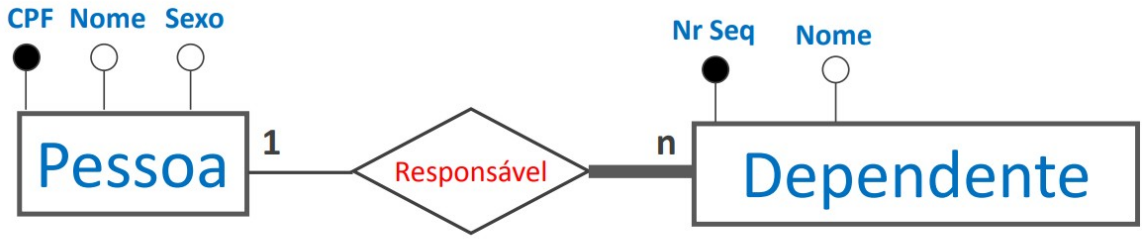
Relacionamento Identificador

Conceitual



Relacionamento Identificador

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

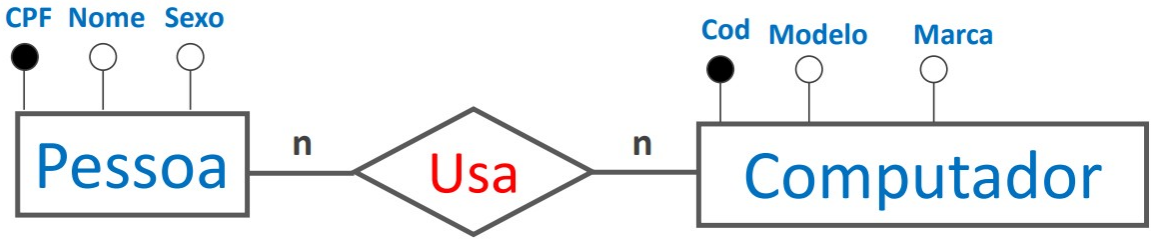
Dependente

Cod	Nome	P_CPF
1	Igor	111
2	Maria	111
1	André	333
1	João	222

Modelo Relacional
Cardinalidade N:N

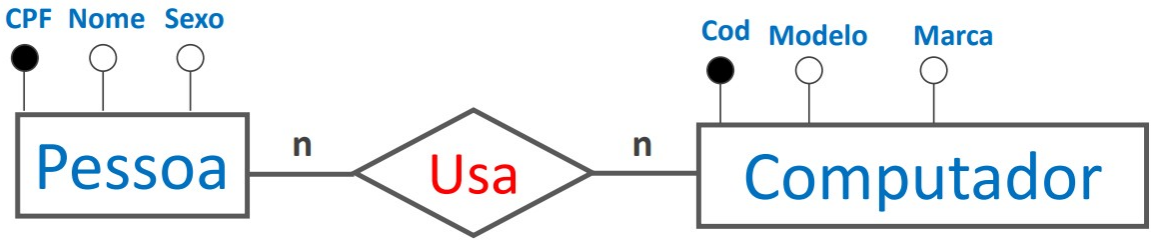
Cardinalidade N:N

Conceitual



Cardinalidade N:N

Conceitual



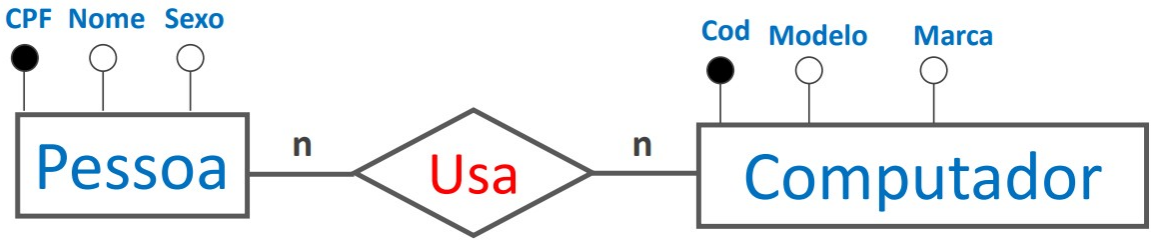
Lógico

Pessoa

CPF	Nome	Sexo	C_Cod
111	Ana	F	1A, 2B
222	Beto	M	2B
333	Lia	F	3C
444	Caio	M	NULL

Cardinalidade N:N

Conceitual



Lógico

Pessoa

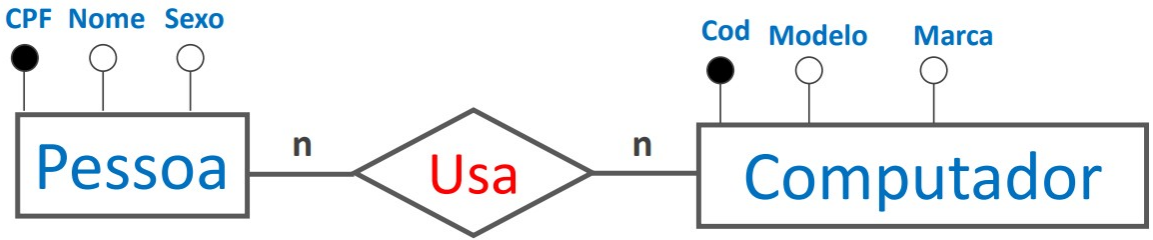
CPF	Nome	Sexo	C_Cod
111	Ana	F	1A, 2B
222	Beto	M	2B
333	Lia	F	3C
444	Caio	M	NULL

Computador

Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	111
2B	Desktop	Dell	111, 222
3C	Desktop	IBM	333
4D	Netbook	HP	NULL

Cardinalidade N:N

Conceitual



Lógico

Pessoa

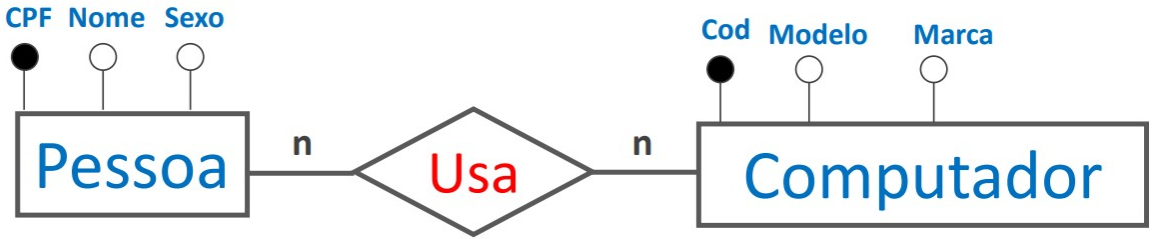
CPF	Nome	Sexo	C_Cod
111	Ana	F	1A, 2B
222	Beto	M	2B
333	Lia	F	3C
444	Caio	M	NULL

Computador

Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	111
2B	Desktop	Dell	111, 222
3C	Desktop	IBM	333
4D	Netbook	HP	NULL

Cardinalidade N:N

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Usa

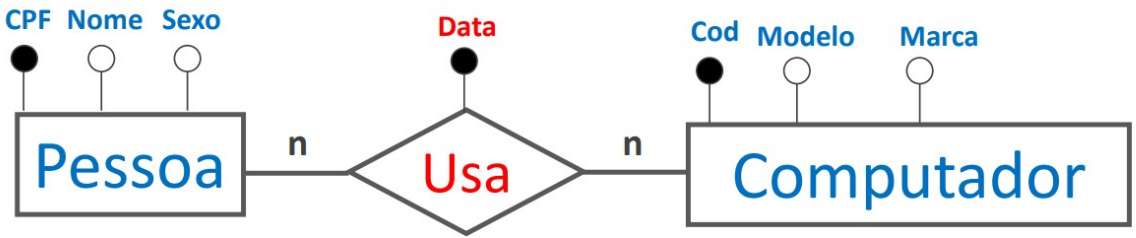
P_CPF	C_Cod
111	1A
111	2B
222	2B
333	3C

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

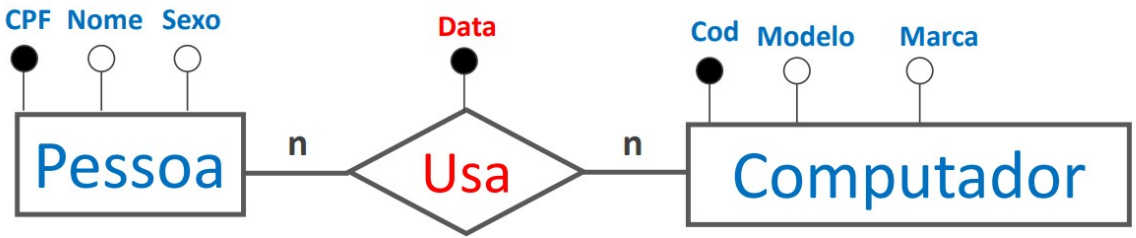
Cardinalidade N:N

Conceitual



Cardinalidade N:N

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Usa

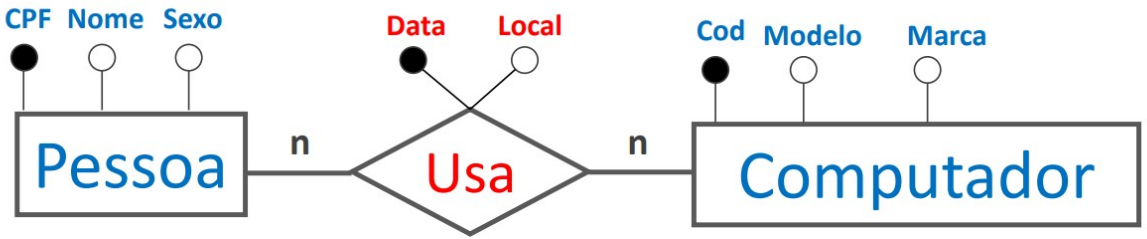
P_CPF	C_Cod	Data
111	1A	01-Jan-2011
111	2B	02-Fev-2012
222	2B	03-Mar-2013
333	3C	01-Abr-2014

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

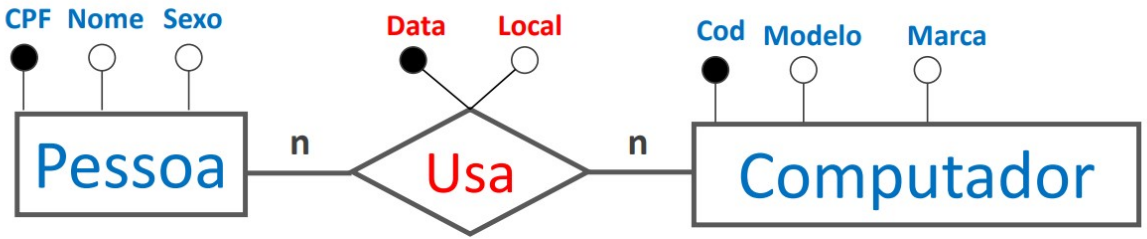
Cardinalidade N:N

Conceitual



Cardinalidade N:N

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Usa

P_CPF	C_Cod	Data	Local
111	1A	01-Jan-2011	RJ
111	2B	02-Fev-2012	SP
222	2B	03-Mar-2013	DF
333	3C	01-Abr-2014	MG

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

Continua na próxima aula...



UniRV
Universidade de Rio Verde

Fasoft
Faculdade de Engenharia de Software

Introdução ao Modelo Lógico 2/2

Auto-Relacionamento

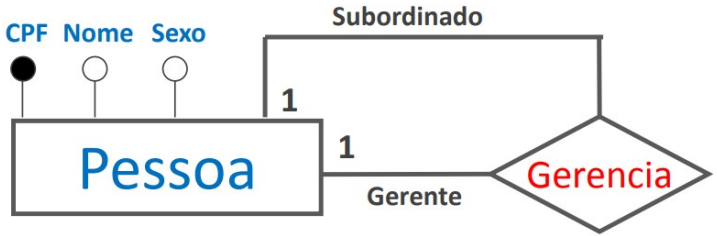
Auto-Relacionamento

- Normalmente um relacionamento associa entidades diferentes. Há um caso especial no qual **há um relacionamento entre a mesma entidade**.

- Nesta situação surge o conceito de **PAPEL** que identifica o relacionamento.

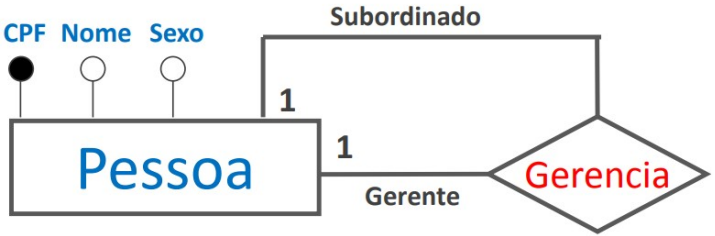
Cardinalidade 1:1

Conceitual



Cardinalidade 1:1

Conceitual



Lógico

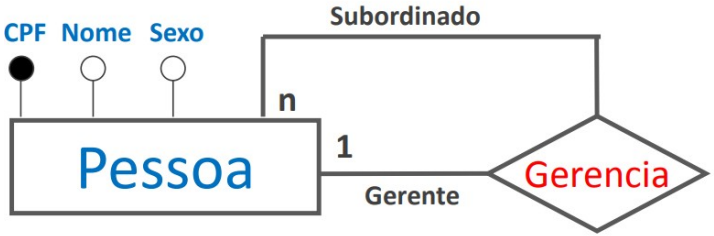
Pessoa

CPF	Nome	Sexo	P_CPF_Ger
111	Ana	F	
222	Beto	M	111
333	Lia	F	222
444	Caio	M	333

Sem repetição (único)

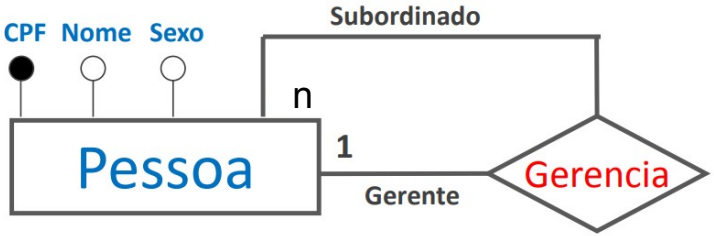
Cardinalidade 1:N

Conceitual



Cardinalidade N:N

Conceitual



Lógico

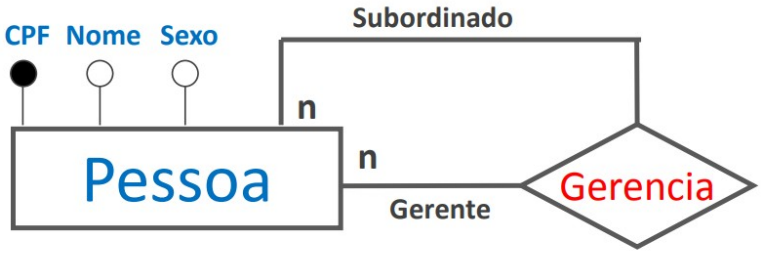
Pessoa

CPF	Nome	Sexo	P_CPF_Ger
111	Ana	F	
222	Beto	M	111
333	Lia	F	111
444	Caio	M	333

Pode possuir repetições

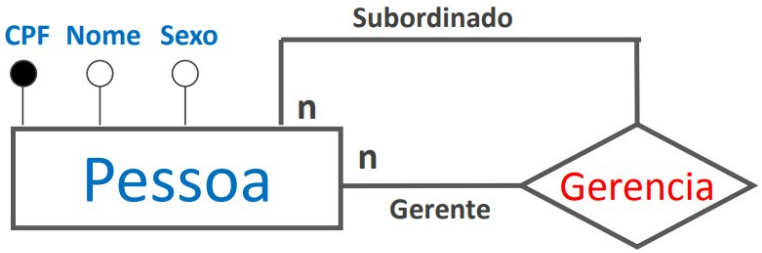
Cardinalidade N:N

Conceitual



Cardinalidade N:N

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Gerencia

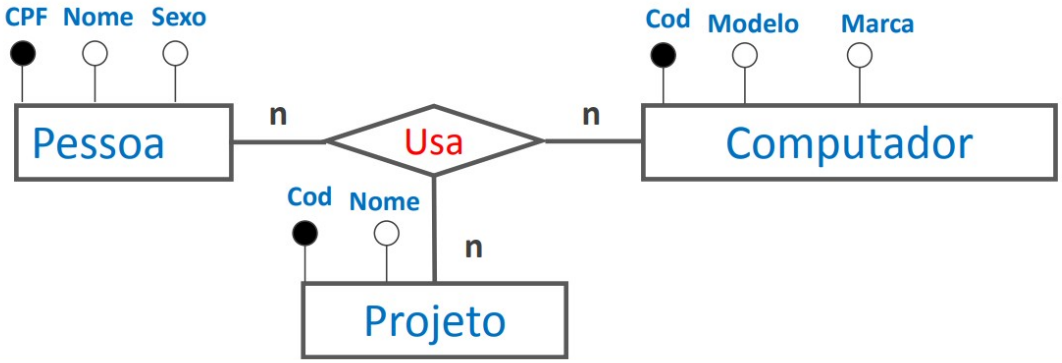
P_CPF_Ger	P_CPF_Sub
111	222
111	333
222	333
333	444

Relacionamentos Ternários

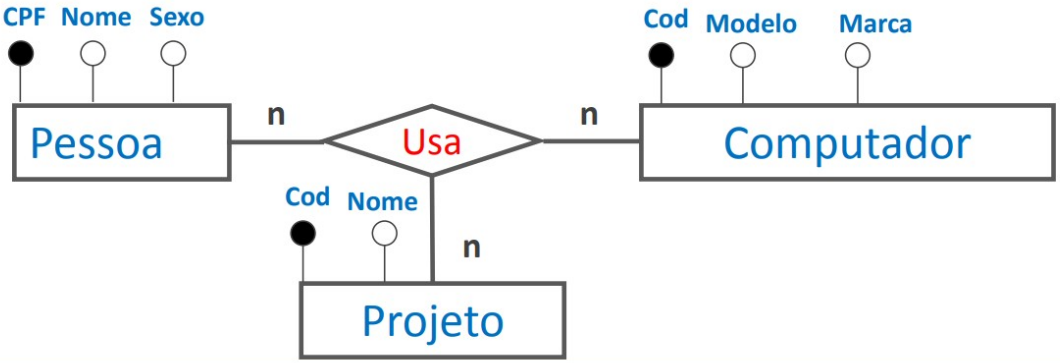
Relacionamentos Ternários

- A abordagem ER permite que sejam definidos relacionamentos de graus maiores que dois.

Relacionamentos Ternários



Relacionamentos Ternários



Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell

Projeto

Cod	Nome
P01	RH
P02	GTI

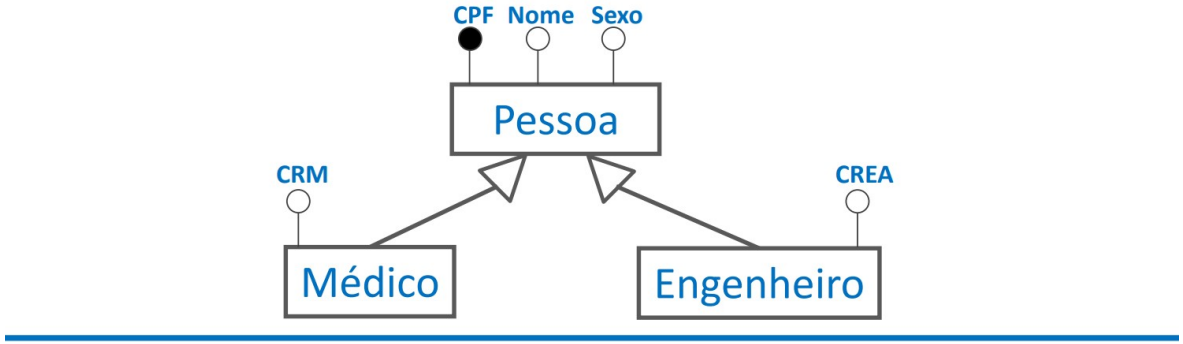
Usa

P_CPF	C_Cod	P_Cod
111	1A	P01
111	2B	P02
222	2B	P02

Generalização/Especialização (Herança)

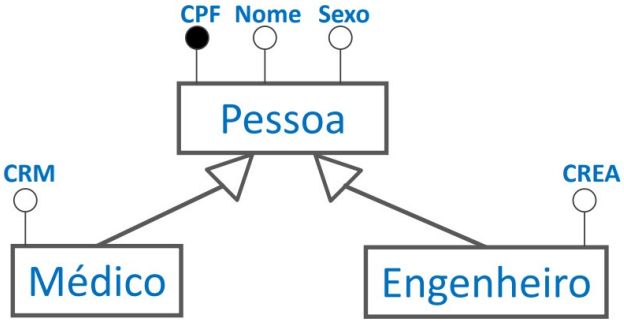
Generalização/Especialização

Conceitual



Herança

Conceitual



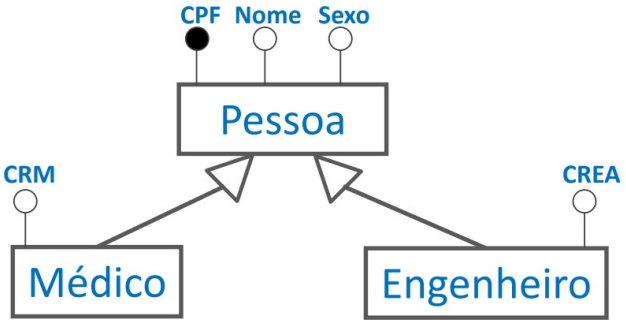
Lógico

Pessoa

CPF	Nome	Sexo	CRM	CREA
111	Ana	F	NULL	NULL
222	Beto	M	CRM01	NULL
333	Lia	F	NULL	CREA01
444	Caio	M	NULL	CREA02

Herança

Conceitual



Lógico

Médico

CPF	CRM
222	CRM01

Pessoa

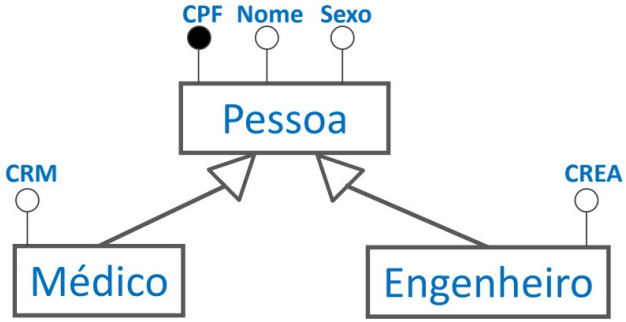
CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Engenheiro

CPF	CREA
333	CREA01
444	CREA02

Herança

Conceitual



Lógico

Médico

CPF	Nome	Sexo	CRM
222	Beto	M	CRM01

Engenheiro

CPF	Nome	Sexo	CREA
333	Lia	F	CREA01
444	Caio	M	CREA02

Especificação de Banco de Dados

Especificação de Banco de Dados

- A especificação do Banco de Dados Relacional (BDR), ou esquema do BDR, deve conter no mínimo:
  - **Tabelas** que formam o BDR;
  - **Colunas** que as tabelas possuem;
  - **Restrições de integridade**.

Especificação de Banco de Dados

- Cada **tabela** é listada pelo seu **nome**.
- Após o nome da tabela e **entre parêntesis são listados os nomes das colunas**, separados por *vírgula*.
- A coluna ou colunas que contém a **chave primária** são sublinhadas.
- **Após a definição** da tabela são listadas as definições das chaves estrangeiras que aparecem na tabela.
- Se a chave estrangeira é formada por várias colunas, elas são declaradas separadas por vírgulas

Especificação de Banco de Dados

Chave Primária Simples

Pessoa

CPF	Nome	Sexo
333	Lia	F
222	Caio	M
111	Ana	F
444	Beto	M

Especificação de Banco de Dados

Chave Primária Simples

Pessoa

CPF	Nome	Sexo
333	Lia	F
222	Caio	M
111	Ana	F
444	Beto	M

Pessoa(CPF, Nome, Sexo)

Especificação de Banco de Dados

Chave Primária Simples

Pessoa

CPF	Nome	Sexo
333	Lia	F
222	Caio	M
111	Ana	F
444	Beto	M

Pessoa(CPF, Nome, Sexo)

Chave Primária Composta

Estante

Corredor	Prateleira	Volume
A	1	10
A	2	20
B	1	20
B	2	15
C	1	10
C	2	30

Especificação de Banco de Dados

Chave Primária Simples

Pessoa

CPF	Nome	Sexo
333	Lia	F
222	Caio	M
111	Ana	F
444	Beto	M

Pessoa(CPF, Nome, Sexo)

Chave Primária Composta

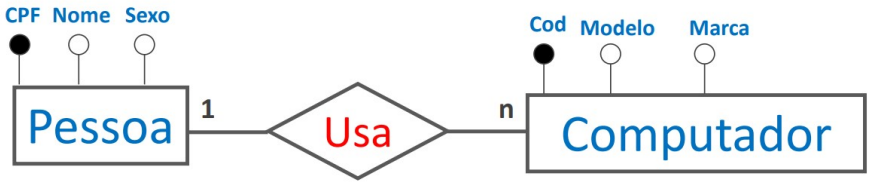
Estante

Corredor	Prateleira	Volume
A	1	10
A	2	20
B	1	20
B	2	15
C	1	10
C	2	30

Estante(Corredor, Prateleira, Volume)

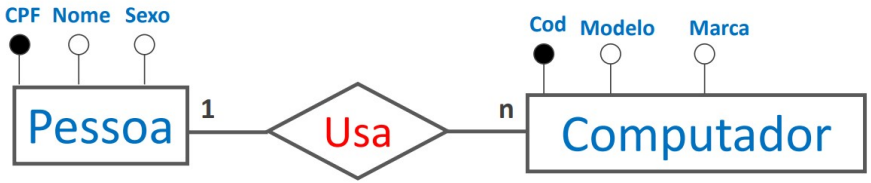
Especificação de Banco de Dados

Conceitual



Especificação de Banco de Dados

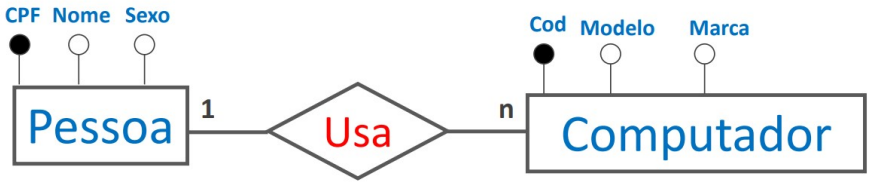
Conceitual



- **Pessoa(CPF, Nome, Sexo)**

Especificação de Banco de Dados

Conceitual



- **Pessoa(CPF, Nome, Sexo)**
- **Computador(Cod, Modelo, Marca, P_CPF)**
  P_CPF referencia Pessoa(CPF)

Lógico

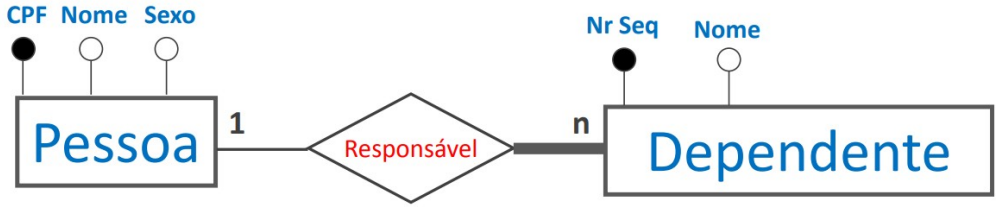
Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

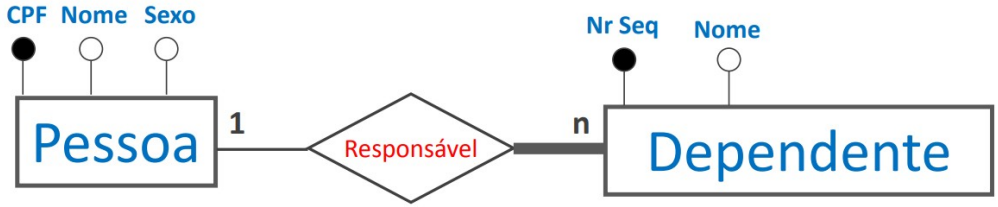
Computador

Cod	Modelo	Marca	P_CPF
1A	Notebook	Apple	111
2B	Desktop	Dell	111
3C	Desktop	IBM	333
4D	Netbook	HP	NULL

Conceitual



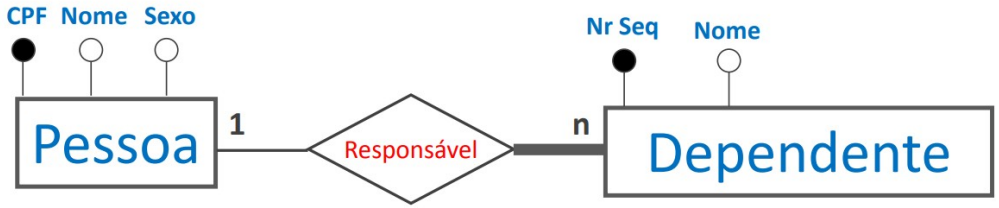
Conceitual



- **Pessoa(CPF, Nome, Sexo)**

Especificação de Banco de Dados

Conceitual



Lógico

- **Pessoa(<u>CPF</u>, Nome, Sexo)**
- **Dependente(<u>NrSeq</u>, Nome, <u>P_CPF</u>)**
  P_CPF referencia Pessoa(CPF)

Pessoa

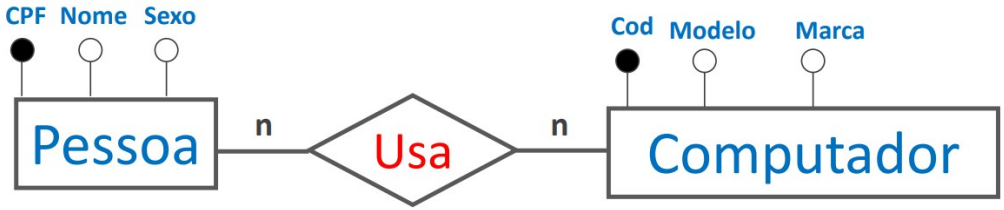
CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Dependente

Cod	Nome	P_CPF
1	Igor	111
2	Maria	111
1	André	333
1	João	222

Especificação de Banco de Dados

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Usa

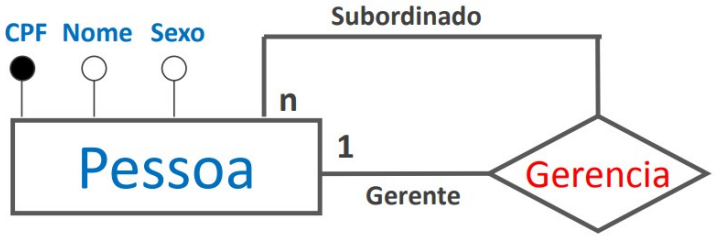
P_CPF	C_Cod
111	1A
111	2B
222	2B
333	3C

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

- **Pessoa(CPF, Nome, Sexo)**
- **Computador(Cod, Modelo, Marca)**
- **Usa(P_CPF, C_Cod)**
  P_CPF referencia Pessoa(CPF)
  C_Cod referencia Computador(Cod)

Conceitual



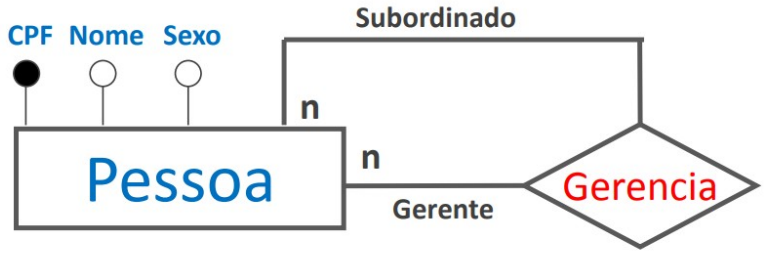
Lógico

Pessoa

CPF	Nome	Sexo	P_CPF_Ger
111	Ana	F	
222	Beto	M	111
333	Lia	F	111
444	Caio	M	333

- **Pessoa(CPF, Nome, Sexo, P_CPF_Ger)**
  P_CPF_Ger referencia Pessoa(CPF)

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo
111	Ana	F
222	Beto	M
333	Lia	F
444	Caio	M

Gerencia

P_CPF_Ger	P_CPF_Sub
111	222
111	333
222	333
333	444

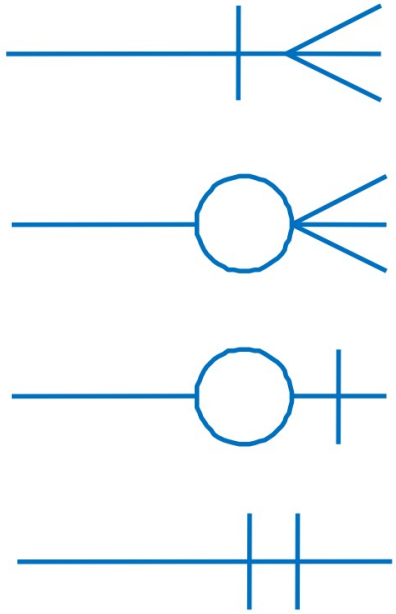
- Pessoa(CPF, Nome, Sexo)
- Gerencia(P_CPF_Ger, P_CPF_Sub)
  P_CPF_Ger referencia Pessoa(CPF)
  P_CPF_Sub referencia Pessoa(CPF)

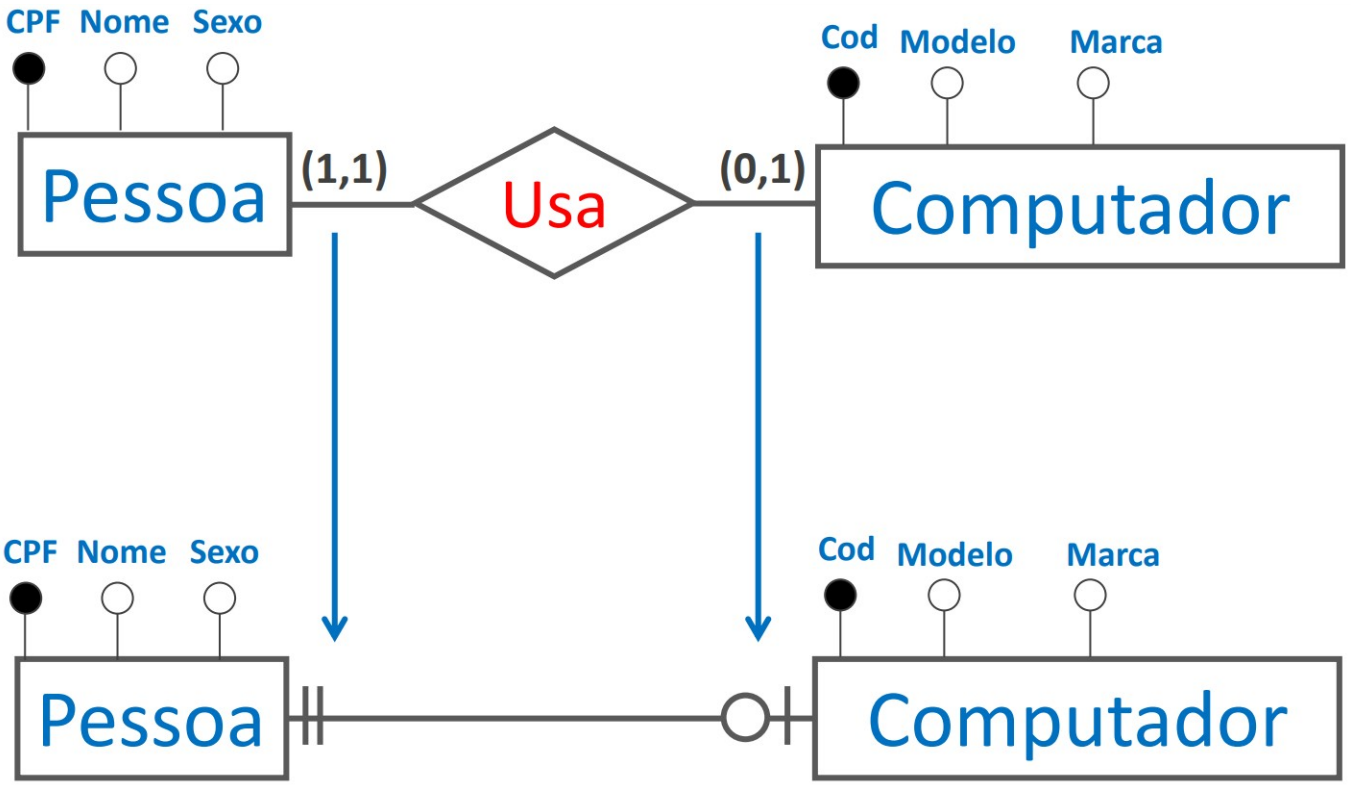
Considerações sobre o modelo ER

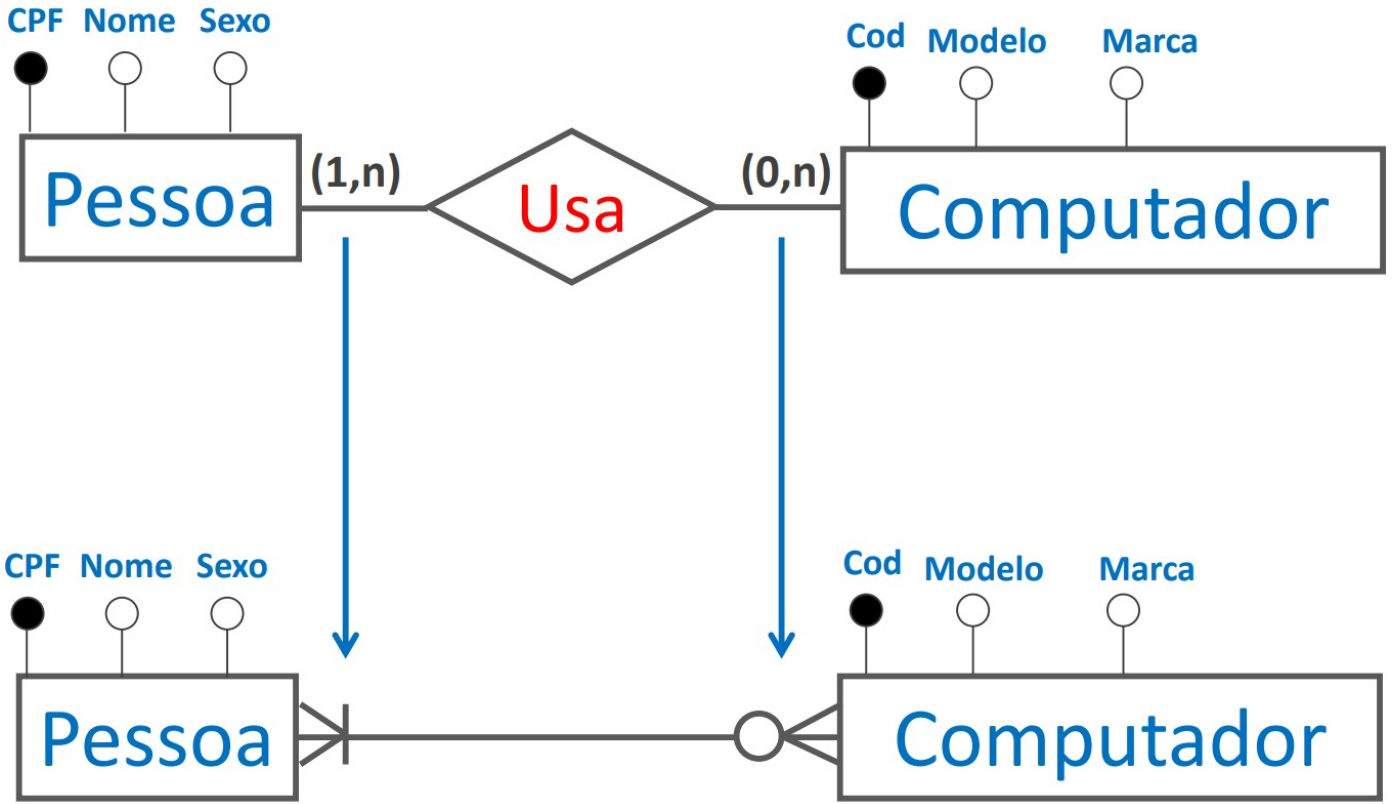
Modelo Conceitual

- Inúmeros autores propuseram inúmeros modelos.
- Na prática e na literatura não há um só modelo ER.
- Há variações na representação gráfica, na sintaxe e na semântica.
- Variantes do Modelo ER:
  - Modelo proposto por Peter Chen;
  - Notação de Engenharia da Informação (James Martin);
  - MERISE (notação europeia);
  - e UML.

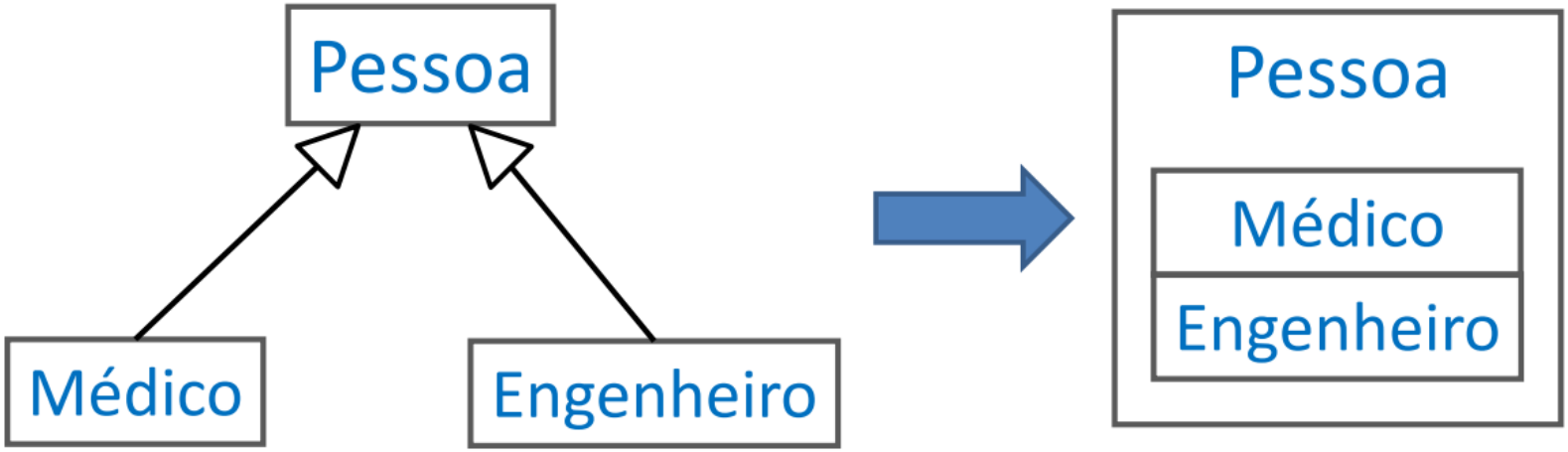
- Cardinalidades
  - um ou mais
  - zero ou mais
  - zero ou um
  - um e apenas um




CPF
Nome
Sexo
Pessoa
(1,1)
Usa
(0,1)
Cod
Modelo
Marca
Computador
CPF
Nome
Sexo
Pessoa
Cod
Modelo
Marca
Computador


CPF
Nome
Sexo
Pessoa
(1,n)
Usa
(0,n)
Cod
Modelo
Marca
Computador
CPF
Nome
Sexo
Pessoa
Cod
Modelo
Marca
Computador

- Generalização/Especialização:



Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software

Normalização 1/2

Introdução

- Ao modelar nossos banco de dados devemos evitar **anomalias**
- Anomalias são mudanças em dados que podem gerar uma inconsistência no banco de dados relacional
- Dependendo do momento em que ocorre a inconsistência podemos ter
  -Anomalia de inserção
  -Anomalia de atualização
  -Anomalia de remoção

Introdução

- **Anomalia de Inserção:** a inconsistência acontece ao inserir um dado
- Por exemplo, um novo funcionário para a agência 1550 deve usar exatamente o mesmo endereço dos outros dois funcionários que também trabalham nesta agência.

FuncN	Nome	Cargo	Salário	NumAg	Endereço	Tel
01	Luiz	Gerente	5000	1524	Prudente de Morais, 12, RN	3605-5223
02	José	Secretario	1500	1524	Prudente de Morais, 12, RN	3605-5223
03	João	Gerente	6000	1550	Hermes da Fonseca, 15, RN	4225-5889
04	Luiz	Secretario	1800	1550	Hermes da Fonseca, 15, RN	4225-5889
05	Irene	Gerente	7000	2051	Eng. Roberto Freire, 20, RN	5223-8556

Introdução

- **Anomalia de Remoção:** a inconsistência acontece ao remover um registro
- Exemplo: o que acontece ser removemos o funcionário de número 05 → os dados da agência também são removidos.

FuncN	Nome	Cargo	Salário	NumAg	Endereço	Tel
01	Luiz	Gerente	5000	1524	Prudente de Morais, 12, RN	3605-5223
02	José	Secretario	1500	1524	Prudente de Morais, 12, RN	3605-5223
03	João	Gerente	6000	1550	Hermes da Fonseca, 15, RN	4225-5889
04	Luiz	Secretario	1800	1550	Hermes da Fonseca, 15, RN	4225-5889
05	Irene	Gerente	7000	2051	Eng. Roberto Freire, 20, RN	5223-8556

Introdução

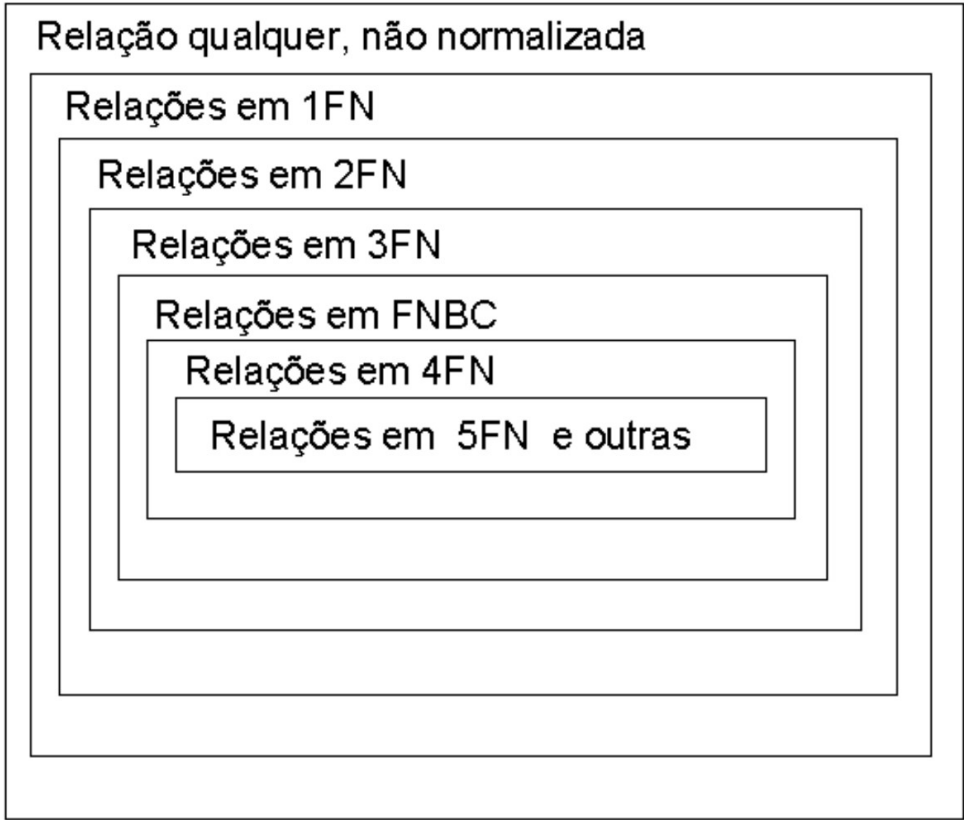
- **Anomalia de Atualização:** a inconsistência acontece ao atualizar um registro
- Uma anomalia de atualização acontece quando modificamos o endereço da agência 1524. Neste caso, teremos que atualizar o endereço de todos os funcionários da agência 1524.

FuncN	Nome	Cargo	Salário	NumAg	Endereço	Tel
01	Luiz	Gerente	5000	1524	Prudente de Morais, 12, RN	3605-5223
02	José	Secretario	1500	1524	Prudente de Morais, 12, RN	3605-5223
03	João	Gerente	6000	1550	Hermes da Fonseca, 15, RN	4225-5889
04	Luiz	Secretario	1800	1550	Hermes da Fonseca, 15, RN	4225-5889
05	Irene	Gerente	7000	2051	Eng. Roberto Freire, 20, RN	5223-8556

Normalização

- “**Bom design**” evita redundância de dados e anomalias
- Mecanismo formal para **analisar esquemas** de relações baseado nas suas chaves e nas dependências funcionais entre seus atributos.
- Consiste da decomposição de esquemas para evitar **anomalias**
- ***Projeto conceitual bem feito resulta naturalmente em esquemas normalizados.***

Normalização



1FN

Primeira Forma Normal (1FN)

- É parte da definição formal de uma relação.
- Foi definida para **não permitir atributos multivalorados**, atributos **compostos** e suas combinações.

"Uma **relação/tabela** está em **1FN** se e somente se todos os seus valores de atributos contêm apenas valores **atômicos** (*simples, indivisíveis*)."

Primeira Forma Normal (1FN)

Tabela Não Normalizada

Pessoa

CPF	Nome	Sexo	Localização	Telefone
333	Lia	F	Rio de Janeiro, RJ, Brasil	999-555, 777-333
222	Caio	M	Brasília, DF , Brasil	555-888, 333-222
111	Ana	F	São Paulo, SP, Brasil	444-999
444	Beto	M	Porto Alegre, RS, Brasil	888-222

Primeira Forma Normal (1FN)

Tabela Não Normalizada

Pessoa

CPF	Nome	Sexo	Localização	Telefone
333	Lia	F	Rio de Janeiro, RJ, Brasil	999-555, 777-333
222	Caio	M	Brasília, DF , Brasil	555-888, 333-222
111	Ana	F	São Paulo, SP, Brasil	444-999
444	Beto	M	Porto Alegre, RS, Brasil	888-222

Atributo composto

CPF	Nome	Sexo	Cidade	UF	País
333	Lia	F	Rio de Janeiro	RJ	Brasil
222	Caio	M	Brasília	DF	Brasil
111	Ana	F	São Paulo	SP	Brasil
444	Beto	M	Porto Alegre	RS	Brasil

Primeira Forma Normal (1FN)

Tabela Não Normalizada

Pessoa

CPF	Nome	Sexo	Localização	Telefone
333	Lia	F	Rio de Janeiro, RJ, Brasil	999-555, 777-333
222	Caio	M	Brasília, DF , Brasil	555-888, 333-222
111	Ana	F	São Paulo, SP, Brasil	444-999
444	Beto	M	Porto Alegre, RS, Brasil	888-222

Atributo composto

Atributo multivalorado

Decomposição de esquemas

CPF	Nome	Sexo	Cidade	UF	País
333	Lia	F	Rio de Janeiro	RJ	Brasil
222	Caio	M	Brasília	DF	Brasil
111	Ana	F	São Paulo	SP	Brasil
444	Beto	M	Porto Alegre	RS	Brasil

CPF	Telefone
333	999-555
333	777-333
222	555-888
222	333-222
111	444-999
444	888-222

DEPENDÊNCIA FUNCIONAL

Dependência Funcional

- Em uma tabela relacional, diz-se que uma coluna C2 depende funcionalmente de uma coluna C1 (ou que uma coluna C1 determina a coluna C2) quando, em todas as linhas da tabela, para cada valor de C1 que aparece na tabela, o mesmo valor de C2.

C1 → C2

- DF deve ser explicitamente definida por alguém que conheça a semântica dos atributos de uma relação.

Dependência Funcional

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto

Dependência Funcional

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto
444	Igor

Dependência Funcional

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto
444	Igor

Dependência Funcional

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto
444	Igor

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto
555	Ana

Dependência Funcional

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto
444	Igor

Pessoa

CPF	Nome
333	Lia
222	Caio
111	Ana
444	Beto
555	Ana

CPF → NOME

2FN

Segunda Forma Normal (2FN)

"Uma **relação/tabela** encontra-se na **2FN** se e somente se estiver em **1FN** e *não contém dependências parciais*, ou seja, todos os atributos não-chave dependem completamente da chave primária, e não apenas de uma parte dela"

- **Dependência Parcial:** ocorre quando uma coluna depende apenas de uma parte de uma chave primária composta.
- Uma relação para estar na 2FN não deve possuir atributo não-chave funcionalmente determinado por parte da chave primária.

Segunda Forma Normal (2FN)

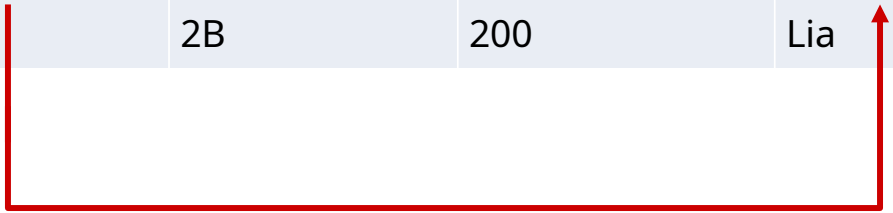
Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS

Segunda Forma Normal (2FN)

Empregado_Projeto

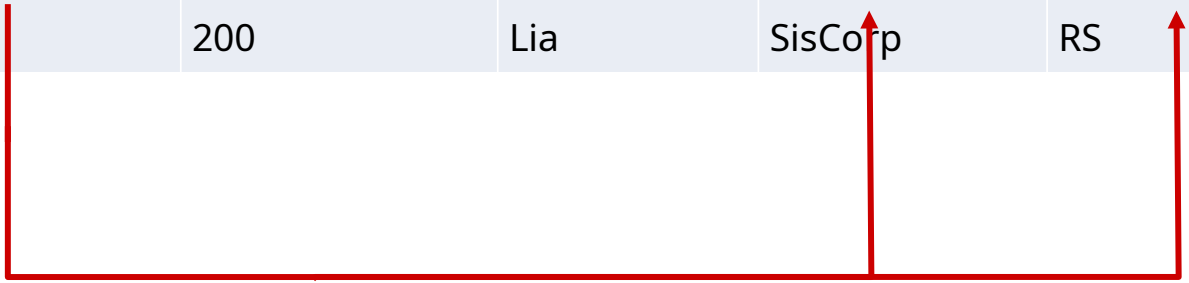
Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS



Segunda Forma Normal (2FN)

Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS



Segunda Forma Normal (2FN)

Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS



Exclusão

Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS

- **Perde o Projeto**
- **Poderia ter o projeto sem pessoas cadastradas.**

Segunda Forma Normal (2FN)

Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS



Segunda Forma Normal (2FN)

Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS



Empregado

CPF	Nome
333	Lia
222	Caio
111	Ana

Segunda Forma Normal (2FN)

Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS



Empregado

CPF	Nome
333	Lia
222	Caio
111	Ana

Projeto

Cod	Nome	Local
1A	SisRH	RJ
3C	GTI	DF
2B	SisCorp	RS

Segunda Forma Normal (2FN)

Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho	Emp_Nome	Proj_Nome	Proj_Local
333	1A	100	Lia	SisRH	RJ
222	3C	400	Caio	GTI	DF
111	3C	300	Ana	GTI	DF
333	2B	200	Lia	SisCorp	RS



Empregado

CPF	Nome
333	Lia
222	Caio
111	Ana

Empregado_Projeto

Emp_CPF	Proj_Cod	Horas_Trabalho
333	1A	100
222	3C	400
111	3C	300
333	2B	200

Projeto

Cod	Nome	Local
1A	SisRH	RJ
3C	GTI	DF
2B	SisCorp	RS

3FN

Terceira Forma Normal (3FN)

"Uma relação está em **3FN** se e somente se estiver na **2FN** e todos os seus atributos não-chave forem mutuamente independentes, ou seja, **não houver dependências transitivas**."

- **Dependência Transitiva:** ocorre quando uma coluna, além de depender da chave primária de uma tabela, depende de outra coluna ou conjunto de colunas da tabela.

- Uma relação para estar na 3FN não deve ter um atributo não-chave funcionalmente determinado por um outro atributo não-chave.

Terceira Forma Normal (3FN)

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu

Terceira Forma Normal (3FN)

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu



Terceira Forma Normal (3FN)

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu

INSERÇÃO

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu
555	João	M	1A	Adm	Igor

Nome errado!

ATUALIZAÇÃO

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu

Repetição de dados

Terceira Forma Normal (3FN)

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu



Terceira Forma Normal (3FN)

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu



Departamento

Cod	Nome	Ger
1A	RH	Igor
2B	Adm	Edu
3C	TI	Tiago

Terceira Forma Normal (3FN)

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu



Empregado

CPF	Nome	Sexo	Dep_Cod
333	Lia	F	1A
222	Caio	M	2B
111	Ana	F	3C
444	Beto	M	2B

Departamento

Cod	Nome	Ger
1A	RH	Igor
2B	Adm	Edu
3C	TI	Tiago

Terceira Forma Normal (3FN)

Empregado_Departamento

Emp_CPF	Emp_Nome	Emp_Sexo	Dep_Cod	Dep_Nome	Dep_Ger
333	Lia	F	1A	RH	Igor
222	Caio	M	2B	Adm	Edu
111	Ana	F	3C	TI	Tiago
444	Beto	M	2B	Adm	Edu



Empregado

CPF	Nome	Sexo	Dep_Cod
333	Lia	F	1A
222	Caio	M	2B
111	Ana	F	3C
444	Beto	M	2B

Departamento

Cod	Nome	Ger
1A	RH	Igor
2B	Adm	Edu
3C	TI	Tiago

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software

Normalização 2/2

Forma Normal de Boyce-Codd (FNBC)

Dependência Funcional

- Em uma tabela relacional, diz-se que uma coluna Y depende funcionalmente de uma coluna X quando, em todas as linhas da tabela, para cada valor de X que aparece na tabela, o mesmo valor de Y.

X → Y

Lê-se que X determina funcionalmente Y

Exemplo: cod_produto → nome_produto

cod_produto	nome_produto
101	Teclado USB
102	Mouse Óptico
103	Monitor 24"

Dependência Funcional

- A **dependência funcional trivial** fala que um determinante com mais de um atributo pode determinar seus próprios membros quando isolados. Um exemplo:
- [banco, agência] --> banco (DF trivial, pois banco é parte do determinante)
- [banco, agência] --> agência (DF trivial, pois agência é parte do determinante)

- Quando um determinante identifica outro atributo qualquer, temos uma dependência **funcional não trivial**. Um exemplo:
- [banco, agência] --> cidade (DF não trivial, pois cidade não faz parte do determinante)

Forma Normal de Boyce-Codd

- A formal normal de Boyce-Codd (FNBC) foi proposta como uma forma mais simples da 3FN, entretanto, essa forma normal acaba sendo mais rigorosa do que 3FN;

- Um esquema de relação está na FNBC Para toda dependência funcional não trivial X → Y, o conjunto de atributos X deve ser uma superchave da relação.

- **Superchave** = conjunto de atributos que identifica unicamente cada linha da tabela.

- Em outras palavras: nenhum atributo que não seja chave pode determinar outro atributo.

Forma Normal de Boyce-Codd

Pizza	Tipo	Ingrediente
1	Queijo	Mussarela
1	Carne	Frango
1	Vegetal	Azeitona
2	Queijo	Provolone
2	Carne	Calabresa
2	Vegetal	Manjericão
3	Queijo	Mussarela
3	Carne	Calabresa
3	Vegetal	Azeitona

- **Chave primária**: Pizza e Tipo

Forma Normal de Boyce-Codd

Pizza	Tipo	Ingrediente
1	Queijo	Mussarela
1	Carne	Frango
1	Vegetal	Azeitona
2	Queijo	Provolone
2	Carne	Calabresa
2	Vegetal	Manjericão
3	Queijo	Mussarela
3	Carne	Calabresa
3	Vegetal	Azeitona

- **Chave primária**: Pizza e Tipo
- **Está na 1FN**
  - *Não há atributos multivalorados*

Forma Normal de Boyce-Codd

Pizza	Tipo	Ingrediente
1	Queijo	Mussarela
1	Carne	Frango
1	Vegetal	Azeitona
2	Queijo	Provolone
2	Carne	Calabresa
2	Vegetal	Manjericão
3	Queijo	Mussarela
3	Carne	Calabresa
3	Vegetal	Azeitona

- **Chave primária**: Pizza e Tipo
- **Está na 2FN**
  - *Não há dependências funcionais parciais;*
  - *O atributo Ingrediente depende totalmente da chave primária;*
  - Dependências Funcionais:
    - {Pizza, Tipo} → Ingrediente

    (para cada pizza e tipo, existe um único ingrediente)

Forma Normal de Boyce-Codd

Pizza	Tipo	Ingrediente
1	Queijo	Mussarela
1	Carne	Frango
1	Vegetal	Azeitona
2	Queijo	Provolone
2	Carne	Calabresa
2	Vegetal	Manjericão
3	Queijo	Mussarela
3	Carne	Calabresa
3	Vegetal	Azeitona

- **Está na 3FN**
  - *Não há dependências funcionais transitivas;*
  - *Dependência funcional transitiva: ocorre quando um atributo não chave depende de outro atributo não chave, que por sua vez depende da chave primária.*
  - Dependências Funcionais:
    - {Pizza, Tipo} → Ingrediente

Forma Normal de Boyce-Codd

Pizza	Tipo	Ingrediente
1	Queijo	Mussarela
1	Carne	Frango
1	Vegetal	Azeitona
2	Queijo	Provolone
2	Carne	Calabresa
2	Vegetal	Manjericão
3	Queijo	Mussarela
3	Carne	Calabresa
3	Vegetal	Azeitona

- **Está na FNBC?**
  - Dependências Funcionais:
    - {Pizza, Tipo} → Ingrediente
      (para cada pizza e tipo, existe um único ingrediente)
    - {Ingrediente} → Tipo **(Extra)**
      (cada ingrediente está associado a apenas um tipo)

Forma Normal de Boyce-Codd

Pizza	Tipo	Ingrediente
1	Queijo	Mussarela
1	Carne	Frango
1	Vegetal	Azeitona
2	Queijo	Provolone
2	Carne	Calabresa
2	Vegetal	Manjericão
3	Queijo	Mussarela
3	Carne	Calabresa
3	Vegetal	Azeitona

- **Dependências Funcionais:**
  - **{Pizza, Tipo} → Ingrediente**
  - {Ingrediente} → Tipo (Extra)
- A primeira dependência funcional não viola a FNBC;
- Os atributos do lado X pertencem à chave primária da relação;

Forma Normal de Boyce-Codd

Pizza	Tipo	Ingrediente
1	Queijo	Mussarela
1	Carne	Frango
1	Vegetal	Azeitona
2	Queijo	Provolone
2	Carne	Calabresa
2	Vegetal	Manjericão
3	Queijo	Mussarela
3	Carne	Calabresa
3	Vegetal	Azeitona

- **Dependências Funcionais:**
  - {Pizza, Tipo} → Ingrediente
  - **{Ingrediente} → Tipo (Extra)**

- Entretanto, a segunda dependência funcional viola a FNBC;

- O atributo **Ingrediente**, que está no lado X da dependência funcional, **não é um atributo chave da relação**;

Forma Normal de Boyce-Codd

Pizza	Tipo	Ingrediente
1	Queijo	Mussarela
1	Carne	Frango
1	Vegetal	Azeitona
2	Queijo	Provolone
2	Carne	Calabresa
2	Vegetal	Manjericão
3	Queijo	Mussarela
3	Carne	Calabresa
3	Vegetal	Azeitona

- **Dependências Funcionais:**
  - {Pizza, Tipo} → Ingrediente
  - **{Ingrediente} → Tipo (Extra)**

- Para resolver isso, criamos outra relação contendo os atributos da dependência funcional que viola a forma normal;

- O atributo do lado Y da dependência deve ser removido da relação original;

Forma Normal de Boyce-Codd

Pizza	Ingrediente
1	Mussarela
1	Frango
1	Azeitona
2	Provolone
2	Calabresa
2	Manjericão
3	Mussarela
3	Calabresa
3	Azeitona

Ingrediente	Tipo
Mussarela	Queijo
Frango	Carne
Azeitona	Vegetal
Provolone	Queijo
Calabresa	Carne
Manjericão	Vegetal

Análise Crítica das Chaves Primárias

- Questões para refletirem:
  - "O CPF pode mudar? É estável?"
  - "A matrícula pode ser alterada pela empresa?"
  - "Essa chave composta é muito grande para performance?"
  - "Dados sensíveis devem ser chaves primárias?"

Ainda na Modelagem Lógica

Análise Crítica das Chaves Primárias

- Refazer o modelo incorporando IDs onde necessário:
  - **Antes**
    - Funcionario(cpf, matricula, nome, cargo_codigo)
  - **Depois**
    - Funcionario(funcionario_id, cpf, matricula, nome, cargo_id)
- Gerar DDL com IDs já decididos

Introdução ao Modelo Físico

Introdução

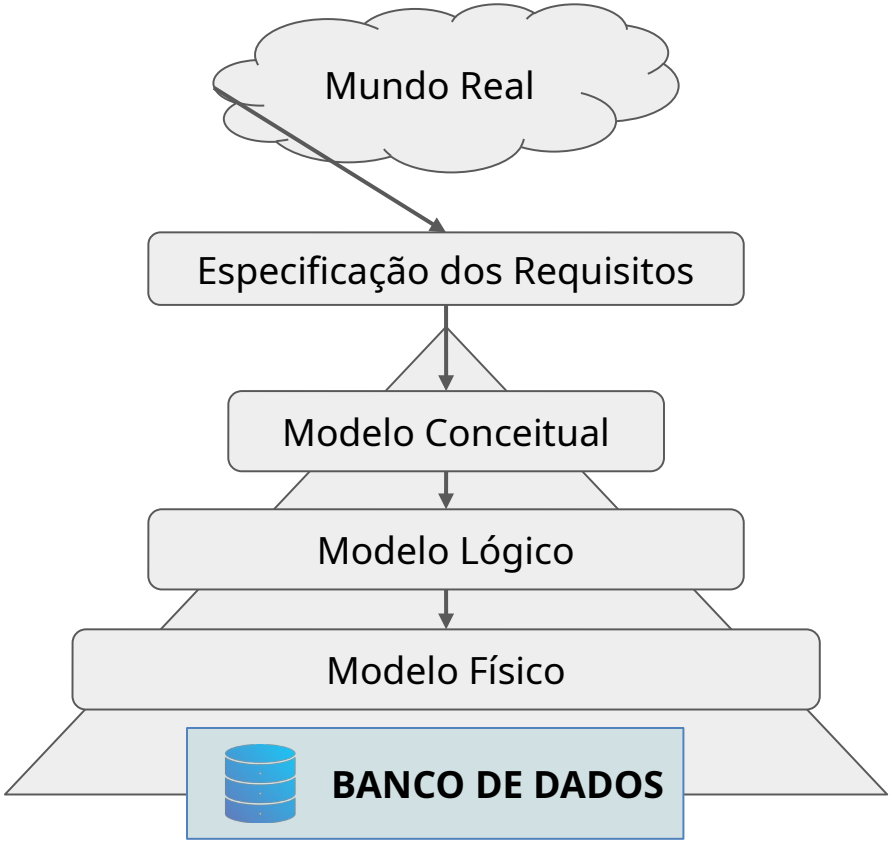
- Usa como base o modelo lógico.
- Refinamento do modelo lógico
- Preocupa em criar as estruturas de dados físicas no SGBD.
- É a etapa final antes de se partir para a criação do banco de dados.

Introdução

- Muitas das ferramentas de modelagem disponíveis no mercado possibilitam a geração do modelo físico a partir do modelo lógico desenvolvido.

- Scripts em uma linguagem denominada:
  - SQL
  - DDL

Introdução


Mundo Real
Especificação dos Requisitos
Modelo Conceitual
Modelo Lógico
Modelo Físico
BANCO DE DADOS

Introdução



- É **fonte dos requisitos** e o ponto de partida.
- Representa o ambiente sobre o qual o banco de dados é construído, incluindo os **processos de negócios** e as necessidades de informação.

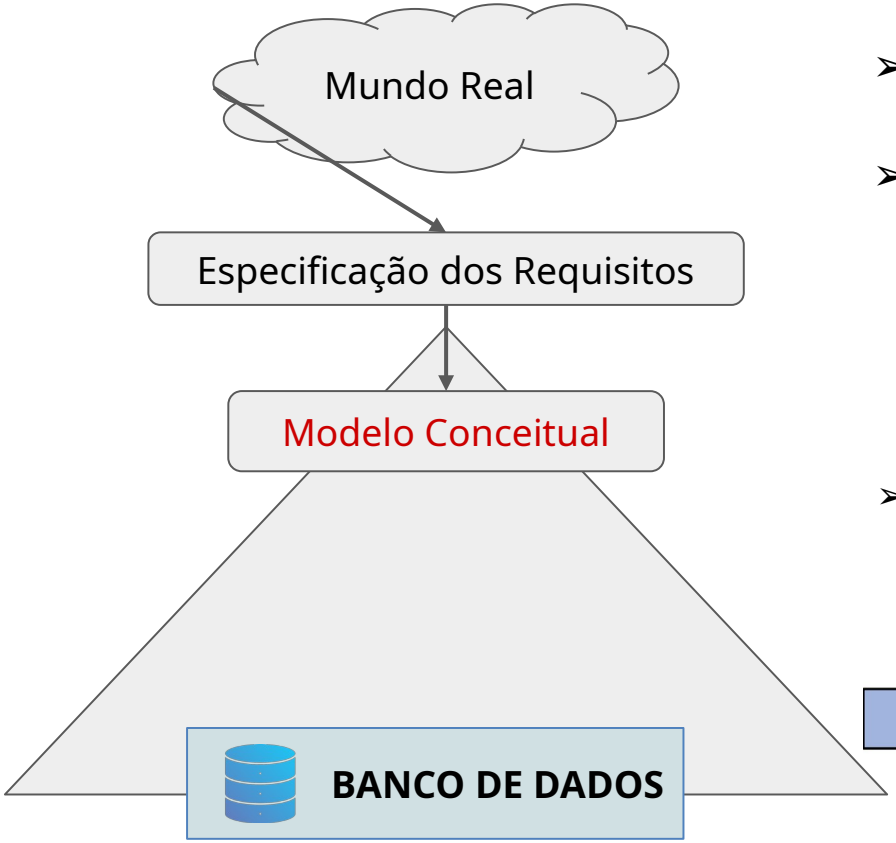
Introdução



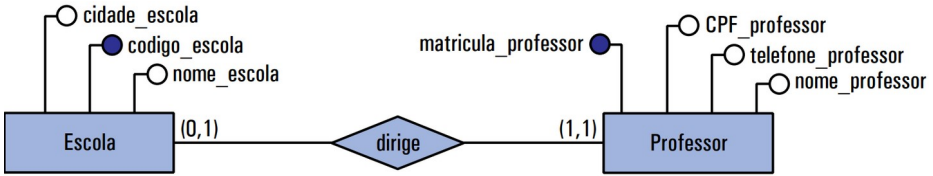
- É o processo de **coleta e análise** das necessidades de informação que o sistema de banco de dados deve atender.

- Entender o que os **usuários finais** esperam do banco de dados em termos de **armazenamento de dados**, **consultas**, **atualizações** e **relatórios**.

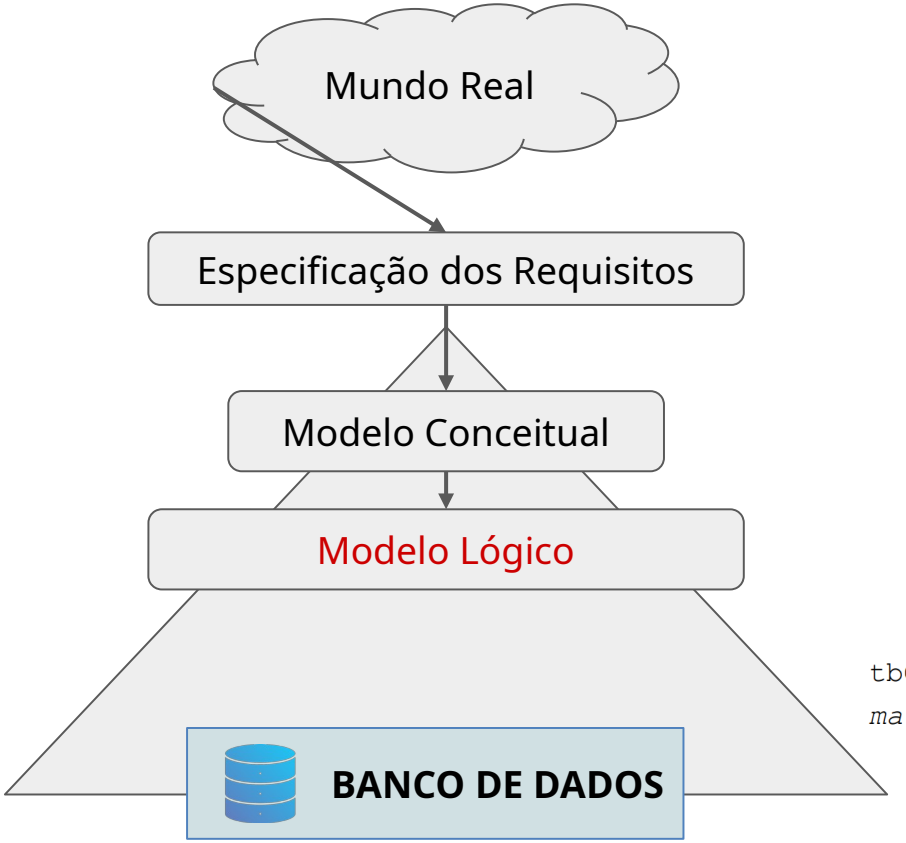
Introdução



- **Modelo abstrato**
- Organiza os dados de acordo com as regras e estruturas do mundo real, *sem levar em conta como esses dados serão armazenados fisicamente*.
- **Diagrama entidade-relacionamento (ER)**



Introdução

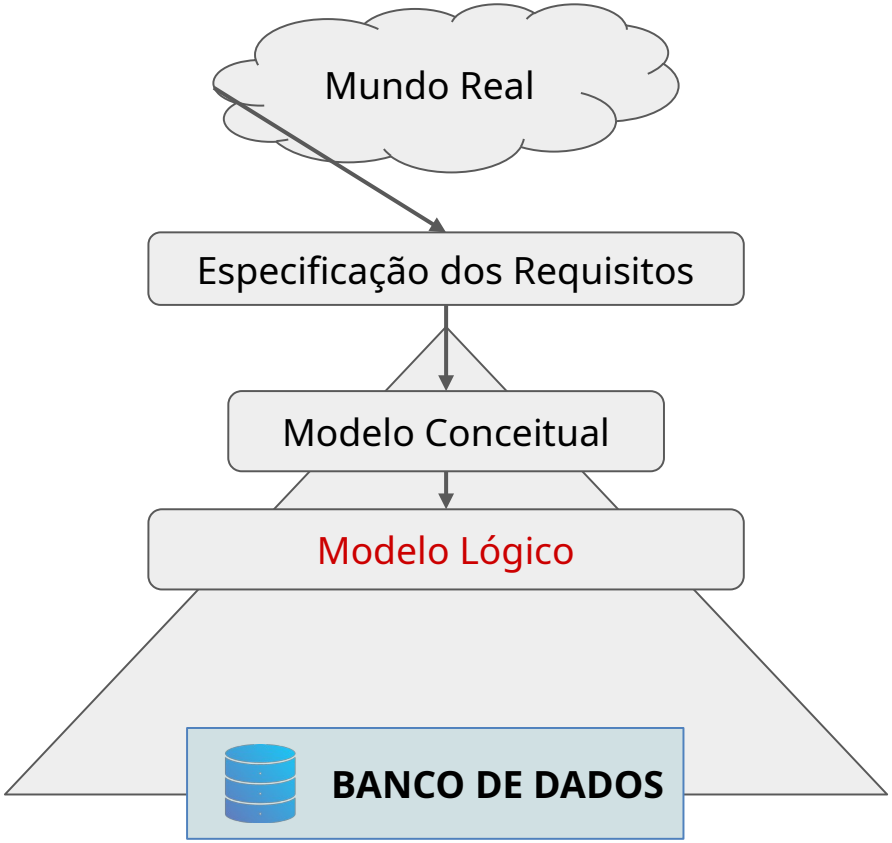


- O modelo conceitual é **traduzido** para um modelo lógico, que é **independente** do SGBD em que são implementados

- Envolve *definir* tabelas, chaves primárias e estrangeiras, e os **relacionamentos** entre as tabelas.

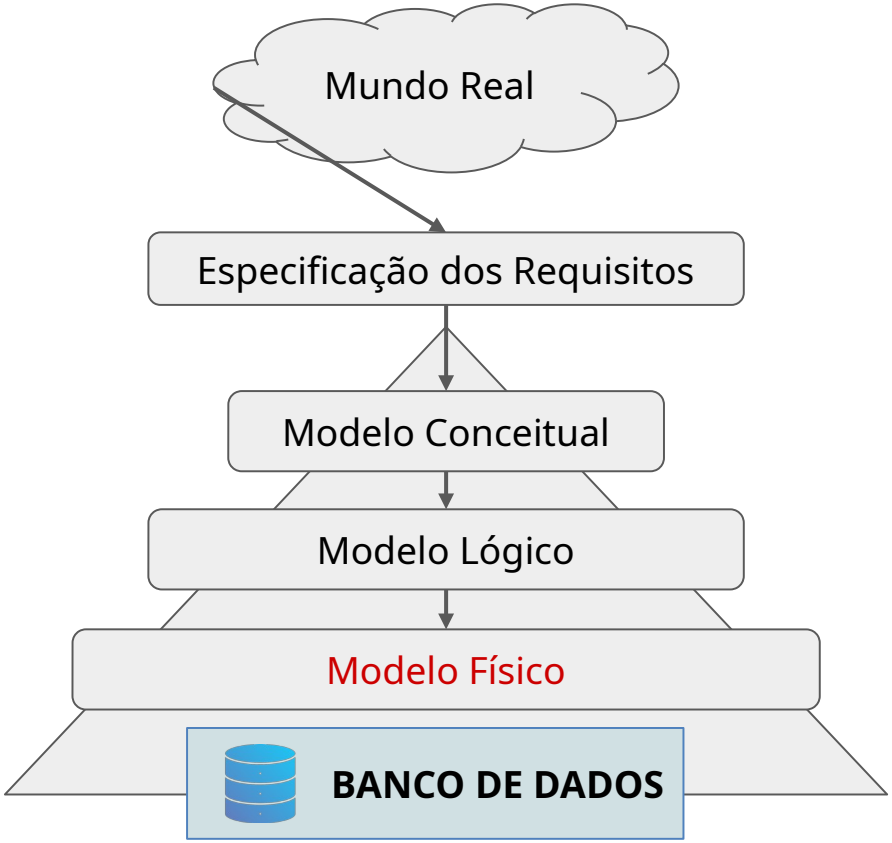
```
tbContato(matricula_aluno, fone_contato, nome_contato
matricula_aluno referencia tbAluno
```

Introdução



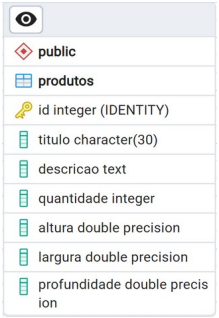
- **Normalização**
  - Evitar a redundância de dados.
  - Melhorar a integridade dos dados.
  - Facilitar a manutenção dos dados.
  - Otimizar o desempenho das consultas.
  - Flexibilidade do banco de dados.

Introdução



- O **modelo lógico** é transformado em um **modelo físico**, que é a representação de *como os dados serão realmente armazenados em um meio físico*.

- Leva em conta o SGBD específico.




```
CREATE TABLE produtos
(
    id integer NOT NULL,
    titulo character(30) NOT NULL,
    descricao text,
    quantidade integer,
    altura double precision,
    largura double precision,
    profundidade double precision,
    CONSTRAINT produtos_pkey PRIMARY KEY (id)
)
```



Structured Query Language (SQL)

SQL

- É uma linguagem para banco de dados **relacional**.
- Antes era chamada de Sequel (*Structured English Query Language*).
- Desenvolvida pela **IBM** no início da década de **70**, por Codd.
- A linguagem evoluiu bastante, passando a se chamar simplesmente Structured Query Language (SQL).

SQL

- Houve diversas variações da SQL (**dialetos**).
- A American National Standards Institute (**ANSI**) apresentou uma padronização em 1986 e a **ISO** em 1987.

- Sybase Advantage Database Server
- Apache Derby
- Caché
- DB2
- Dataflex
- Firebird
- HSQLDB
- IDMS
- IMSInformix
- Ingres
- InterBase
- Microsoft Access
- Microsoft SQL Server
- MySQL
- Oracle
- PointBase Micro
- PostgreSQL
- SQLite
- LiteBase Mobile
- Sybase Adaptive Server Enterprise
- Teradata

https://www.iso.org/standard/76583.html
https://blogs.oracle.com/sql/post/general-availability-of-the-sql2023-standard

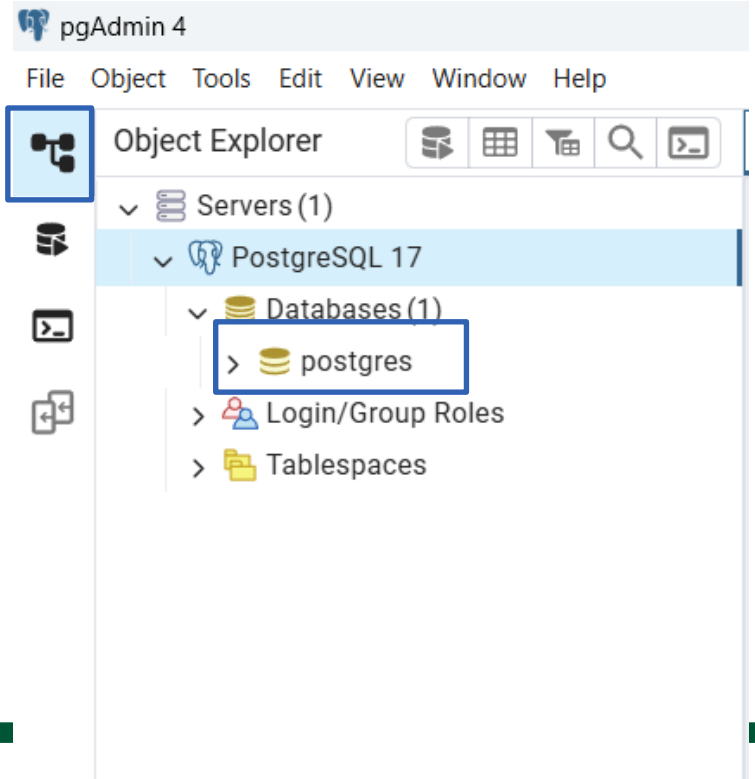
Estrutura da SQL

- A SQL é composta por um conjunto de linguagens que permitem a **definição**, **consulta** e **atualização de dados**, além de recursos adicionais para a *segurança e gestão* do BD.
  - **DDL:** Linguagem de Definição de Dados;
  - **DQL:** Linguagem de Consulta de Dados
  - **DML**: Linguagem de Manipulação de Dados;
  - **DCL**: Linguagem de Controle de Dados;
  - **DTL**: Linguagem de Controle de Transações.

Mãos à Obra!



- Abram o **pgAdmin**

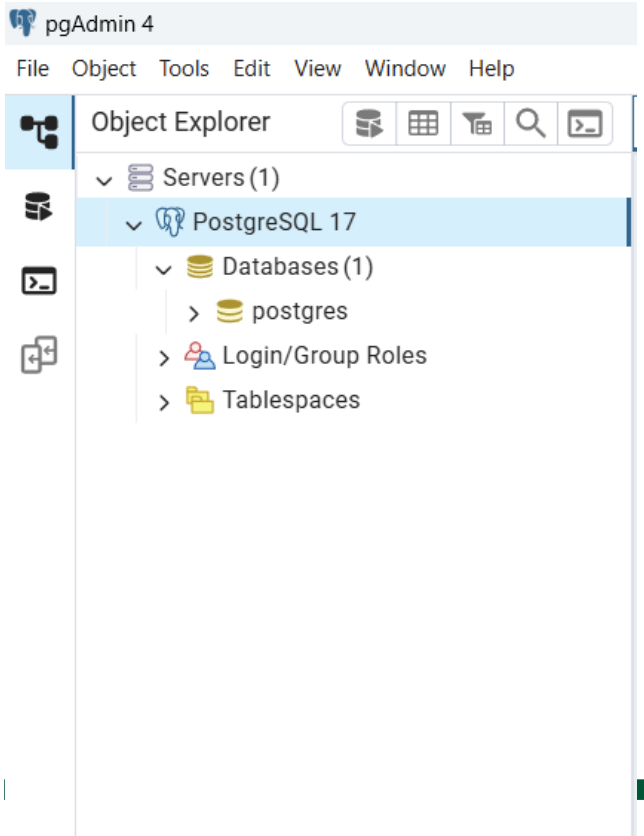


- Quando você instala o PostgreSQL, o sistema automaticamente cria alguns DB

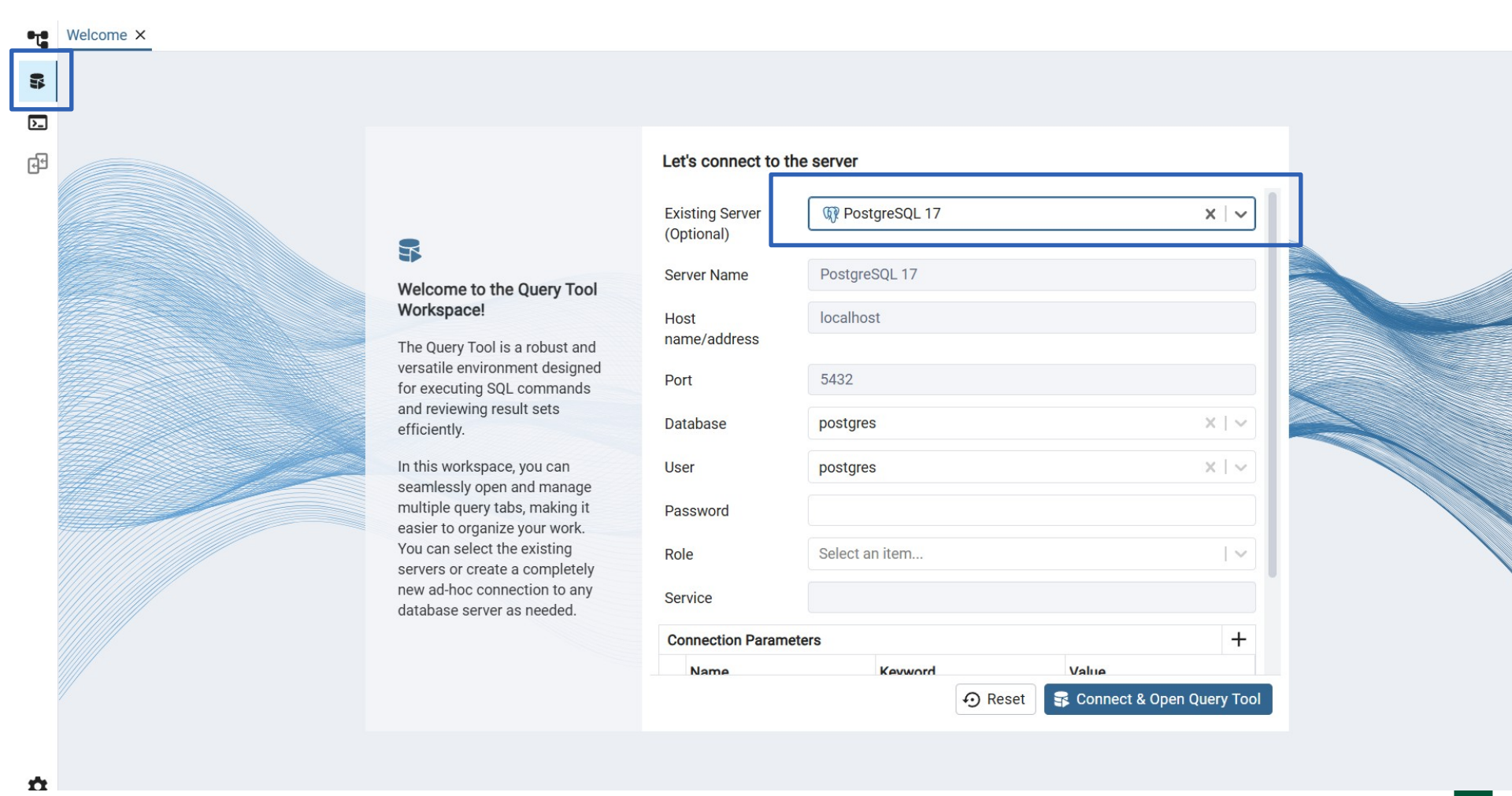
- DB **postgres**
  - Esse banco é criado para que o administrador possa ter um ponto de acesso inicial ao servidor e executar comandos administrativos ou criar novos bancos.

Mãos à Obra!





- Um banco de dados chamado **template1:**
  - Serve como modelo para criação de novos bancos. Sempre que você cria um banco novo sem especificar um modelo, o PostgreSQL copia a estrutura e os objetos do template1.
- Um banco chamado **template0:**
  - Usado como uma versão "limpa" do banco, sem personalizações, para permitir restaurações ou criação de bancos sem herdar modificações feitas no template1.


Welcome
Welcome to the Query Tool Workspace!
The Query Tool is a robust and versatile environment designed for executing SQL commands and reviewing result sets efficiently.
In this workspace, you can seamlessly open and manage multiple query tabs, making it easier to organize your work. You can select the existing servers or create a completely new ad-hoc connection to any database server as needed.
Let's connect to the server
Existing Server (Optional)
PostgreSQL 17
Server Name
PostgreSQL 17
Host name/address
localhost
Port
5432
Database
postgres
User
postgres
Password
Role
Select an item...
Service
Connection Parameters
Name
Keyword
Value
Reset
Connect & Open Query Tool

Linguagem de Definição de Dados (DDL)

- **Criar um Database (Banco de Dados):**

Sintaxe

```
CREATE DATABASE nome_do_banco_de_dados;
```

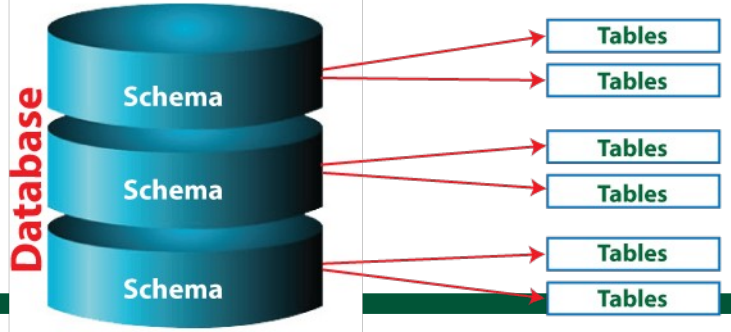
Data Output | Messages | Notifications

```
CREATE DATABASE

Query returned successfully in 277 msec.
```

```
CREATE DATABASE loja_do_joao;
```

- **Criar um Schema (Esquema):**
  - Após criar o banco de dados e conectar-se a ele, você pode criar um schema
  - Um **schema** no PostgreSQL funciona como um contêiner lógico dentro de um banco de dados, atuando como um namespace para organizar objetos do banco, como tabelas, views, funções, tipos de dados, procedures armazenadas e outros objetos.
  - De forma muito simplificada, o schema é como uma “pasta” dentro de um banco de dados, que organiza tabelas. views. funções.....

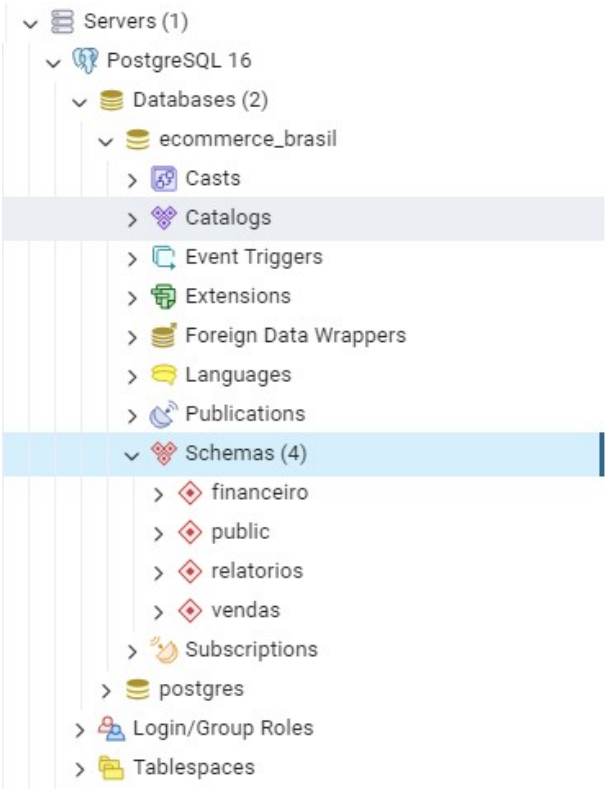


Linguagem de Definição de Dados (DDL)

- **Criar um Schema (Esquema):**

Sintaxe

```
CREATE SCHEMA nome_do_esquema;
```



Linguagem de Definição de Dados (DDL)

- **Criar uma Table (Tabela):**
  - Dentro de um schema, você pode criar tabelas para definir a estrutura onde os dados serão armazenados:

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

Linguagem de Definição de Dados (DDL)

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

- **nome_do_esquema:** O esquema ao qual a tabela pertencerá. Se você não especificar um esquema, a tabela será criada no esquema padrão (geralmente é o **public**).
- **nome_da_tabela:** O nome que você deseja dar à sua tabela.

Linguagem de Definição de Dados (DDL)

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

- **coluna:** O nome de uma coluna na tabela.
- **tipo_de_dado:** O tipo de dados (como **INTEGER**, **VARCHAR**, **TIMESTAMP**) que a coluna irá armazenar.

Linguagem de Definição de Dados (DDL)

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

- **CONSTRAINTS:** Restrições opcionais para a coluna, como **NOT NULL, PRIMARY KEY, UNIQUE, etc**.

- **Criar uma Table (Tabela):**
  - Dentro de um schema, você pode criar tabelas para definir a estrutura onde os dados serão armazenados:

```sql
CREATE TABLE loja.cliente (
    id_cliente SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    data_nascimento DATE,
    data_de_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



Structured Query Language (SQL)

DDL: Linguagem de Definição de Dados

Linguagem de Definição de Dados (DDL)

```sql
CREATE DATABASE loja_do_joao;

CREATE SCHEMA loja;

CREATE TABLE loja.cliente (
    id_cliente SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    data_nascimento DATE,
    data_de_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- **PRIMARY KEY (Chave Primária):**
  - Uma **PRIMARY KEY** é uma **restrição** que identifica de forma única cada linha em uma tabela de banco de dados. Não podem existir duas linhas com a mesma chave primária, e ela não pode conter valores NULL.

Sintaxe

```
CREATE TABLE Pessoa (
    PessoaID INT NOT NULL,
    Nome VARCHAR(100),
    PRIMARY KEY (PessoaID)
);
```

Linguagem de Definição de Dados (DDL)

- **FOREIGN KEY (Chave Estrangeira):**
  - Uma **FOREIGN KEY** é uma chave usada para **vincular duas tabelas juntas**. É uma coluna ou uma combinação de colunas usadas para estabelecer e impor um link entre os dados de duas tabelas para controlar os dados que podem ser armazenados na tabela de foreign key.

Sintaxe

```
CREATE TABLE Pedido (
    PedidoID INT NOT NULL,
    PedidoNumero INT NOT NULL,
    PessoaID INT,
    PRIMARY KEY (PedidoID),
    FOREIGN KEY (PessoaID) REFERENCES Pessoa(PessoaID)
);
```

Linguagem de Definição de Dados (DDL)

- **COMPOSITE KEY (Chave Composta):**
  - É uma **chave primária** que consiste em **<u>duas ou mais colunas usadas juntas</u>** para criar uma chave primária para uma tabela. Isso é usado quando você deseja ter uma chave primária que contenha múltiplos atributos.

Sintaxe

```
CREATE TABLE ProdutoPedido (
    PedidoID INT,
    ProdutoID INT,
    Quantidade INT,
    PRIMARY KEY (PedidoID, ProdutoID),
    FOREIGN KEY (PedidoID) REFERENCES Pedido(PedidoID),
    FOREIGN KEY (ProdutoID) REFERENCES Produto(ProdutoID)
);
```

Linguagem de Definição de Dados (DDL)

- O tipo de dado que você escolhe ao criar uma tabela em SQL pode depender do SGBD que você está usando.

- Cada SGBD pode suportar diferentes tipos de dados ou ter variações nos nomes e no comportamento dos tipos de dados.

Linguagem de Definição de Dados (DDL)

- **Exemplos**

  - **MySQL**: Utiliza tipos como VARCHAR, TEXT, INT, DATETIME, entre outros.

  - **PostgreSQL**: Também usa tipos como VARCHAR e TEXT, mas também possui tipos como **SERIAL** para chaves primárias **autoincrementáveis** e tipos específicos como **JSON** e **ARRAY**.

Linguagem de Definição de Dados (DDL)

- **Exemplos**

  - **SQLite**: Tende a ser mais flexível com os tipos de dados, seguindo um sistema de **tipagem dinâmica**, mas reconhece tipos comuns como **TEXT**, **INTEGER**, e **REAL**.

  - **Microsoft SQL Server**: Tem tipos como **NVARCHAR** para texto Unicode, **INT**, **DATETIME**, entre outros, e também tipos específicos para dados hierárquicos como **XML** e **HIERARCHYID**.

Atividade

1) Escolha um SGBD relacional de sua preferência que utilize SQL e:
    a) Realize uma pesquisa sobre os tipos de dados suportados pelo SGBD escolhido.
    b) Quais são os tipos de dados básicos disponíveis? (Por exemplo, tipos numéricos, de data e hora, strings, etc.)
    c) Existem tipos de dados exclusivos ou especializados oferecidos por esse SGBD? (Por exemplo, JSON em PostgreSQL)
    d) Qual é o tamanho máximo para tipos de string como VARCHAR?
    e) Como são tratados os dados binários?
    f) Apresente as vantagens e desvantagens.
        i) P/ cada tipo de dado importante, descreva possíveis vantagens e desvantagens de seu uso em contextos de desenvolvimento reais. Por exemplo: quando usar um tipo TEXT em vez de VARCHAR?



Structured Query Language (SQL)

DDL: Linguagem de Definição de Dados

Linguagem de Definição de Dados (DDL)

- **ALTER TABLE**
  - Após criarmos uma tabela em um banco de dados, podemos alterar sua estrutura com o uso da cláusula ALTER TABLE.
  - Deve-se notar que esta alteração é sempre sobre a estrutura da tabela, e não sobre os dados da tabela.

```
ALTER TABLE schema.tabela
...
alteração
...
```

Linguagem de Definição de Dados (DDL)

- **ALTER TABLE com ADD COLUMN:**
  - Usado para adicionar uma coluna

```
ALTER TABLE schema.tabela
ADD COLUMN nome_coluna tipo_dado;
```

```
ALTER TABLE loja.cliente
ADD COLUMN rg VARCHAR(100);
```

Linguagem de Definição de Dados (DDL)

- **ALTER TABLE com ALTER COLUMN:**
  - Usado para modificar uma coluna

```
ALTER TABLE schema.tabela
ALTER COLUMN nome_coluna TYPE novo_tipo;
```

```
ALTER TABLE loja.cliente
ALTER COLUMN rg TYPE VARCHAR(50);
```

Linguagem de Definição de Dados (DDL)

- **ALTER TABLE com DROP COLUMN:**
  - Usado para excluir uma coluna

```
ALTER TABLE loja.cliente
DROP COLUMN rg
```

Linguagem de Definição de Dados (DDL)

- **ALTER TABLE com DEFAULT:**
  - O **DEFAULT** é usado para definir um valor padrão para uma coluna.

Sintaxe

```
ALTER TABLE ProdutoPedido
ALTER COLUMN Quantidade SET DEFAULT 1;
```

- **ALTER TABLE com CHECK:**

  - **CHECK** é uma constraint que limita o intervalo de valores que podem ser colocados em uma coluna.

Sintaxe

```
ALTER TABLE ProdutoPedido
ADD CONSTRAINT CHK_Quantidade CHECK (Quantidade > 0);
```

Linguagem de Definição de Dados (DDL)

- **DROP e IF EXISTS:**
  - DROP TABLE é usado para excluir uma tabela
  - O comando **IF EXISTS** é frequentemente usado com **DROP TABLE** para evitar erros ao tentar excluir uma tabela que não existe.

Sintaxe

```
DROP TABLE IF EXISTS nome_da_tabela;
```

Atividade

1) Crie uma base de dados usando comando SQL para o modelo relacional a seguir:

```
tbEmpregado (cod_emp, nome_emp, rua_emp, cidade_emp, fone_emp,
cod_gerente)
cod_gerente referencia tbGerente

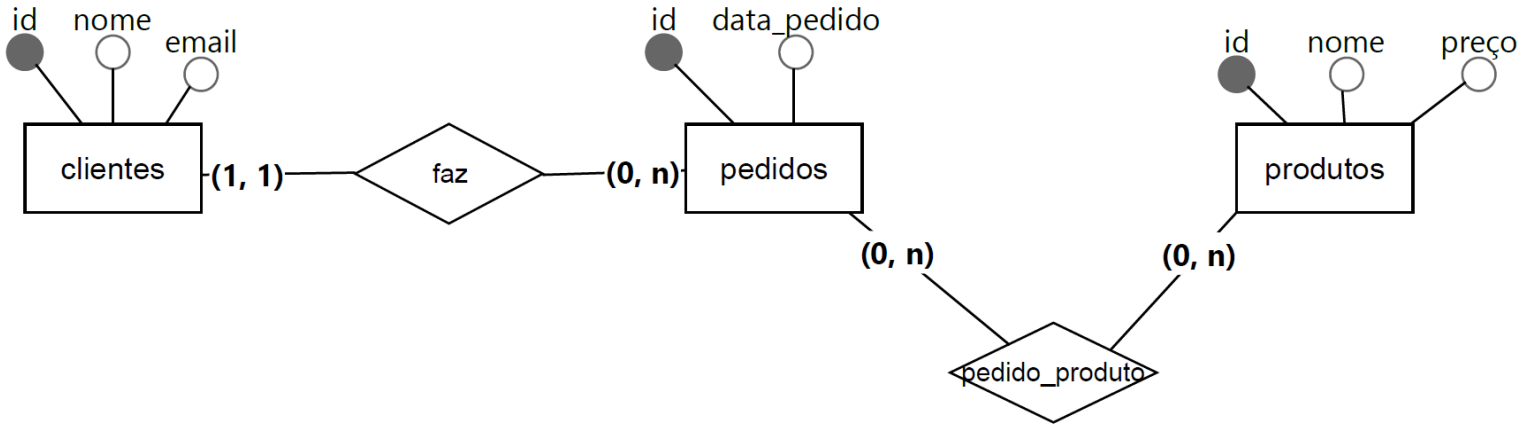
tbTrabalha (cod_emp, cod_companhia, salário)
cod_emp referencia tbEmpregado
cod_companhia referencia tbCompanhia

tbCompanhia (cod_companhia, nome_companhia, cidade_companhia)

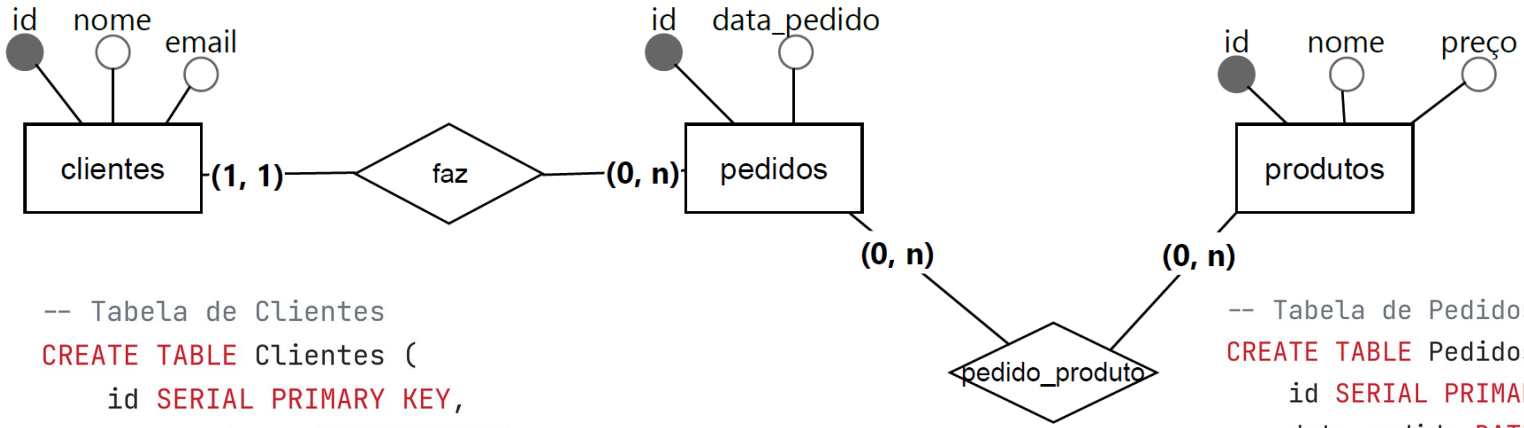
tbGerente (cod_gerente, nome_gerente)
```

Linguagem de Definição de Dados (DDL)

Linguagem de Definição de Dados (DDL)



Linguagem de Definição de Dados (DDL)

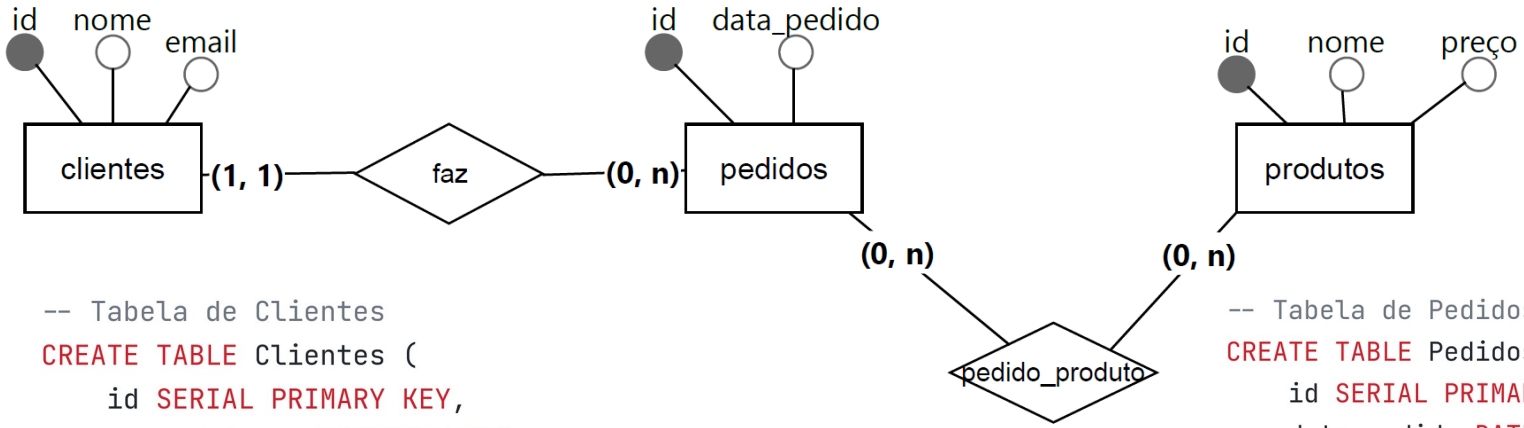



```sql
-- Tabela de Clientes
CREATE TABLE Clientes (
    id SERIAL PRIMARY KEY,
    nome_cliente VARCHAR(100),
    email_cliente VARCHAR(100)
);

-- Tabela de Produtos
CREATE TABLE Produtos (
    id SERIAL PRIMARY KEY,
    nome_produto VARCHAR(100),
    preco NUMERIC(10, 2)
);
```

```sql
-- Tabela de Pedidos
CREATE TABLE Pedidos (
    id SERIAL PRIMARY KEY,
    data_pedido DATE,
    cliente_id INT REFERENCES Clientes(id)
);
```

Linguagem de Definição de Dados (DDL)

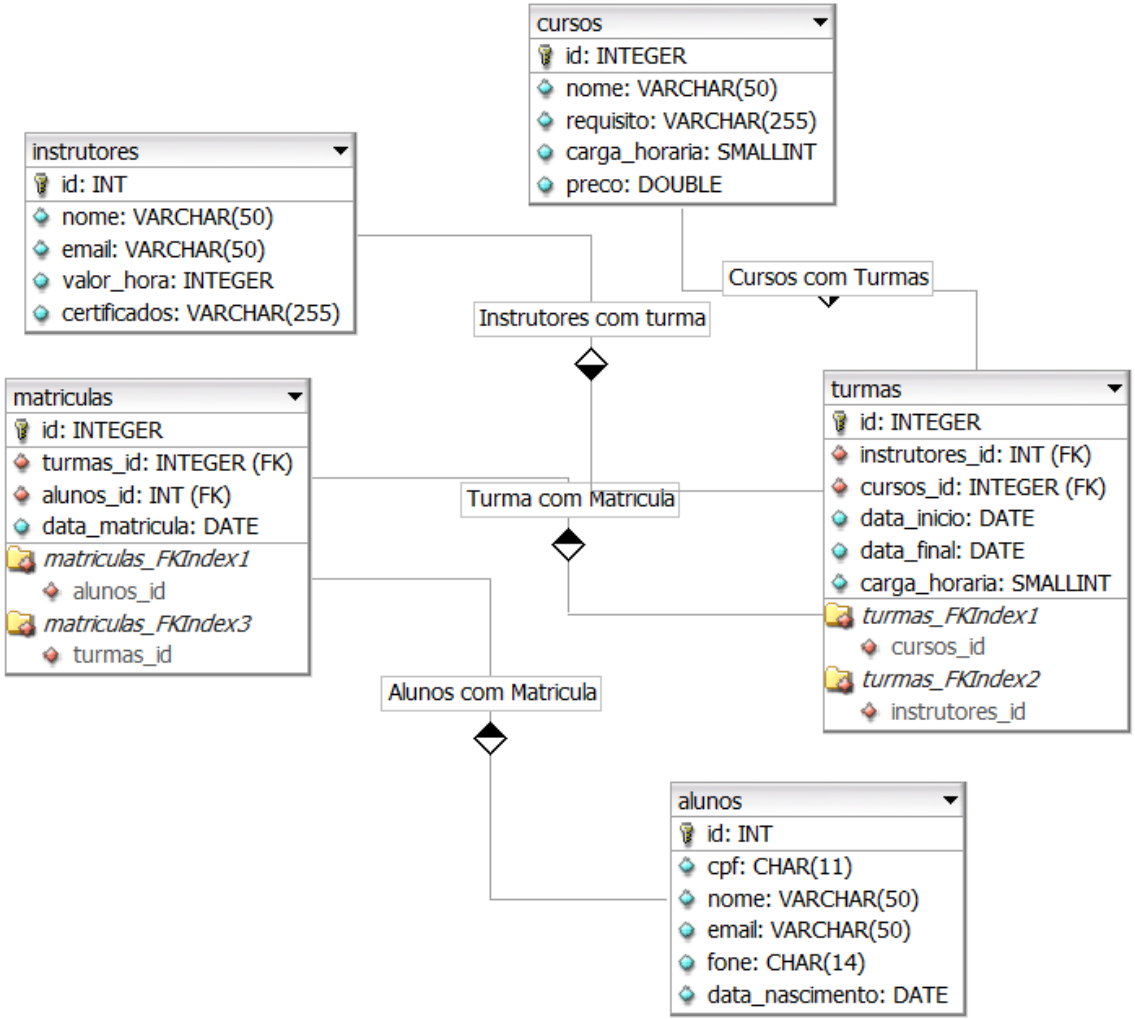



```sql
-- Tabela de Clientes
CREATE TABLE Clientes (
    id SERIAL PRIMARY KEY,
    nome_cliente VARCHAR(100),
    email_cliente VARCHAR(100)
);
```

```sql
-- Tabela de Produtos
CREATE TABLE Produtos (
    id SERIAL PRIMARY KEY,
    nome_produto VARCHAR(100),
    preco NUMERIC(10, 2)
);
```

```sql
-- Tabela de Pedidos
CREATE TABLE Pedidos (
    id SERIAL PRIMARY KEY,
    data_pedido DATE,
    cliente_id INT REFERENCES Clientes(id)
);
```

```sql
-- Tabela de Relacionamento entre Pedidos e Produtos (N:N)
CREATE TABLE Pedido_Produto (
    pedido_id INT REFERENCES Pedidos(id),
    produto_id INT REFERENCES Produtos(id),
    quantidade INT,
    PRIMARY KEY (pedido_id, produto_id)
);
```


cursos
id: INTEGER
nome: VARCHAR(50)
requisito: VARCHAR(255)
carga_horaria: SMALLINT
preco: DOUBLE
instrutores
id: INT
nome: VARCHAR(50)
email: VARCHAR(50)
valor_hora: INTEGER
certificados: VARCHAR(255)
Cursos com Turmas
Instrutores com turma
matriculas
id: INTEGER
turmas_id: INTEGER (FK)
alunos_id: INT (FK)
data_matricula: DATE
matriculas_FKIndex1
alunos_id
matriculas_FKIndex3
turmas_id
Turma com Matricula
turmas
id: INTEGER
instrutores_id: INT (FK)
cursos_id: INTEGER (FK)
data_inicio: DATE
data_final: DATE
carga_horaria: SMALLINT
turmas_FKIndex1
cursos_id
turmas_FKIndex2
instrutores_id
Alunos com Matricula
alunos
id: INT
cpf: CHAR(11)
nome: VARCHAR(50)
email: VARCHAR(50)
fone: CHAR(14)
data_nascimento: DATE

Linguagem de manipulação de dados (DML)

Introdução

Vamos criar o banco da aula de hoje

```
CREATE DATABASE escola;

CREATE TABLE alunos (
    aluno_id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    idade INTEGER,
    curso VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE notas (
    nota_id SERIAL PRIMARY KEY,
    aluno_id INTEGER NOT NULL,
    disciplina VARCHAR(100) NOT NULL,
    nota NUMERIC(5,2) NOT NULL,
    data_avaliacao DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (aluno_id) REFERENCES alunos(aluno_id)
);
```

Linguagem de Manipulação de Dados (DML)

*A linguagem de manipulação de dados (DML) é usada para **consultar** e **modificar** dados dentro das tabelas de um banco de dados.*

Os principais comandos DML são:

- **SELECT**: Recupera dados de uma tabela.
- **INSERT**: Insere novos dados em uma tabela.
- **UPDATE**: Modifica dados existentes em uma tabela.
- **DELETE**: Remove dados de uma tabela.

Linguagem de Manipulação de Dados (DML)

- **INSERT: Inserindo Dados**

O comando INSERT é usado para adicionar novas linhas a uma tabela.

Sintaxe Básica:

comandos.sql

```sql
INSERT INTO nome_da_tabela (coluna1, coluna2, ...)
VALUES (valor1, valor2, ...);
```

- **EXEMPLO**

  Inserir um novo aluno chamado Maria Silva, com 22 anos, no curso de Engenharia de Software.

comandos.sql

```sql
INSERT INTO Alunos (nome, idade, curso)
VALUES ('Maria Silva', 22, 'Engenharia de Software');
```

id	nome	idade	curso
1	Willian	30	Engenharia de Software
2	João	18	Engenharia de Software
3	Maria	22	Direito
4	José	18	Letras
5	Maria Silva	22	Engenharia de Software

Linguagem de Manipulação de Dados (DML)

- **SELECT: Recuperando Dados**

O comando SELECT é utilizado para consultar dados em uma tabela. É um dos comandos mais utilizados em SQL.

Sintaxe Básica:

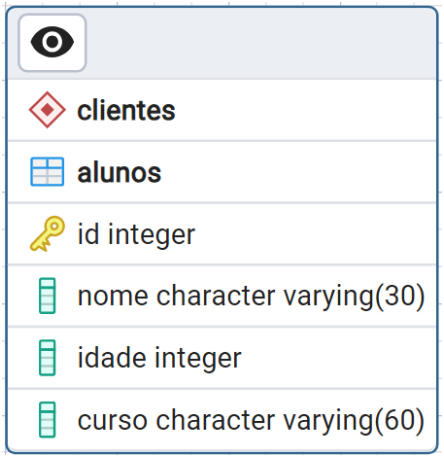
comandos.sql

```sql
SELECT coluna1, coluna2, ...
FROM nome_da_tabela
WHERE condição;
```

Linguagem de Manipulação de Dados (DML)

- **EXEMPLO**

  *Suponha que temos uma tabela chamada **Alunos** com as seguintes colunas: **id**, **nome**, **idade**, **curso**.*



id	nome	idade	curso
1	Willian	30	Engenharia de Software
2	João	18	Engenharia de Software
3	Maria	22	Direito
5	José	18	Letras

- **EXEMPLO**

  *Com o comando a seguir, podemos retornar os **nomes** e as **idades** dos alunos que estão matriculados no curso de **Engenharia de Software**.*

comandos.sql

```sql
SELECT nome, idade
FROM Alunos
WHERE curso = 'Engenharia de Software';
```

nome	idade
Willian	30
João	18

Linguagem de Manipulação de Dados (DML)

- **EXEMPLO**

Para selecionar todas as Colunas:

comandos.sql

```sql
SELECT *
FROM Alunos;
```

id	nome	idade	curso
1	Willian	30	Engenharia de Software
2	João	18	Engenharia de Software
3	Maria	22	Direito
5	José	18	Letras

- **UPDATE: Atualizando Dados**

O comando UPDATE é utilizado para modificar dados existentes em uma tabela.

Sintaxe Básica:

comandos.sql

```sql
UPDATE nome_da_tabela
SET coluna1 = valor1, coluna2 = valor2, ...
WHERE condição;
```

Linguagem de Manipulação de Dados (DML)

- **EXEMPLO**

Atualizar a idade da Maria Silva para 23 anos.

comandos.sql

```sql
UPDATE Alunos
SET idade = 23
WHERE nome = 'Maria Silva';
```

id	nome	idade	curso
1	Willian	30	Engenharia de Software
2	João	18	Engenharia de Software
3	Maria	22	Direito
4	José	18	Letras
5	Maria Silva	23	Engenharia de Software

- **DELETE: Removendo Dados**

O comando DELETE é usado para remover linhas de uma tabela.

Sintaxe Básica:

comandos.sql

```sql
DELETE FROM nome_da_tabela
WHERE condição;
```

Linguagem de Manipulação de Dados (DML)

- **EXEMPLO**

Remover o aluno chamado Maria Silva da tabela Alunos.

comandos.sql

```sql
DELETE FROM Alunos
WHERE nome = 'Maria Silva';
```

id	nome	idade	curso
1	Willian	30	Engenharia de Software
2	João	18	Engenharia de Software
3	Maria	22	Direito
4	José	18	Letras

Treinando SQL

Vamos criar alguns registros

- Insiram na tabela Alunos pelo menos 8 alunos com pelo menos 2 cursos diferentes
- Modifiquem o dado de um desses alunos
- Retornem os dados de todos os alunos da tabela
- Removam 1 aluno
- Salvem seus SQL, vamos precisar dessas informações na próxima aula

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software

Linguagem de manipulação de dados (Avançada)

Linguagem de Manipulação de Dados (DML)

*A linguagem de manipulação de dados (DML) é usada para **consultar** e **modificar** dados dentro das tabelas de um banco de dados.*

Os principais comandos DML são:

- **SELECT**: Recupera dados de uma tabela.
- **INSERT**: Insere novos dados em uma tabela.
- **UPDATE**: Modifica dados existentes em uma tabela.
- **DELETE**: Remove dados de uma tabela.

Funções de Agregação

*Funções de agregação realizam cálculos sobre um **conjunto de valores** e retornam um único valor.*

- **COUNT()**: Retorna o número de linhas.
- **SUM()**: Retorna a soma dos valores.
- **AVG()**: Retorna a média dos valores.
- **MAX()**: Retorna o valor máximo.
- **MIN()**: Retorna o valor mínimo.

SELECT Avançado

- **EXEMPLO -** *Retornar o número de alunos, a média de idade, a idade máxima e mínima por curso.*

Sintaxe Básica:

comandos.sql

```sql
SELECT curso, COUNT(*), AVG(idade), MAX(idade), MIN(idade)
FROM Alunos
GROUP BY curso;
```

SELECT Avançado

- **EXEMPLO -** *Retornar o número de alunos, a média de idade, a idade máxima e mínima por curso.*

Sintaxe Básica:

comandos.sql

```sql
SELECT curso, COUNT(*), AVG(idade), MAX(idade), MIN(idade)
FROM Alunos
GROUP BY curso;
```

curso	count	avg	max	min
Engenharia de Software	2	24,0000000000000000	30	18
Letras	1	18,0000000000000000	18	18
Direito	1	22,0000000000000000	22	22

SELECT Avançado

- **SUBCONSULTAS**

Subconsultas são consultas aninhadas dentro de outra consulta.

Sintaxe Básica:

comandos.sql

```sql
SELECT nome
FROM Alunos
WHERE idade > (SELECT AVG(idade) FROM Alunos);
```

SELECT Avançado

- **SUBCONSULTAS**

comandos.sql

```sql
SELECT nome
FROM Alunos
WHERE idade > (SELECT AVG(idade) FROM Alunos);
```

1	Willian	30	Engenharia de Software
2	João	18	Engenharia de Software
3	Maria	22	Direito
4	José	18	Letras

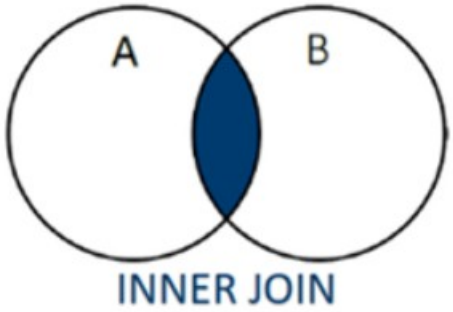
nome
Willian

SELECT Avançado

- **JOINS**
  - ***JOIN*** *é utilizado para* ***combinar informações*** *de duas ou mais tabelas*
  - Existem tipos diferentes de JOINs
    - INNER JOIN
    - LEFT JOIN
    - RIGHT JOIN
    - FULL JOIN

- **INNER JOIN**

  - *O **INNER JOIN** retorna apenas as linhas que têm correspondência em ambas as tabelas envolvidas na junção.*
  - ***Combina linhas de duas tabelas** onde há uma correspondência nos valores das colunas especificadas.*



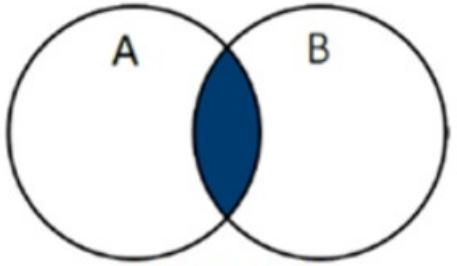
- **EXEMPLOS**

Alunos

id	nome	idade	curso_id
2	Bruno	22	102
3	Carlos	23	101
4	Daniela	20	103
1	Ana	21	101
5	Willian	31	(Null)

Cursos

id	nome
101	Engenharia de Software
102	Ciência da Computação
103	Administração



INNER JOIN

- **EXEMPLOS**

Alunos

id	nome	idade	curso_id
2	Bruno	22	102
3	Carlos	23	101
4	Daniela	20	103
1	Ana	21	101
5	Willian	31	(Null)

Cursos

id	nome
101	Engenharia de Software
102	Ciência da Computação
103	Administração

Como retornar apenas os alunos que estão matriculados em cursos que têm correspondência na tabela Cursos?

SELECT Avançado

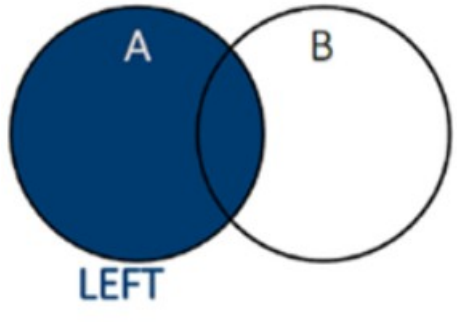
- **EXEMPLOS**

comandos.sql

```sql
SELECT alunos.nome, cursos.nome AS curso_nome
FROM alunos
INNER JOIN cursos ON alunos.curso_id = cursos.id;
```

nome	curso_nome
Bruno	Ciência da Computação
Carlos	Engenharia de Software
Daniela	Administração
Ana	Engenharia de Software

- **LEFT JOIN**
  - *O **LEFT JOIN** (ou **LEFT OUTER JOIN**) retorna todas as linhas da tabela **à esquerda** e as **correspondentes** da **tabela à direita**.*
  - *Para as linhas da tabela à esquerda que não têm correspondência na tabela à direita, os valores das colunas da tabela à direita serão NULL.*



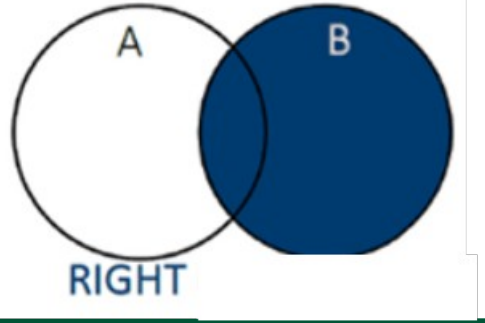
- **EXEMPLOS**

comandos.sql

```sql
SELECT alunos.nome, cursos.nome AS curso_nome
FROM alunos
LEFT JOIN cursos ON alunos.curso_id = cursos.id;
```

nome	curso_nome
Bruno	Ciência da Computação
Carlos	Engenharia de Software
Daniela	Administração
Ana	Engenharia de Software
Willian	(Null)

- **RIGHT JOIN**
  - *O **RIGHT JOIN** (ou **RIGHT OUTER JOIN**) retorna todas as linhas da **tabela à direita** e as correspondentes da **tabela à esquerda**.*
  - *Para as linhas da tabela à direita que não têm correspondência na tabela à esquerda, os valores das colunas da tabela à esquerda serão NULL.*



- **EXEMPLOS**

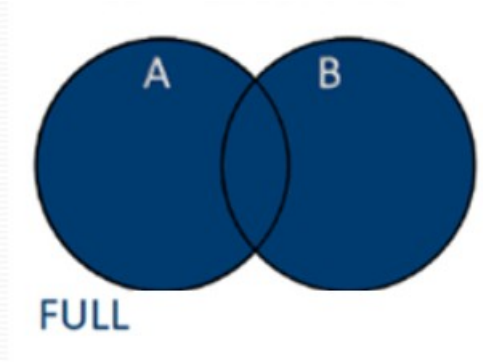
comandos.sql

```sql
SELECT alunos.nome, cursos.nome AS curso_nome
FROM alunos
RIGHT JOIN cursos ON alunos.curso_id = cursos.id;
```

nome	curso_nome
Bruno	Ciência da Computação
Carlos	Engenharia de Software
Daniela	Administração
Ana	Engenharia de Software
(Null)	Direito

- **FULL JOIN**

  - *O **FULL JOIN** (ou **FULL OUTER JOIN**) **retorna todas as linhas** quando há uma correspondência em uma das tabelas.*
  - *Retorna NULL para linhas sem correspondência em ambas as tabelas.*



SELECT Avançado

- **EXEMPLOS**

comandos.sql

```sql
SELECT alunos.nome, cursos.nome AS curso_nome
FROM alunos
FULL JOIN cursos ON alunos.curso_id = cursos.id;
```

nome	curso_nome
Bruno	Ciência da Computação
Carlos	Engenharia de Software
Daniela	Administração
Ana	Engenharia de Software
Willian	(Null)
(Null)	Direito

- **Exemplo com Subconsulta e INNER JOIN**

  - Suponha que temos as tabelas **Alunos**, **Cursos**, e **Professores**.

  - Queremos encontrar os **nomes dos alunos que estão matriculados em cursos ministrados por um professor específico**.

SELECT Avançado

- **EXEMPLOS**

Alunos

id	nome	idade	curso_id
2	Bruno	22	102
3	Carlos	23	101
4	Daniela	20	103
1	Ana	21	101
5	Willian	31	(Null)

Cursos

id	nome	professor_id
104	Direito	(Null)
101	Engenharia de Software	201
102	Ciência da Computação	202
103	Administração	203

Professores

id	nome
201	Prof. A
202	Prof. B
203	Prof. C

SELECT Avançado

- **EXEMPLOS**

comandos.sql

```sql
SELECT alunos.nome
FROM alunos
WHERE alunos.curso_id IN (
    SELECT cursos.id
    FROM cursos
    INNER JOIN professores ON cursos.professor_id = professores.id
    WHERE professores.nome = 'Prof. A'
);
```

nome
Carlos
Ana

Cláusulas Avançadas em SQL (CREATE)

Linguagem de Definição de Dados (DDL)

- **Criar uma Table (Tabela):**
  - Dentro de um schema, você pode criar tabelas para definir a estrutura onde os dados serão armazenados:

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

Sintaxe

```
CREATE TABLE clientes.informacoes (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    data_de_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Linguagem de Definição de Dados (DDL)

- Frequentemente criamos **relações** entre tabelas usando chaves estrangeiras (**FOREIGN KEY**).

- Essas relações são essenciais para **garantir a integridade referencial**, ou seja, assegurar que os dados relacionados permaneçam consistentes entre as tabelas.

- Há situações em que devemos considerar o que acontece quando os valores das chaves primárias (referenciadas por outras tabelas) mudam.

O que são ON DELETE e ON UPDATE?

- As cláusulas **ON DELETE** e **ON UPDATE** são usadas em conjunto com chaves estrangeiras (**FOREIGN KEY**) em SQL.

- Elas definem o comportamento do banco de dados quando a linha referenciada por uma chave estrangeira é deletada (**ON DELETE**) ou atualizada (**ON UPDATE**).

- Essas cláusulas ajudam a manter a **integridade referencial** e garantem que as relações entre tabelas permaneçam consistentes.

Quando usar ON DELETE e ON UPDATE?

- **ON DELETE:** define o que acontece com os registros dependentes (na “tabela filha”) quando uma linha da “tabela pai” (referenciada pela chave primária/única) é excluída.

- **ON UPDATE:** define o que acontece com os registros dependentes (na tabela filha) quando **atualizar o valor da chave primária na “tabela pai”**

Sintaxe Básica

comandos.sql

```sql
CREATE TABLE nome_da_tabela (
    coluna1 tipo_de_dado CONSTRAINTS,
    coluna2 tipo_de_dado CONSTRAINTS,
    ...
    FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
    ON DELETE ação
    ON UPDATE ação
);
```

Possibilidades para ON DELETE e ON UPDATE

➔ CASCADE

- **Descrição:** Quando uma linha referenciada é deletada ou atualizada, todas as linhas na tabela que possuem a chave estrangeira correspondente são deletadas ou atualizadas automaticamente.

- **Quando usar:** Use CASCADE quando a existência das linhas dependentes (referenciadas pela chave estrangeira) não fizer sentido sem a linha pai.

comandos.sql

```
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE CASCADE
ON UPDATE CASCADE
```

Possibilidades para ON DELETE e ON UPDATE

➔ SET NULL

- **Descrição:** Quando uma linha referenciada é deletada ou atualizada, as colunas de chave estrangeira nas linhas dependentes são definidas como NULL.

- **Quando usar:** Use SET NULL quando quiser manter as linhas dependentes, mas remover a referência à linha pai.

comandos.sql

```sql
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE SET NULL
ON UPDATE SET NULL
```

Possibilidades para ON DELETE e ON UPDATE

➔ SET DEFAULT

- **Descrição:** Quando uma linha referenciada é deletada ou atualizada, as colunas de chave estrangeira nas linhas dependentes são definidas com um valor padrão especificado.

- **Quando usar:** Use SET DEFAULT quando quiser que as linhas dependentes assumam um valor padrão que indica uma condição especial.

comandos.sql

```
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE SET DEFAULT
ON UPDATE SET DEFAULT

```

Possibilidades para ON DELETE e ON UPDATE

➔ RESTRICT

- **Descrição:** Impede a deleção ou atualização de uma linha referenciada se existem linhas dependentes que fazem referência a ela.
- **Quando usar:** Use RESTRICT para evitar a perda acidental de integridade referencial.

comandos.sql

```sql
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE RESTRICT
ON UPDATE RESTRICT
```

Possibilidades para ON DELETE e ON UPDATE

➔ NO ACTION

- **Descrição:** Similar a RESTRICT, mas a verificação é diferida até o final da transação. É o comportamento padrão se nenhuma ação for especificada.

- **Quando usar:** Use NO ACTION quando quiser que as verificações de integridade referencial ocorram após a execução de outras operações

comandos.sql

```
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE NO ACTION
ON UPDATE NO ACTION
```

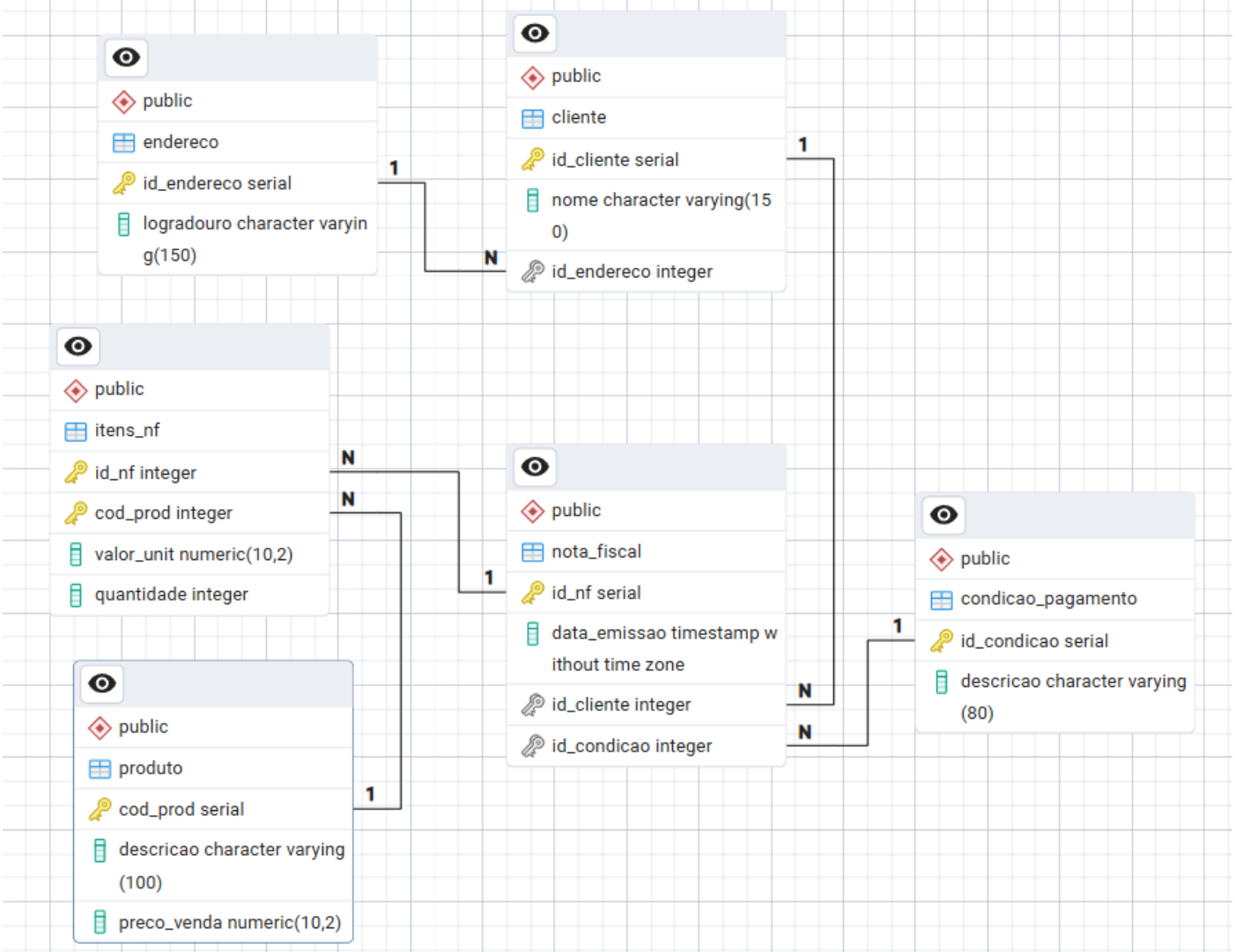
Possibilidades para ON DELETE e ON UPDATE

➔ NO ACTION

- No **PostgreSQL** (e na maioria dos SGBDs relacionais), quando você **não especifica nada** no ON DELETE ou ON UPDATE o **padrão** é NO ACTION

- **RESTRICT** → A verificação é **imediata**: o SGBD barra a operação na hora que encontra o conflito.

- **NO ACTION** → A verificação é **adiada** até o final da instrução ou da transação.


public
endereco
id_endereco serial
logradouro character varyin
g(150)
1
N
public
cliente
id_cliente serial
nome character varying(15
0)
id_endereco integer
1
public
itens_nf
id_nf integer
cod_prod integer
valor_unit numeric(10,2)
quantidade integer
N
N
1
public
nota_fiscal
id_nf serial
data_emissao timestamp w
ithout time zone
id_cliente integer
id_condicao integer
N
N
1
public
condicao_pagamento
id_condicao serial
descricao character varying
(80)
1
public
produto
cod_prod serial
descricao character varying
(100)
preco_venda numeric(10,2)

Cláusulas Avançadas em SQL utilizadas com SELECT

Cláusulas

- HAVING
- ORDER BY
- LIMIT e OFFSET
- DISTINCT
- UNION, UNION ALL
- WINDOW FUNCTIONS

HAVING

- A cláusula HAVING é usada para filtrar grupos de linhas criados pela cláusula GROUP BY.

- Funciona de maneira semelhante à cláusula WHERE, mas é aplicada após a agregação.

```sql
SELECT
    i.id_nf,
    SUM(i.valor_unit * i.quantidade) AS total_nf
FROM itens_nf i
GROUP BY i.id_nf
HAVING SUM(i.valor_unit * i.quantidade) > 20;
```

ORDER BY

- A cláusula ORDER BY é usada para ordenar os resultados de uma consulta em ordem crescente (ASC) ou decrescente (DESC).

```sql
SELECT
    id_cliente,
    nome,
    id_endereco
FROM cliente
ORDER BY nome ASC;
```

LIMIT

- A cláusula LIMIT é usada para limitar o número de linhas retornadas pela consulta.

```sql
SELECT
    id_cliente,
    nome
FROM cliente
ORDER BY nome ASC
LIMIT 5;
```

LIMIT e OFFSET

- A cláusula OFFSET é usada para pular um número específico de linhas antes de começar a retornar as linhas da consulta, usada junto com o limit cria paginação

- Esse comando retorna apenas 5 registros, começando a partir do 6º cliente.

```sql
SELECT
    id_cliente,
    nome
FROM cliente
ORDER BY nome ASC
LIMIT 5 OFFSET 5;
```

DISTINCT

- A cláusula DISTINCT é usada para remover duplicatas das linhas retornadas pela consulta.

```
SELECT DISTINCT id_endereco
FROM cliente;
```

UNION e UNION ALL

- A cláusula UNION é usada para combinar os resultados de duas ou mais consultas SELECT, removendo duplicatas.

- A cláusula UNION ALL faz o mesmo, mas mantém todas as duplicatas.

comandos.sql

```sql
SELECT nome FROM Alunos
UNION
SELECT nome FROM Professores;

SELECT nome FROM Alunos
UNION ALL
SELECT nome FROM Professores;
```

```sql
SELECT nome AS dado
FROM cliente
UNION
SELECT logradouro
FROM endereco;
```

WINDOW FUNCTIONS

- Funções de janela (window functions) permitem que você realize cálculos sobre um conjunto de linhas relacionadas à linha atual. São usadas com a cláusula OVER.

- Diferem das funções de agregação normais (SUM, AVG, etc.) porque não colapsam o resultado em uma única linha por grupo; elas mantêm cada linha original, mas adicionam uma coluna calculada.

WINDOW FUNCTIONS

- O exemplo abaixo calcula a idade média dos alunos para cada curso (curso_id) e adiciona essa média como uma coluna avg_idade na tabela de resultados.

comandos.sql

```sql
SELECT nome, curso_id,
       AVG(idade) OVER (PARTITION BY curso_id) AS avg_idade
FROM Alunos;
```

WINDOW FUNCTIONS

```sql
SELECT
    id_nf,
    cod_prod,
    (quantidade * valor_unit) AS valor_total,
    AVG(valor_unit * quantidade)
        OVER (PARTITION BY id_nf) AS media_nf
FROM itens_nf;
```

Views

O que são Views?

- Uma view é uma tabela virtual baseada no resultado de uma consulta SQL.

- Ela não armazena dados fisicamente, mas sim a consulta que gera os dados.

- As views permitem que você crie uma camada de abstração sobre suas tabelas, facilitando a consulta e a manipulação de dados.

Suponha as Seguintes Tabelas

comandos.sql

```sql
CREATE VIEW nome_da_view AS
SELECT colunas
FROM tabela
WHERE condição;
```

Suponha as seguintes Tabelas

alunos

id	nome	idade	curso_id
1	Ana	21	101
2	Bruno	22	102
3	Carlos	23	101
4	Daniela	20	103

cursos

id	nome
101	Engenharia de Software
102	Ciência da Computação
103	Administração

Criando uma View

comandos.sql

```sql
CREATE VIEW AlunosComCursos AS
SELECT Alunos.id, Alunos.nome, Alunos.idade, Cursos.nome AS curso_nome
FROM Alunos
INNER JOIN Cursos ON Alunos.curso_id = Cursos.id;
```

id	nome	idade	curso_nome
2	Bruno	22	Ciência da Computação
3	Carlos	23	Engenharia de Software
4	Daniela	20	Administração
1	Ana	21	Engenharia de Software

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software



Data Control Language

Linguagem de Controle de Dados (DCL)

- É um subconjunto da linguagem SQL, voltado ao controle de permissões e segurança no banco de dados, ou seja, define quem pode acessar, alterar ou manipular determinados objetos (tabelas, views, procedures, etc.).

- **Comandos**

  CREATE ROLE

  ALTER ROLE

  DROP ROLE

  GRANT

  REVOKE

CREATE ROLE

- Gerenciar acesso e permissões é essencial em qualquer sistema de banco de dados. O PostgreSQL oferece um modelo flexível e poderoso baseado em **usuários e funções (roles)**.

- Contudo, no PostgreSQL, usuários e roles (funções) não são entidades completamente separadas.

- Tecnicamente, o PostgreSQL trata tudo como uma função . A diferença está em se uma função tem privilégios de login ou não.

CREATE ROLE

- **Role**: Um conceito mais amplo que pode representar usuários, grupos ou conjuntos de privilégios. Funções podem ser concedidas a outras funções, permitindo um gerenciamento flexível de permissões.

```
CREATE ROLE readonly;
CREATE ROLE admin_role;
CREATE ROLE sales_user;
```

CREATE ROLE

- **Usuário**: Uma função com privilégios de login. Pode se autenticar e executar comandos conforme seus privilégios.

```
CREATE ROLE john LOGIN PASSWORD 'secure_password';
```

```
CREATE ROLE leo
WITH
  LOGIN
  PASSWORD 'secure_password'
  VALID UNTIL '2025-12-31'
  CONNECTION LIMIT 5;
```

GRANT

- O GRANT no PostgreSQL (e em outros bancos de dados SQL) é o comando utilizado para conceder permissões a um usuário ou role

- Também pode ser usado para atribuir um role a um usuário, algo como atribuir perfis de usuários

- Ele define quem pode realizar determinadas operações (como SELECT, INSERT, UPDATE, DELETE, etc.) sobre objetos como tabelas, views, funções ou esquemas.

- SINTAXE: GRANT permissao ON nivel TO role_name

GRANT

GRANT SELECT ON produtos **TO** readonly;

GRANT readonly **TO** john;

GRANT SELECT, **INSERT**, **UPDATE ON** clientes **TO** john;

-- Concede permissão para acessar (USAGE) e criar objetos (CREATE) dentro do schema public.
-- OBS: USAGE não lê nem lista dados; apenas permite utilizar o schema.
GRANT USAGE, **CREATE ON SCHEMA** vendas **TO** sales_user;

GRANT ALL PRIVILEGES **ON ALL** TABLES **IN SCHEMA** public **TO** admin_role;

GRANT ALL PRIVILEGES **ON** DATABASE meu_banco **TO** admin_role;

GRANT

- Importante: conceder privilégios em nível de DATABASE (ex.: GRANT ALL ON DATABASE ...) não concede automaticamente permissões sobre as tabelas, views ou sequências existentes.

- Os privilégios dos níveis DATABASE, TABLE, SCHEMA e SEQUENCE devem ser atribuídos separadamente.

- Ou seja, os níveis de permissão são independentes: um usuário pode ter acesso ao banco, mas continuar sem permissão para consultar ou alterar tabelas.

REVOKE

- É utilizado para revogar (retirar) permissões anteriormente concedidas a usuários ou roles no banco de dados.

- Serve para restringir acessos ou ajustar privilégios, garantindo maior controle e segurança sobre os objetos do banco.

- Sintaxe básica:

REVOKE <PRIVILÉGIOS> ON <OBJETO> FROM <ROLE>;

REVOKE

REVOKE INSERT, UPDATE ON clientes **FROM** john**;**

REVOKE ALL PRIVILEGES ON produtos **FROM** readonly**;**

REVOKE SELECT ON vendas **FROM** operador**;**

REVOKE ALL PRIVILEGES ON DATABASE meu_banco **FROM** admin_role;

ALTER ROLE

- O comando ALTER ROLE é utilizado para modificar propriedades de uma role (usuário ou grupo) já existente no PostgreSQL.

- Ele pode alterar permissões gerais do usuário, como:

LOGIN ou NOLOGIN – permitir ou impedir acesso ao banco
SUPERUSER – conceder privilégios administrativos totais
CREATEDB – permitir criar bancos de dados
CREATEROLE – permitir criar novas roles
PASSWORD – alterar a senha de acesso
VALID UNTIL – definir data de expiração

ALTER ROLE

- **Importante:** ALTER ROLE não altera privilégios sobre objetos existentes (tabelas, esquemas, bancos). Para isso, ainda é necessário usar GRANT / REVOKE.

- **Sintaxe básica:**

ALTER ROLE nome_da_role [WITH] opção1 opção2 ... ;

REVOKE

```
-- Alterar a senha de uma role existente.
ALTER ROLE leo WITH PASSWORD 'nova_senha';

-- Eleva o usuário 'leo' ao nível de SUPERUSER.
-- Um SUPERUSER no PostgreSQL ignora todas as regras de permissões (GRANT/REVOKE), passando a ter acesso irrestrito a todos os bancos, schemas e tabelas.
ALTER ROLE leo WITH SUPERUSER;

-- Alterar validade da role.
ALTER ROLE leo VALID UNTIL '2026-12-31';

-- Ajustar limite de conexões permitidas.
ALTER ROLE leo CONNECTION LIMIT 10;
```

DROP ROLE

- O comando DROP ROLE é usado para excluir uma role (usuário ou grupo) do PostgreSQL.

- Você só pode remover uma role se:

- Ela não tiver privilégios ativos
- Ela não estiver atribuída a outra role

DROP ROLE sales_user;

PG_ROLE

- pg_roles é uma view que lista todas as roles existentes no PostgreSQL, incluindo usuários e grupos.

- Ela mostra informações como:

- Nome da role (rolname)
- Se pode fazer login (rolcanlogin)
- Se é superusuário (rolsuper)
- Se pode criar bancos (rolcreatedb)
- Se pode criar outras roles (rolcreaterole)
- Data de expiração (rolvaliduntil)

```
SELECT * FROM pg_roles;
```

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software

Indexação e Estruturas de Índices

O que são índices

- São estruturas de dados que organizam referências as localizações dos dados reais das tabelas.
- Por analogia, os índices nos banco de dados são similares aos índices apresentados em livros

- **Importância:**

- Lembre-se, que conforme o banco de dados cresce, em um determinado momento o desempenho das consultas tende a cair, não sendo mais satisfatório
- Os índices são utilizados para melhorar o tempo de execução das consultas, sendo recursos bastante utilizados para a resolução desse problema

Tabelas (padrão)

- Tabela é um conjunto desordenado de **tuplas** exclusivas.

- Por padrão, os dados são armazenados sem uma ordem particular, isto significa que:

- A adição de novos registo não necessita de uma reorganização dos dados existentes → melhor desempenho na inserção de dados

- A recuperação de informações é feita por busca por cada registro presente na tabela → maior tempo para a obtenção da informação.

Tabelas organizadas por índice

- Com os índices temos a nossa disposição uma estrutura adicional de organização dos dados de todas as colunas das tabelas, melhorando a pesquisa de informações

- Mas precisamos ter cuidado com a quantidade de índices por tabela, isso devido a sobrecarga durante as operações DML (INSERT, UPDATE e DELETE), pois quando cada uma das operações realizadas, é necessária a atualização dos índices

```
CREATE INDEX nome_index ON nome_tabela (nome_coluna);
```

```
CREATE INDEX idx_produtos_nome ON produtos(nome);
```

Tipos de índices

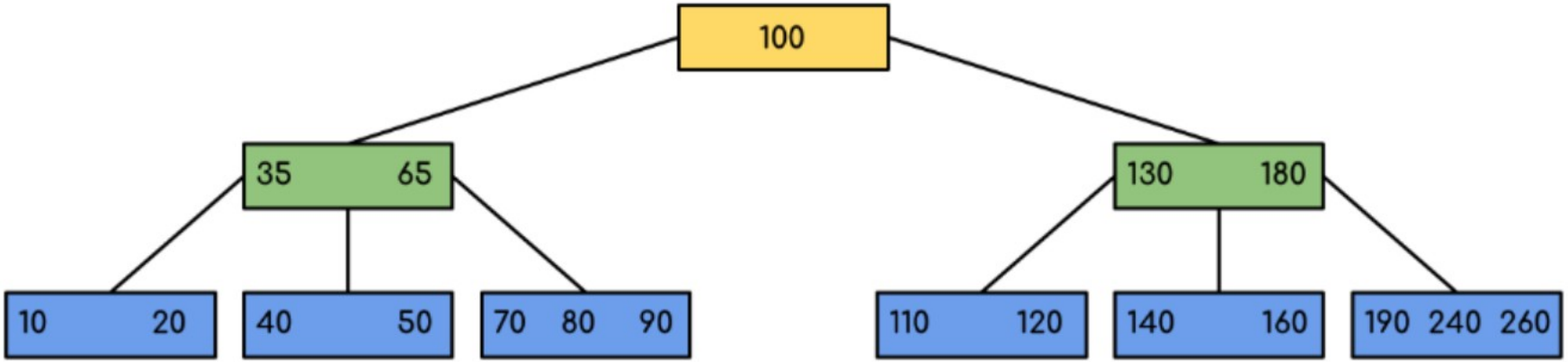
- No PostgreSQL, temos a nossa disposição vários tipos de índices, entre eles:
- B-tree
- Hash
- GiST
- GIN.

- Estes usam algoritmos diferentes que são utilizados para diferentes tipos de consultas.

B-tree

- Padrão utilizado no CREATE INDEX
- Utiliza árvore balanceada: organiza os valores em uma estrutura de árvore com vários ramos que se reequilibram automaticamente para garantir performance constante nas consultas.
- Pode operar todos os tipos de dados,
- Pode ser utilizado para recuperar valores nulos.
- Este tipo de índice é projetado para trabalhar muito bem com cache

B-tree



Hash

- É mais utilizado em bancos de dados não relacionais.
- Utilizam uma função de hash para converter um valor (ex: CPF, e-mail, código) em um código numérico fixo

```
CPF (123.456.789-00)  →  função hash →  8F23AB
```

- Na busca, em vez de comparar valor por valor, o SGBD compara os códigos hash, o que é muito mais rápido — mas só funciona para igualdade (=).

Hash

Situação	Índice Hash funciona bem?
`WHERE cpf = '123'`	✅ Excelente
`WHERE email = 'x@y.com'`	✅ Excelente
`WHERE idade BETWEEN 20 AND 30`	❌ Não funciona (sem range)
`ORDER BY nome`	❌ Não ajuda
`LIKE 'Jo%'`	❌ Não funciona

- Indices hash são especialmente úteis para buscas por igualdade em colunas com valores aleatórios ou não sequenciais, como CPFs, e-mails, UUIDs, etc

Hash

- Até versões anteriores ao PostgreSQL 10, índices hash não garantiam transações seguras, exigindo reconstrução manual em caso de falhas.

- Nas versões mais recentes, passaram a ser seguros, mas ainda são pouco utilizados, já que os índices B-Tree costumam oferecer desempenho semelhante com mais flexibilidade.

```
CREATE INDEX nome_index ON nome_tabela  USING hash (nome_coluna);

CREATE INDEX idx_clientes_email_hash ON clientes USING hash (email);

SELECT * FROM clientes WHERE email = ‘aaaa@gmail.com’;
```

GIN

- GIN (Generalized Inverted Indexes) foi projetado para lidar com casos em que os itens a serem indexados são valores compostos e as consultas a serem tratadas pelo índice precisam buscar valores de elementos que aparecem dentro dos itens compostos.

- Diferente do B-Tree que são otimizados para valores chave únicos

id	tags
1	{'promoção', 'verão'}
2	{'inverno', 'promoção'}
3	{'verão'}

GIN

- O índice GIN funciona como uma estrutura semelhante a uma árvore invertida (árvore de componentes), que armazena todos os valores possíveis encontrados dentro das colunas indexadas.

- Para cada valor encontrado, o índice mantém uma lista (ou conjunto) de ponteiros para todas as linhas da tabela onde aquele valor aparece.

- Por isso, uma única linha da tabela pode aparecer várias vezes dentro do índice,

GIN

- 'verão' → {linha 1, linha 3}

id	tags
1	{'promoção', 'verão'}
2	{'inverno', 'promoção'}
3	{'verão'}

CREATE INDEX nome_index ON nome_tabela **USING GIN** (nome_coluna);

GIN

-- habilitar extensão uma vez no banco
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- busca por pedaços do nome do produto (ex.: ILIKE '%aço%')
-- gin_trgm_ops permite criar índices GIN baseados em trigramas
CREATE INDEX idx_produtos_nome_trgm ON produtos **USING gin** (nome gin_trgm_ops);

--Busca todos os produtos cujo nome contém a palavra "aço" em qualquer posição.
--ILIKE é semelhante ao LIKE, mas ignora maiúsculas e minúsculas.
SELECT produto_id, nome **FROM** produtos **WHERE** nome **ILIKE** '%caixa%';

-- Usa o operador de similaridade (%) oferecido pela extensão pg_trgm.
--Retorna registros onde o texto em nome é similar ou se parece com 'cimento', mesmo que não seja exatamente igual.
SELECT * **FROM** produtos **WHERE** nome % 'camisa';

GIN

```
CREATE TABLE catags( id SERIAL PRIMARY KEY, tags TEXT[] -- coluna com array de texto );
INSERT INTO catags (tags) VALUES (ARRAY['promoção', 'verão']);
INSERT INTO catags (tags) VALUES (ARRAY['inverno', 'liquidação']);
INSERT INTO catags (tags) VALUES (ARRAY['infantil', 'novidade']);
INSERT INTO catags (tags) VALUES (ARRAY['esporte', 'leve', 'confortável']);
INSERT INTO catags (tags) VALUES (ARRAY['casual']);
INSERT INTO catags (tags) VALUES (ARRAY['luxo', 'exclusivo', 'premium']);
INSERT INTO catags (tags) VALUES (ARRAY['tecnologia', 'lançamento']);
INSERT INTO catags (tags) VALUES (ARRAY['ecológico', 'reciclável']);
INSERT INTO catags (tags) VALUES (ARRAY['importado', 'alta performance']);
INSERT INTO catags (tags) VALUES (ARRAY['básico', 'cotidiano’]);
CREATE INDEX idx_catags_tags_gin ON catags USING gin (tags);
--@> significa contém
-- os dois vão retornar o mesmo
SELECT * FROM catags WHERE tags @> ARRAY['promoção'];
SELECT * FROM catags WHERE 'promoção' = ANY(tags);
-- Busca itens que contenham AO MESMO TEMPO 'promoção' E 'verão'
SELECT * FROM catags WHERE tags @> ARRAY['promoção', 'verão'];
-- Busca itens que contenham 'promoção' OU 'verão'
SELECT * FROM catags WHERE 'promoção' = ANY(tags) OR 'verão' = ANY(tags);
--Verifica se existe algum elemento em comum entre os dois lados
SELECT * FROM catags WHERE tags && ARRAY['inverno', 'luxo’];11
```

GiST

- GiST é uma abreviação de “árvore de busca generalizada”.
- Esta é uma árvore de busca balanceada, assim como o b-tree
- Diferente da B-Tree, o GiST não depende de operadores de comparação tradicionais (<, >, =).

GiST

- O GiST é útil quando para os tipos de dados armazenados os operadores de comparação tradicionais simplesmente não fazem sentido, ou seja, dados que não têm ordem natural, como:

- Coordenadas geográficas (PostGIS)
- Intervalos de datas (ranges)
- Imagens

- O GiST é flexível e ideal para estes tipos de dados pois:

- Permite definir uma regra para distribuir dados de um tipo arbitrário em uma árvore balanceada
- Permite definir um método para usar essa representação para acesso por algum operador.

GiST

- Pense o seguinte, para criar um método de indexação do zero, um desenvolvedor precisaria dominar profundamente o funcionamento interno do SGBD.
- Isso inclui definir como os dados serão estruturados na memória, como serão armazenados em disco e ainda implementar mecanismos de controle de concorrência e bloqueio para garantir segurança em acessos simultâneos.
- O GiST funciona como um “template de índice”: ele cuida de toda a parte estrutural e o desenvolvedor só precisa informar como comparar os dados. Por isso ele é considerado um framework para criação de novos métodos de acesso.

```
CREATE INDEX nome_index ON nome_tabela  USING gist (nome_coluna);
```

GiST

```
CREATE TABLE reservas (
  id SERIAL,
  periodo daterange
);

INSERT INTO reservas (periodo) VALUES
(daterange('2025-01-05', '2025-01-12')),   -- sobrepõe
(daterange('2025-01-10', '2025-01-15')),   -- sobrepõe
(daterange('2025-01-01', '2025-01-08')),   -- NÃO sobrepõe
(daterange('2025-01-18', '2025-01-25')),   -- sobrepõe (final)
(daterange('2025-02-01', '2025-02-10')),   -- NÃO sobrepõe
(daterange('2025-01-19', '2025-01-21'));   -- sobrepõe (final)

CREATE INDEX idx_reservas_gist ON reservas USING gist (periodo);
```

- && significa “tem interseção?”
- Verifica se há algum elemento em comum entre os dois lados.

```
SELECT * FROM reservas WHERE periodo && daterange('2025-01-10', '2025-01-20');
```

GiST

```
-- Criar tabela de locais com coordenadas
CREATE TABLE locais (
    id SERIAL PRIMARY KEY,
    nome TEXT NOT NULL,
    coordenada POINT -- formato: (longitude, latitude)
);

-- Inserir alguns pontos de exemplo (coordenadas fictícias)
INSERT INTO locais (nome, coordenada) VALUES
('Restaurante Central',      POINT(-49.25, -16.68)),
('Supermercado Popular',     POINT(-49.23, -16.67)),
('Posto de Gasolina',        POINT(-49.30, -16.70)),
('Praça Municipal',          POINT(-49.24, -16.65)),
('Escola Estadual',          POINT(-49.20, -16.69)),
('Hospital Regional',        POINT(-49.28, -16.72));

CREATE INDEX idx_locais_gist ON locais USING gist (coordenada);

-- <-> é o operador de “distância” -> Traga os 3 locais mais próximos da coordenada (-49.26, -16.69).
SELECT nome, coordenada
FROM locais
ORDER BY coordenada <-> POINT(-49.26, -16.69)
LIMIT 3;
```

Índices em colunas múltiplas

- Um índice pode ser criado utilizando mais de uma coluna da tabela.
- B-Tree multicoluna: pode ser utilizado em consultas que envolvem qualquer subconjunto das colunas, mas funciona melhor quando a primeira coluna do índice é usada na condição de busca.
- GiST e GIN multicoluna: são mais flexíveis na combinação das colunas — a consulta pode usar qualquer uma delas e ainda aproveitar o índice.

```
CREATE INDEX nome_index ON nome_tabela  (coluna1, coluna2);

CREATE INDEX produtos_mult_index ON produtos (codigo_produto, preco);

SELECT * FROM produtos WHERE codigo_produto < 400 AND preco < 300;
```

Índices concorrentes

- Ao criar um índice de forma tradicional, o PostgreSQL bloqueia a tabela para escritas (INSERT, UPDATE, DELETE) até que o índice seja finalizado.

- Em tabelas grandes, esse processo pode demorar bastante, causando impacto em sistemas em produção.

- Para evitar essa interrupção, o PostgreSQL oferece a opção de criação concorrente de índices, por meio da cláusula CONCURRENTLY.

- Ela permite que o índice seja criado sem bloquear operações de escrita, o que torna essa abordagem ideal para ambientes que precisam continuar funcionando enquanto o índice é construído.

```
CREATE INDEX CONCURRENTLY nome_index ON tabela (coluna);
```

REINDEX

- Por fim, temos a reindexação, onde quando um índice torna-se ineficiente devido ao inchaço, a utilização da reindexação passa a ser uma opção favorável para a obtenção de um máximo desempenho dos índices

```
REINDEX TABLE nome_tabela;
```

```
CREATE INDEX produtos_mult_index ON produtos (codigo_produto, preco);
```

```
SELECT * FROM produtos WHERE codigo_produto < 400 AND preco < 300;
```

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software

CONTROLE DE TRANSAÇÃO

INTRODUÇÃO

- Uma transação é uma série de operações sequenciais realizadas no Banco de Dados.

- As operações da transação compõem uma unidade de trabalho, ou seja, é como se fossem uma coisa só.

- Existem 4 propriedades das transações ACID: atomicidade, consistência, isolamento e durabilidade

PROPRIEDADES DAS TRANSAÇÕES

- A atomicidade trata as transações como um conjunto, de forma atômica. Em outras palavras, ou a transação será executada totalmente ou não será executada. “É o tudo ou nada”

- A consistência prega que o Banco de Dados saia de um estado consistente para outro também consistente, após a realização de uma transação.

PROPRIEDADES DAS TRANSAÇÕES

- Imagine que precisamos realizar em uma única transação duas operações de manipulação de dados (DML, do inglês Data Manipulation Language), como INSERT e UPDATE.

- Estas operações só podem se tornar permanentes no banco de dados se todas forem executadas com sucesso.

- Em caso de falhas em uma das duas é possível cancelar a transação, porém todas modificações realizadas durante a mesma serão descartadas, inclusive a que obtive sucesso.

- Assim, os dados permanecem íntegros e consistentes.

PROPRIEDADES DAS TRANSAÇÕES

- O isolamento trata as transações de forma independente, isoladas umas das outras. A ideia é que o funcionamento de uma transação não interfira na outra, visando minimizar conflitos por concorrências.

- A durabilidade é a persistência das transações no Banco de Dados, após a sua efetivação. Em outras palavras, uma vez que as transações foram confirmadas, elas não podem ser desfeitas por erros futuros.

BEGIN COMMIT ROLLBAK

```
BEGIN;

CREATE TABLE cliente (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100)
);

CREATE TABLE pedido (
    id SERIAL PRIMARY KEY,
    cliente_id INT REFERENCES cliente(id),
    data_pedido DATE DEFAULT CURRENT_DATE
);

COMMIT;
```

FUNÇÕES E PROCEDIMENTOS

INTRODUÇÃO

- **Funções** e **Procedimentos** são ambos blocos de código armazenados que podem ser executados no servidor do banco de dados

- servem para automatizar cálculos, regras de negócio ou transformações de dados, etc

- mas possuem diferenças fundamentais.

FUNÇÕES

- Podem retornar um valor (ou conjunto de valores).
- Podem ser usadas em consultas SQL (SELECT, WHERE, etc.).
- Executadas dentro de uma única transação.
- Não podem executar comandos de controle de transação (COMMIT, ROLLBACK).

FUN

Você pode chamar uma função diretamente dentro de um SELECT, como se fosse um campo para retorno, ou usá-la para filtrar dados no WHERE. Isso permite reaproveitar regras e cálculos com facilidade.

- Podem ser usadas em consultas SQL (SELECT, WHERE, etc.).
- Executadas dentro de uma única transação.
- Não podem executar comandos de controle de transação (COMMIT, ROLLBACK).

FUNÇÕES

Uma unidade lógica de trabalho que contém uma ou mais **operações** (como *inserções, atualizações ou exclusões*) que devem ser executadas de forma atômica - ou seja, ou todas as operações são realizadas com sucesso, ou nenhuma delas é efetivada.

Se der erro no meio, ela não deixa alterações parciais no banco.

- Executadas dentro de uma única **transação**.

- Não podem executar comandos de controle de transação (COMMIT, ROLLBACK).

FUN

Dentro de uma função não é permitido fazer COMMIT ou ROLLBACK, porque isso quebraria justamente a regra anterior. Quem controla a transação é quem chamou a função, e não a função em si.

COMMIT:

- O COMMIT é o comando para confirmar/efetivar todas as alterações feitas na transação no banco de dados.

ROLLBACK:

- O ROLLBACK é o comando para desfazer/reverter todas as alterações feitas na transação no banco de dados.

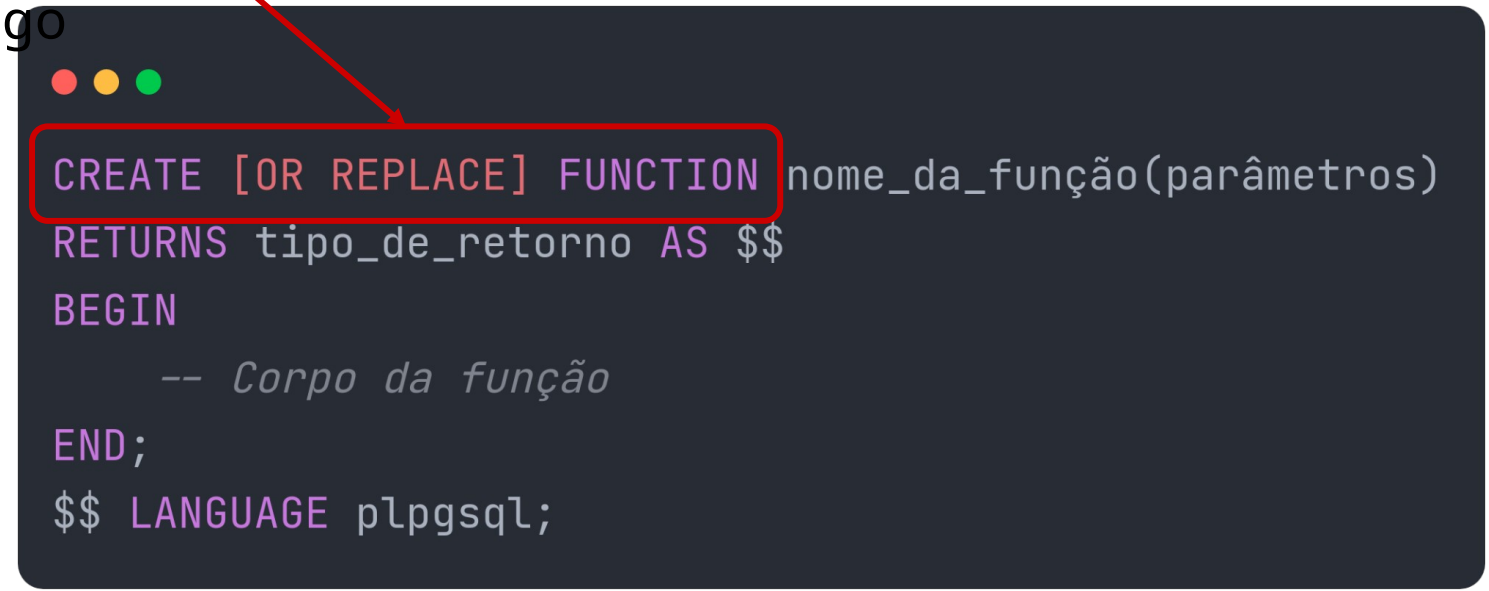
- Não pode executar comandos de controle de transação (**COMMIT**, **ROLLBACK**).

SINTAXE

- Podem ser escritas em várias linguagens, sendo a mais comum a PL/pgSQL (Procedural Language/PostgreSQL).

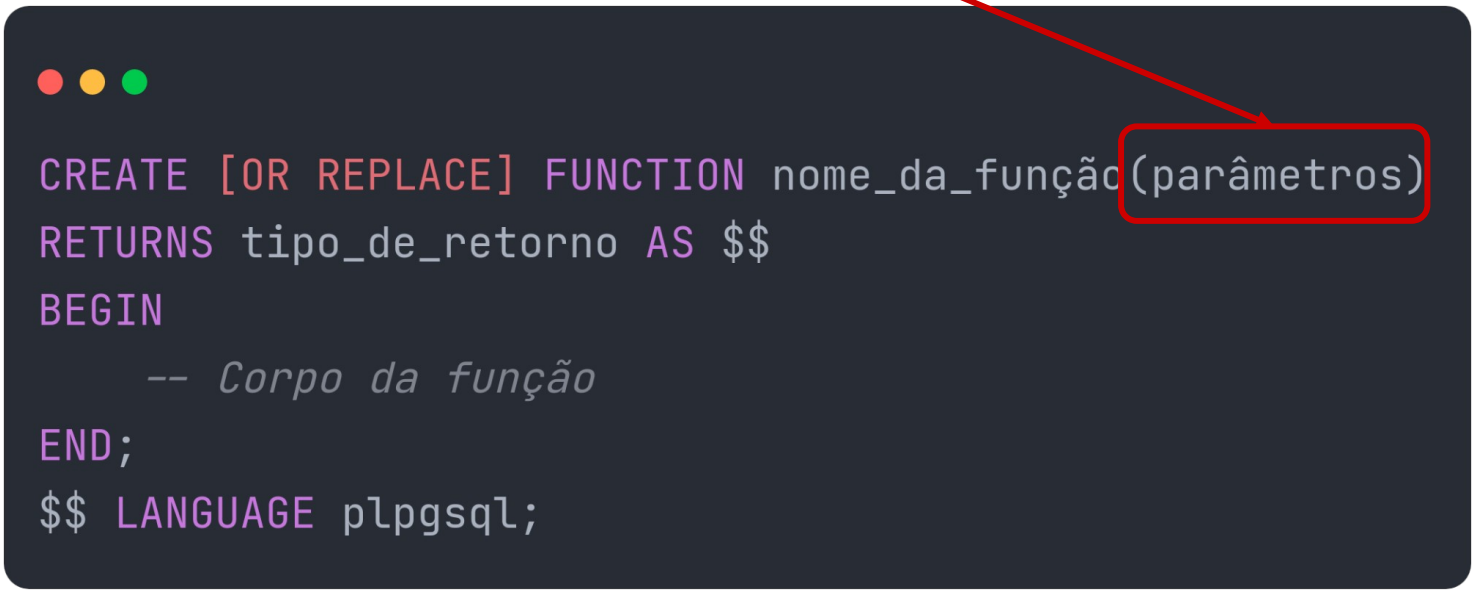
```
CREATE [OR REPLACE] FUNCTION nome_da_função(parâmetros)
RETURNS tipo_de_retorno AS $$
BEGIN
    -- Corpo da função
END;
$$ LANGUAGE plpgsql;
```

Cria uma nova função, se não existir, ou substitui uma existente pelo novo código



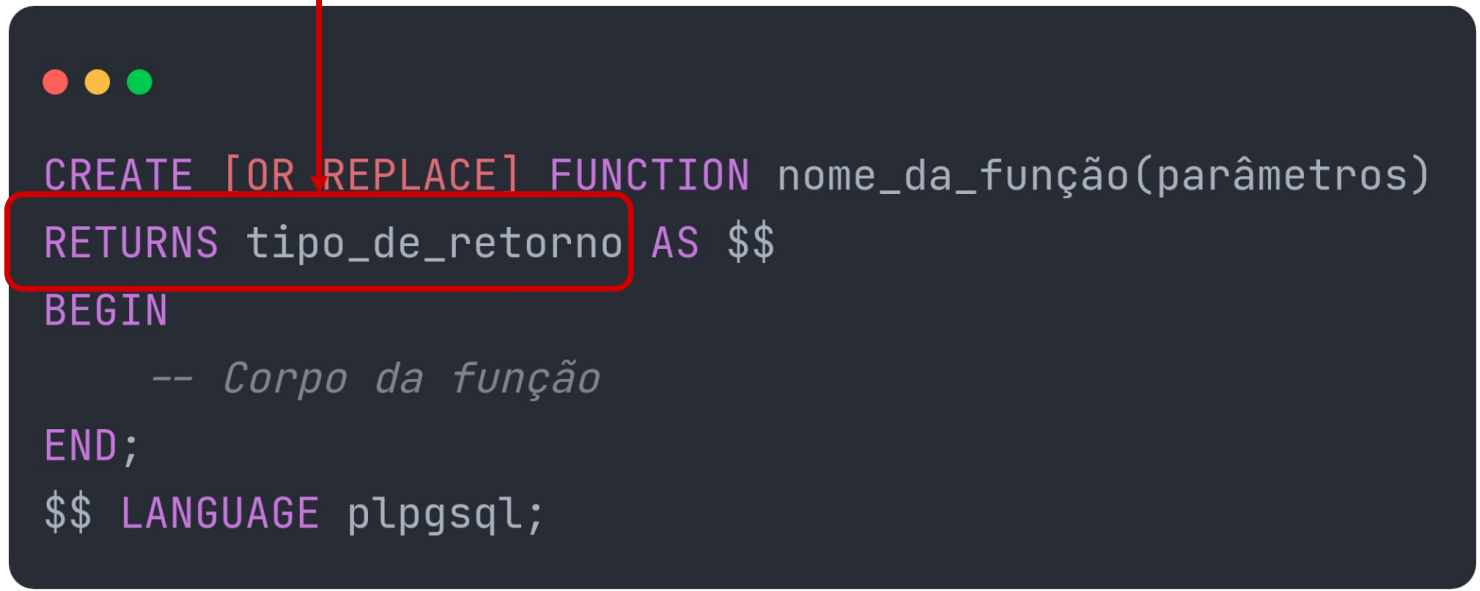
```
CREATE [OR REPLACE] FUNCTION nome_da_função(parâmetros)
RETURNS tipo_de_retorno AS $$
BEGIN
    -- Corpo da função
END;
$$ LANGUAGE plpgsql;
```

Lista de parâmetros de entrada



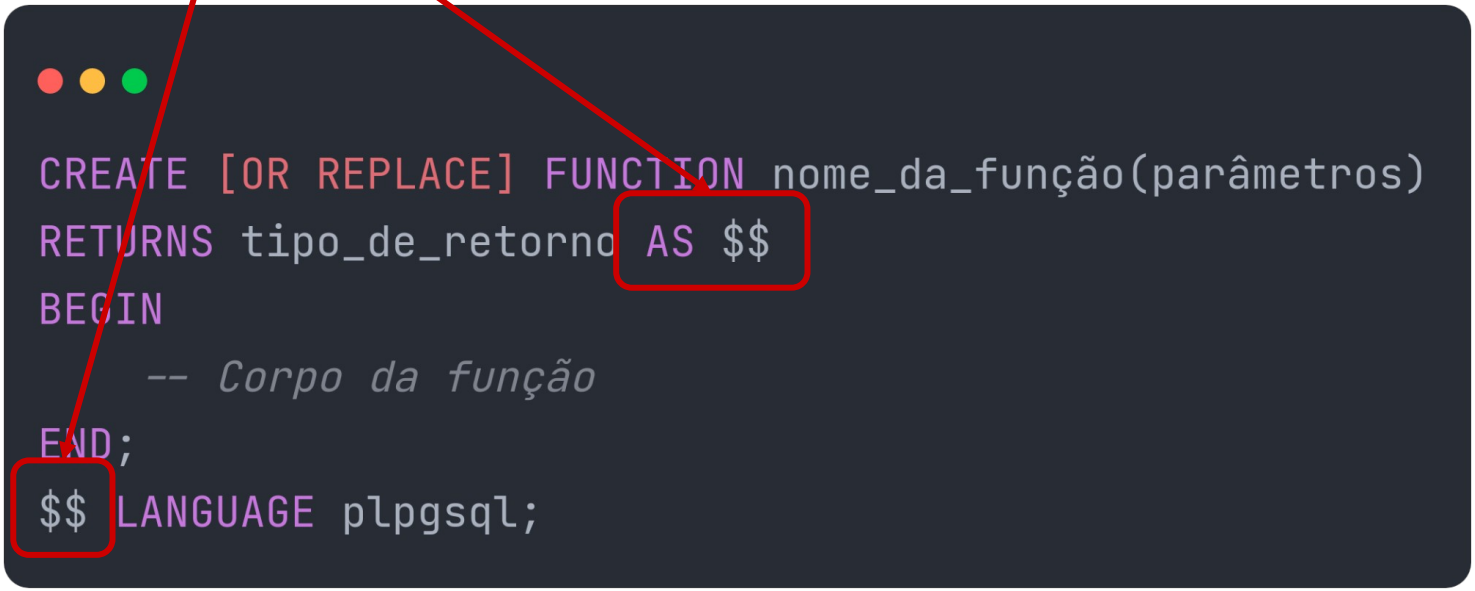
```
CREATE [OR REPLACE] FUNCTION nome_da_função(parâmetros)
RETURNS tipo_de_retorno AS $$
BEGIN
    -- Corpo da função
END;
$$ LANGUAGE plpgsql;
```

Define o tipo de retorno da função.

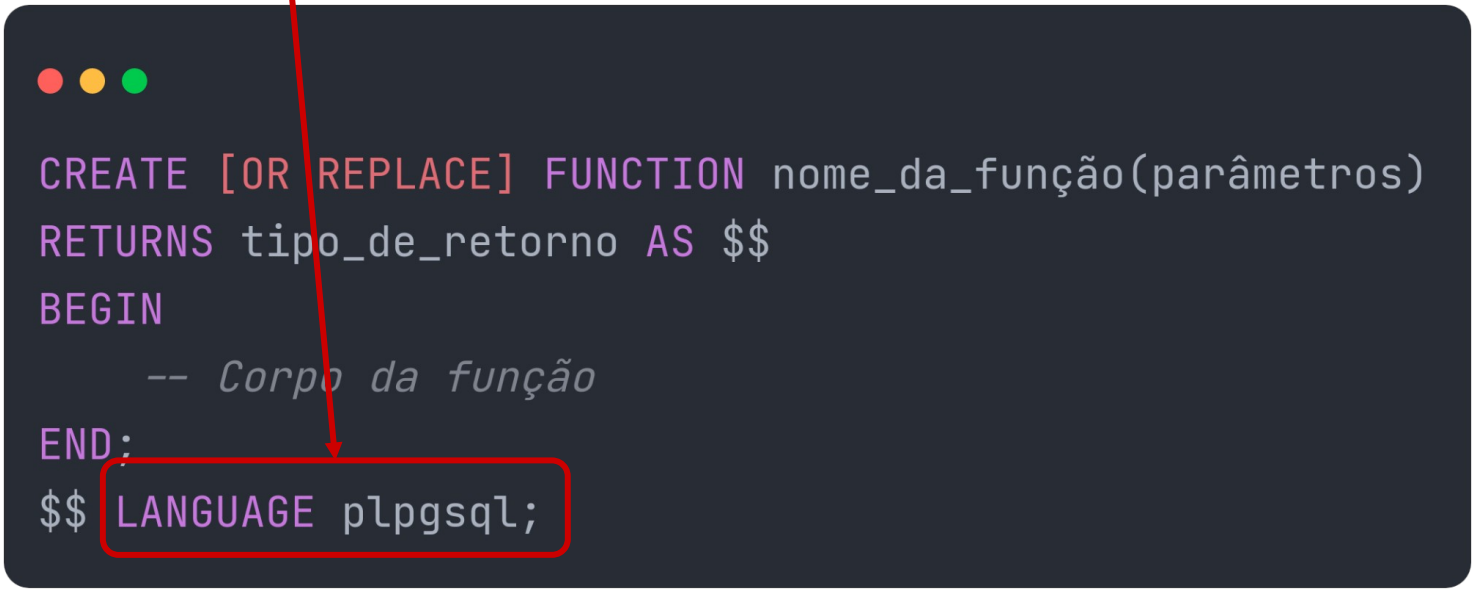


```
CREATE [OR REPLACE] FUNCTION nome_da_função(parâmetros)
RETURNS tipo_de_retorno AS $$
BEGIN
    -- Corpo da função
END;
$$ LANGUAGE plpgsql;
```

Delimitador do corpo da função.



Especifica a linguagem utilizada.



EXEMPLO (1) - FUNÇÕES

Criar uma função que recebe o **ID de um cliente** e retorna o ***total gasto por esse cliente em todos os seus pedidos***.

```sql
CREATE OR REPLACE FUNCTION calcular_total_cliente(cliente_id_input INTEGER)
RETURNS NUMERIC(10, 2) AS $$
DECLARE
    total_cliente NUMERIC(10, 2);
BEGIN
    SELECT SUM(total) INTO total_cliente
    FROM pedidos
    WHERE cliente_id = cliente_id_input;

    -- Se o cliente não tiver pedidos, retornar zero
    IF total_cliente IS NULL THEN
        total_cliente := 0;
    END IF;

    RETURN total_cliente;
END;
$$ LANGUAGE plpgsql;
```

```sql
CREATE OR REPLACE FUNCTION calcular_total_cliente(cliente_id_input INTEGER)
RETURNS NUMERIC(10, 2) AS $$
DECLARE
    total_cliente NUMERIC(10, 2);
BEGIN
    SELECT SUM(total) INTO total_cliente
    FROM pedidos
    WHERE cliente_id = cliente_id_input;

    -- Se o cliente não tiver pedidos, retornar zero
    IF total_cliente IS NULL THEN
        total_cliente := 0;
    END IF;

    RETURN total_cliente;
END;
$$ LANGUAGE plpgsql;
```

Declaramos uma variável local

Armazena o resultado da soma os valores da coluna total para aquele cliente específico dentro da variável total_cliente

```sql
CREATE OR REPLACE FUNCTION calcular_total_cliente(cliente_id_input INTEGER)
RETURNS NUMERIC(10, 2) AS $$
DECLARE
    total_cliente NUMERIC(10, 2);
BEGIN
    SELECT SUM(total) INTO total_cliente
    FROM pedidos
    WHERE cliente_id = cliente_id_input;

    -- Se o cliente não tiver pedidos, retornar zero
    IF total_cliente IS NULL THEN
        total_cliente := 0;
    END IF;

    RETURN total_cliente;
END;
$$ LANGUAGE plpgsql;
```

IF condição THEN
-- comandos
END IF;

:= é o operador de atribuição

- Utilizando a função

```
SELECT nome, calcular_total_cliente(cliente_id) AS total_gasto
FROM clientes;
```

- Resultado

	nome	total_gasto
1	Theresa Collins	150.00
2	Joshua Tucker	0.00
3	James Smith	0.00

EXEMPLO (2) - FUNÇÕES

Criar uma função que retorna o estoque disponível de um produto específico.

```sql
CREATE OR REPLACE FUNCTION obter_estoque_disponivel(produto_id_input INTEGER)
RETURNS INTEGER AS $$
DECLARE
    estoque_disponivel INTEGER;
BEGIN
    SELECT disponivel INTO estoque_disponivel
    FROM produtos
    WHERE produto_id = produto_id_input;

    RETURN estoque_disponivel;
END;
$$ LANGUAGE plpgsql;
```

- Utilizando a função

```sql
SELECT nome, obter_estoque_disponivel(produto_id) AS estoque
FROM produtos
WHERE produto_id = 1;
```

- Resultado

	nome	estoque
1	Notebook Dell Inspiron	10

EXEMPLO (3) - FUNÇÕES

Função que retorna todos os produtos pertencentes a uma categoria específica.

```sql
CREATE OR REPLACE FUNCTION listar_produtos_por_categoria(categoria_id_input INTEGER)
RETURNS TABLE(produto_id INTEGER, nome VARCHAR, preco NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT p.produto_id, p.nome, p.preco
    FROM produtos p
    JOIN produtos_categorias pc ON p.produto_id = pc.produto_id
    WHERE pc.categoria_id = categoria_id_input;
END;
$$ LANGUAGE plpgsql;
```

- Utilizando a função

```
SELECT * FROM listar_produtos_por_categoria(14);
```

- Resultado

	produto_id	nome	preco
1	31	Calça Legging Fitness	90.00
2	4	Vestido Longo Floral	120.00

IF / ELSE IF / ELSE

```
CREATE OR REPLACE FUNCTION verificar_idade(idade INTEGER) RETURNS TEXT AS $$
BEGIN
    IF idade < 18 THEN
        RETURN 'Menor de idade';
    ELSIF idade ⩾ 18 AND idade < 65 THEN
        RETURN 'Adulto';
    ELSE
        RETURN 'Idoso';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

e muito mais...

- LOOP - continua executando até encontrar um EXIT.

```
LOOP
    -- comandos
    EXIT WHEN condição;  -- encerra o loop quando a condição for verdadeira
END LOOP;
```

- WHILE - enquanto uma condição é verdadeira.

```
WHILE condição LOOP
    -- comandos
END LOOP;
```

e muito mais...

- CASE

```
CASE
    WHEN condição1 THEN
        -- comandos
    WHEN condição2 THEN
        -- comandos
    ELSE
        -- comandos padrão
END CASE;
```

e muito mais...

- FOR ... IN (para iterações sobre valores) ou sobre resultados de uma consulta.

```
FOR i IN 1..10 LOOP
    -- comandos usando i
END LOOP;
```

```
FOR registro IN SELECT id, nome FROM clientes LOOP
    -- usar registro.id ou registro.nome
END LOOP;
```

PROCEDIMENTOS

- Introduzidos no PostgreSQL 11.
- Não retornam valores diretamente (podem usar parâmetros OUT).
- Executados usando o comando CALL.
- Não podem ser usadas em consultas SQL.
- Podem executar comandos de controle de transação.

PRO

Diferente das funções, os procedimentos não usam RETURN valor.
Para devolver dados, precisam usar parâmetros de saída (OUT) ou registrar resultados em tabelas.

- Não retornam valores diretamente (podem usar parâmetros OUT).
- Executados usando o comando CALL.
- Não podem ser usadas em consultas SQL.
- Podem executar comandos de controle de transação.

PROCEDIMENTOS

Enquanto funções são chamadas com SELECT nome_funcao(), os procedimentos são executados com:

CALL nome_procedimento(param1, param2);

Procedimentos só podem ser chamados com CALL, nunca em um SELECT.

- Executados usando o comando CALL.
- Não podem ser usadas em consultas SQL.
- Podem executar comandos de controle de transação.

PROCEDIMENTOS

- Não podem ser usadas em consul

Essa é a maior diferença em relação às funções.

Dentro de um procedimento, você pode usar COMMIT, ROLLBACK,

Por esse motivo, se você precisar de um bloco de lógica que atualiza dados e controla transações diretamente, o ideal é usar um PROCEDIMENTO

- Podem executar comandos de controle de transação.

CRIANDO PROCEDIMENTOS

SINTAXE

- Procedimentos permitem executar blocos de código armazenados no banco de dados. Eles são úteis para encapsular operações que não precisam retornar valores diretamente.

```
CREATE [OR REPLACE] PROCEDURE nome_do_procedimento(parâmetros)
LANGUAGE plpgsql AS $$
BEGIN
    -- Corpo do procedimento
END;
$$;
```

EXEMPLO (1) - PROCEDIMENTOS

Procedimento que atualiza o estoque de um produto após a venda.

```sql
CREATE OR REPLACE PROCEDURE atualizar_estoque_produto(
    produto_id_input INTEGER,
    quantidade_vendida INTEGER
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE produtos
    SET
        estoque = estoque - quantidade_vendida,
        disponivel = disponivel - quantidade_vendida
    WHERE produto_id = produto_id_input;

    -- Verificar se o estoque ficou negativo
    IF (SELECT estoque FROM produtos WHERE produto_id = produto_id_input) < 0 THEN
        RAISE NOTICE 'Estoque insuficiente para o produto ID %.', produto_id_input;
    ELSE
        RAISE NOTICE 'Estoque atualizado com sucesso para o produto ID %.', produto_id_input;
    END IF;
END;
$$;
```

- Utilizando o procedimento

```
CALL atualizar_estoque_produto(1, 5);
```

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software

Revisão
Modelo Conceitual

Revisão

➔ **Modelo Conceitual**

- Está mais próximo da visão que o usuário tem dos dados.
- É o nível mais abstrato e independente de tecnologia
- O Modelo Entidade-Relacionamento (MER), proposto por Peter Chen, é a principal técnica.
- Representa entidades, atributos e relacionamentos de forma gráfica, sem pensar ainda em tabelas, chaves estrangeiras ou restrições de implementação.
- Como esses dados se relacionam.

Revisão

- **Modelo conceitual**

Tem por base a percepção de que o mundo real é formado por conjunto de objetos chamados *entidades* e pelo conjunto dos *relacionamentos* entre esses objetos.

Revisão

➔ **Modelo Conceitual**

- Retângulos, que representam os conjuntos de entidades;
- Uma **entidade** representa um conjunto de **objetos** do mesmo tipo do mundo real e sobre os quais se pretende armazenar dados.

- Elipses, que representam os atributos
- **Atributo** é um dado que é associado a cada ocorrência de uma entidade ou de um relacionamento, descrevendo suas características e propriedades
- Todo atributo possui um **tipo de dado.**
  - os valores permitidos para aquele atributo**.**

Revisão

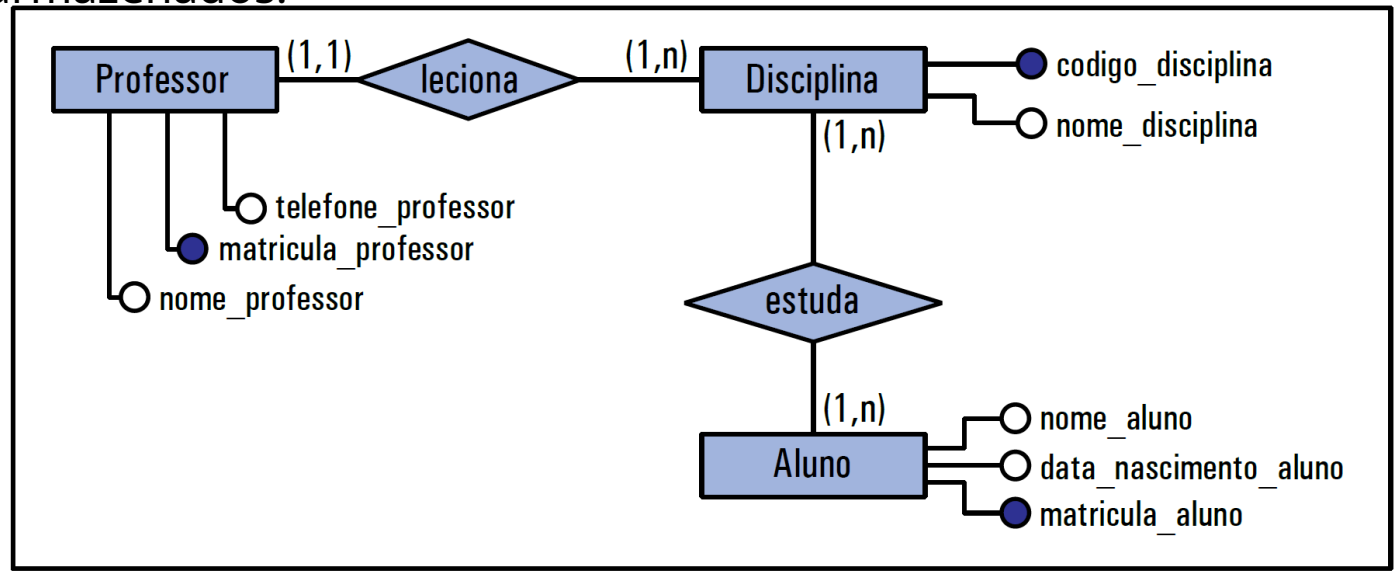
➔ **Modelo Conceitual**

- Losangos, que representam os relacionamentos entre os conjuntos de entidades;
- Um **relacionamento** é uma associação entre as **entidades**.
- Podem haver **atributos dos relacionamentos**

- Linhas, que unem os atributos aos conjuntos de entidades e o conjunto de entidades aos seus relacionamentos.

Revisão

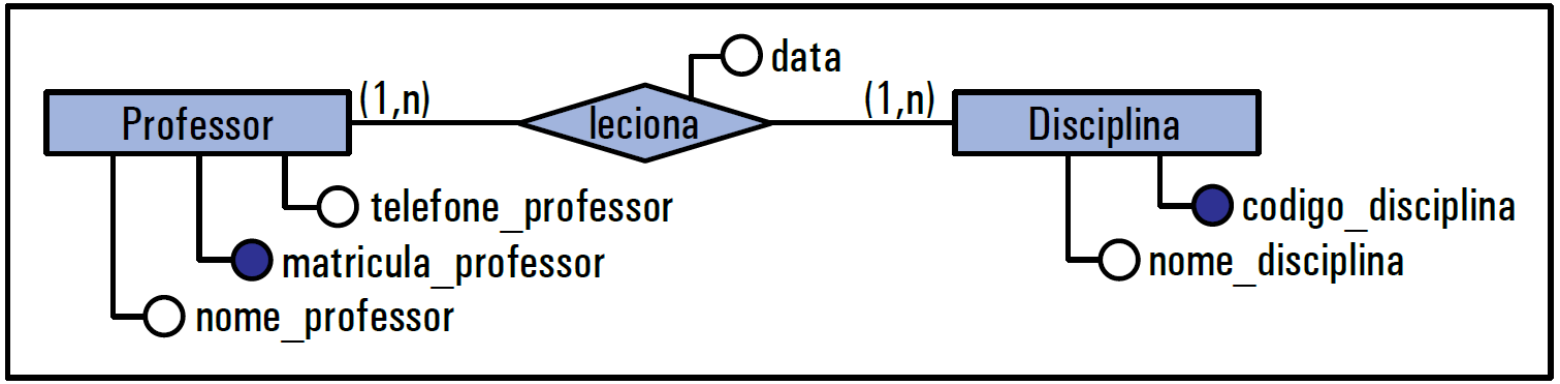
- Não há preocupação em saber como os dados serão armazenados.



Exemplo de modelo conceitual

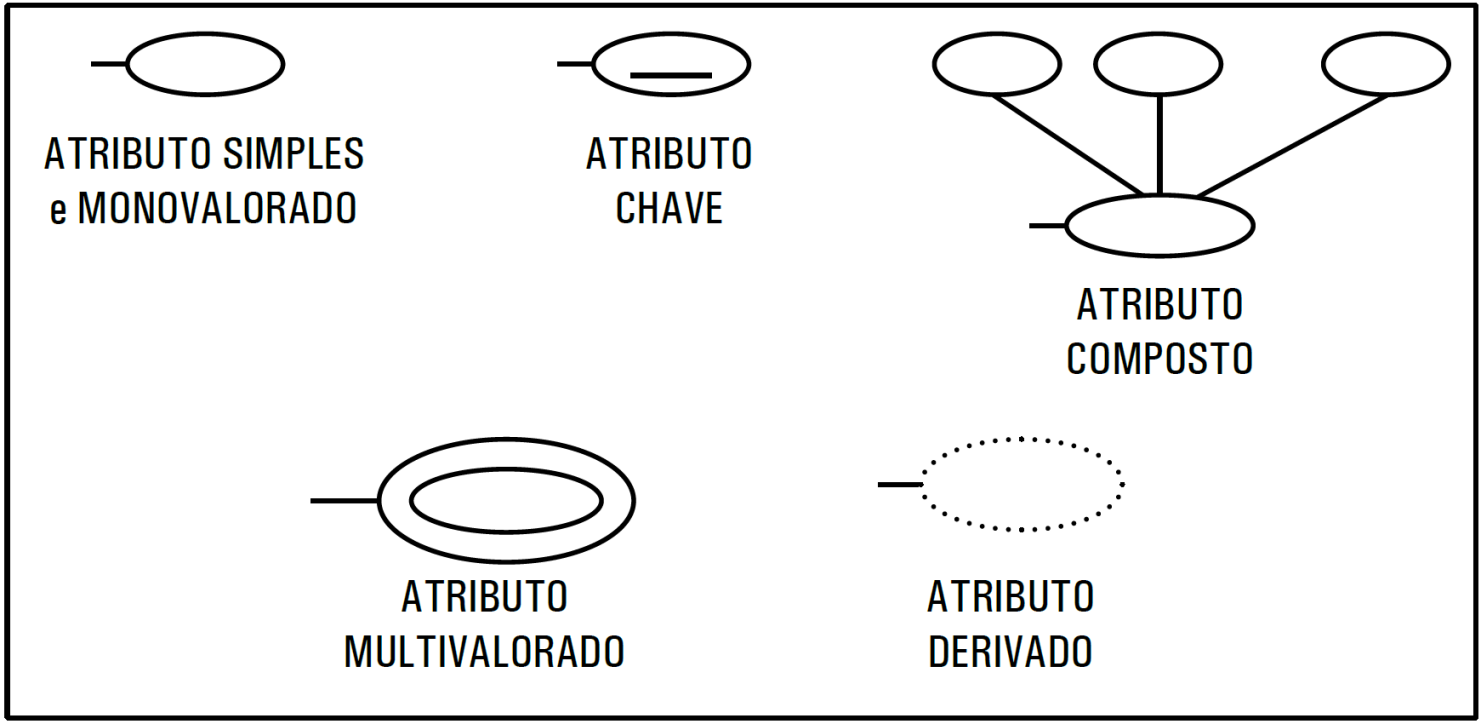
Revisão

- Não há preocupação em saber como os dados serão armazenados.



Exemplo de modelo conceitual

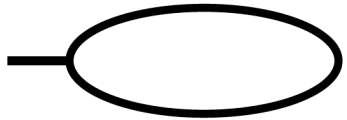
Revisão



Representação gráfica dos atributos

ATRIBUTO SIMPLES E MONOVALORADO

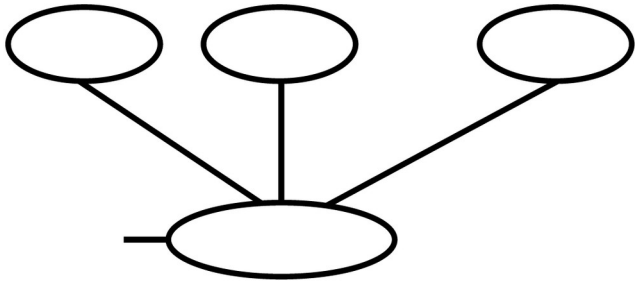
- Simples: não pode ou não deve ser decomposto.
- Monovalorado: permite apenas o armazenamento de um valor por vez
- **Indivisível**
  - CPF
  - nº matrícula
  - RG
  - preço
  - etc.



ATRIBUTO SIMPLES
e MONOVALORADO

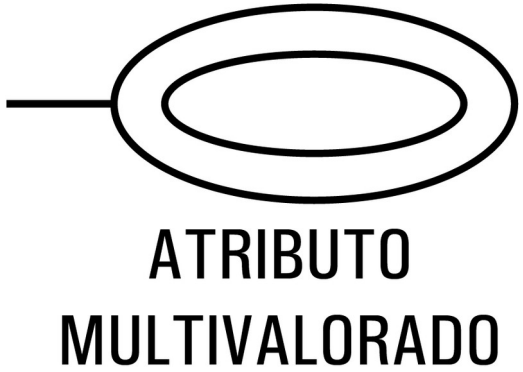
ATRIBUTO COMPOSTO

- pode ser decomposto.
  - Endereço
    - Nome da rua
    - número
    - complemento



ATRIBUTO MULTIVALORADO

- Armazena mais de um valor ao mesmo tempo no mesmo campo
  - E-mail
  - Uma pessoa pode ter vários e-mails.



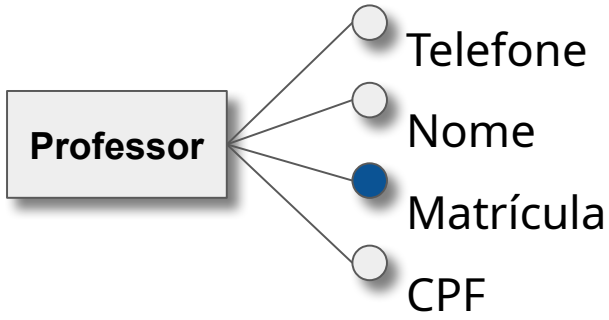
ATRIBUTO DERIVADO

- O valor para ele deriva de outro(s) atributo(s).
- número da compra, data da compra, valor da compra, percentual de desconto e **valor da compra com o desconto**.
- “**valor da compra com o desconto**” é *calculado/derivado* de outros atributos.
- Aplicação ou SGBD



Revisão

- **Primary Key** ou ainda **PK**
- É um atributo da entidade que **identifica apenas um objeto** dessa entidade.
- Esse valor não pode se repetir (**único**)
- Não pode receber nulo.
- Professor (Entidade)
  - CPF, Matrícula.





ATRIBUTO CHAVE

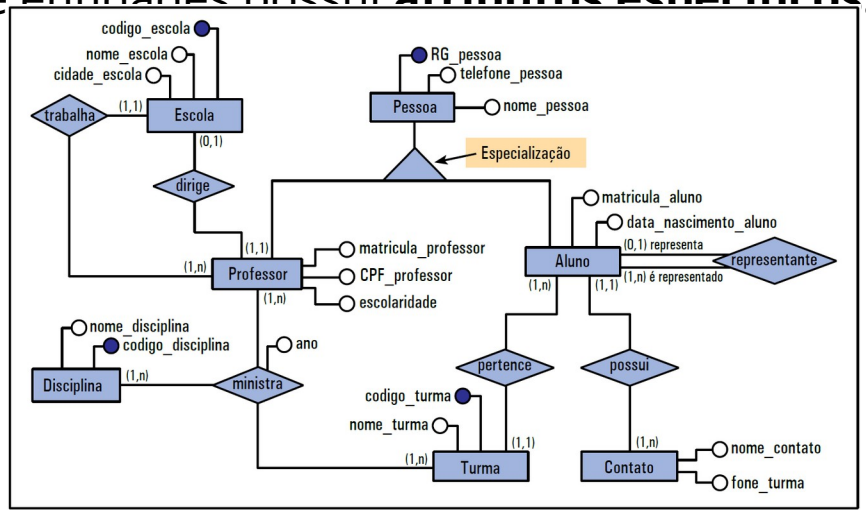
Revisão

- **PK SIMPLES**
  - Formada por **apenas um** atributo
  - Exemplo:
    - matrícula
- **PL COMPOSTA**
  - Formada por **dois ou mais** atributos.
  - Exemplo:
    - Entidade (Localização)
      - nome da cidade e nome do estado

Revisão

- **<u>Especialização:</u>** definir entidades mais específicas (subclasses) a partir de uma entidade mais genérica (superclasse), herdando os atributos da superclasse

- Cada subgrupo de entidades possui **atributos específicos**.



Revisão
Modelo Lógico

Revisão

- **<u>Modelo lógico,</u>** diferentemente do projeto conceitual, que é voltado para a validação com o cliente, esse é **voltado para a equipe de desenvolvimento;**

- Descreve ***como será a estrutura do banco de dados***;

- “Ele traduz o MER para uma estrutura compatível com o tipo de banco que será usado (relacional, orientado a objetos, etc.).”

Revisão

- O **Modelo Relacional: o**rganiza os dados em uma estrutura de **tabelas** bidimensionais denominadas **relações**;
- Cada linha da tabela (**tupla)** representa um elemento do conjunto de dados;
- Cada coluna da tabela **(atributo)** contém valores de um conjunto definido **(domínio)**
- **No caso do relacional, o modelo lógico já mostra tabelas, chaves primárias, chaves estrangeiras, relacionamentos como colunas etc.**

Revisão

- Tabelas

Atributo

Nome	Matrícula	Cargo	Salário
João	111111	Diretor	4500,00
Maria	121212	Atendente	600,00
Sérgio	131313	Eletricista	550,00
Carlos	141414	Vigilante	400,00

Tupla?
registro completo que representa uma ocorrência específica de uma entidade.

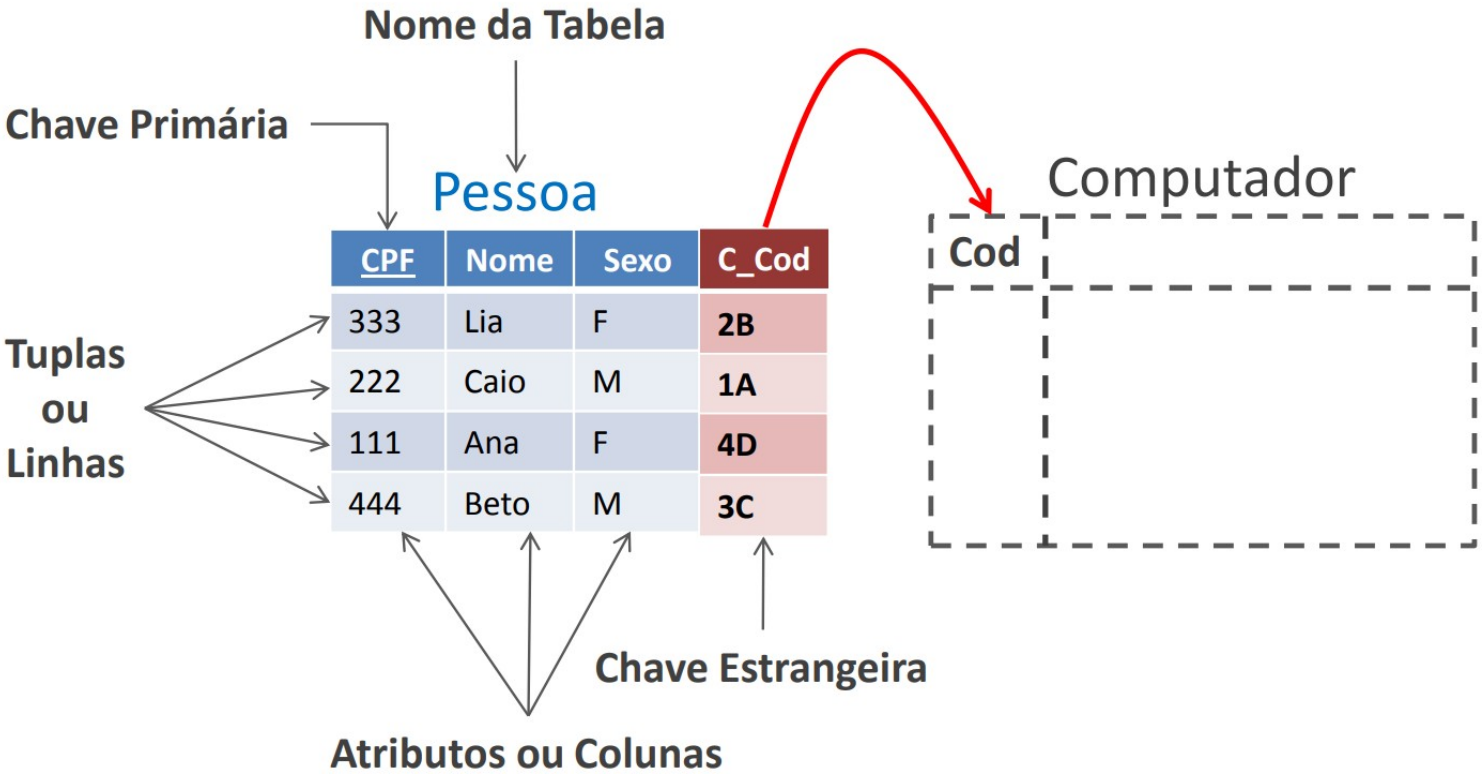
Cada tabela é um conjunto desordenado de **tuplas** exclusivas.

Revisão

- Valor do campo deve ser **atômico**:
  - Simples (não é divisível); e
  - Monovalorado (só possui um valor).

- O relacionamento das tabelas se dá por meio de **Chaves estrangeiras**.

Revisão


Nome da Tabela
Chave Primária
Pessoa
CPF
Nome
Sexo
C_Cod
333
Lia
F
2B
222
Caio
M
1A
111
Ana
F
4D
444
Beto
M
3C
Tuplas
ou
Linhas
Computador
Cod
Chave Estrangeira
Atributos ou Colunas

Revisão

- Chave primária → Coluna ou combinação de colunas cujos valores distinguem uma linha das demais dentro de uma tabela.

- Candidatas → “uma possível chave primária”

Uma tabela pode ter várias chaves candidatas. Entre elas, uma é escolhida como chave primária; as outras ficam como chaves alternativas.

Revisão

Pessoa

CPF	Matricula	Nome	Sexo
333	2B	Lia	F
222	1A	Caio	M
111	3C	Ana	F
444	5E	Beto	M

Possibilidade 1

- Chaves candidatas:
  - CPF e Matricula.
- Chave Primária:
  - CPF.
- Chave Alternativa ou alternada:
  - Matrícula.

Possibilidade 2

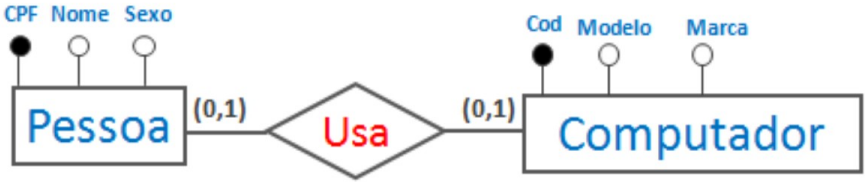
- Chaves candidatas:
  - CPF e Matricula.
- Chave Primária:
  - Matrícula.
- Chave Alternativa ou alternada:
  - CPF.

Revisão

- **Chave estrangeira:** Coluna ou combinação de colunas, cujos valores aparecem na chave primária (candidata) da tabela referenciada.

- Mecanismo que permite a implementação de relacionamentos em um banco de dados relacional.

Conceitual



Lógico

Pessoa

CPF	Nome	Sexo	C_Cod
111	Ana	F	1A
222	Beto	M	2B
333	Lia	F	3C
444	Caio	M	NULL

Computador

Cod	Modelo	Marca
1A	Notebook	Apple
2B	Desktop	Dell
3C	Desktop	IBM
4D	Netbook	HP

Revisão de Normalização

Revisão

- Ao modelar nossos banco de dados devemos evitar **anomalias**
- Anomalias são mudanças em dados que podem gerar uma inconsistência no banco de dados relacional
- Dependendo do momento em que ocorre a inconsistência podemos ter
  -Anomalia de inserção
  -Anomalia de atualização
  -Anomalia de remoção

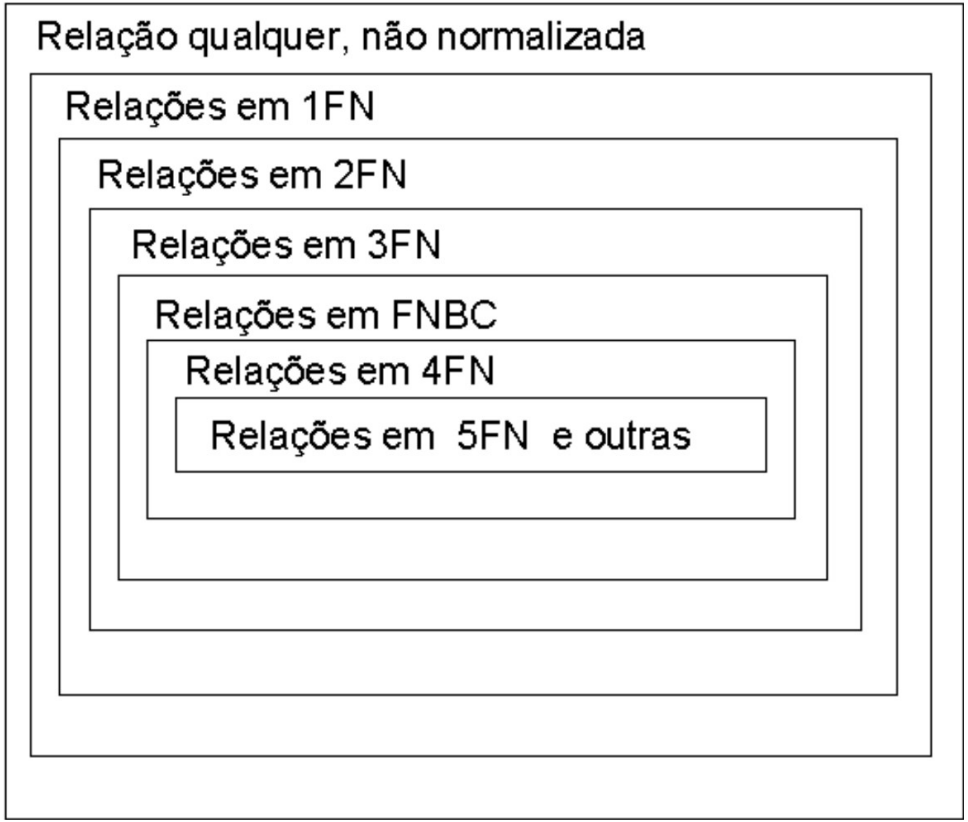
Revisão

- **Normalização:** Mecanismo formal para **analisar esquemas** de relações baseado nas suas chaves e nas dependências funcionais entre seus atributos.

- Consiste da decomposição de esquemas para evitar **anomalias**

- ***Projeto conceitual bem feito resulta naturalmente em esquemas normalizados.***

Revisão


Relação qualquer, não normalizada
Relações em 1FN
Relações em 2FN
Relações em 3FN
Relações em FNBC
Relações em 4FN
Relações em 5FN e outras

Primeira Forma Normal (1FN)

- É parte da definição formal de uma relação.
- Foi definida para **não permitir atributos multivalorados**, atributos **compostos** e suas combinações.

"Uma **relação/tabela** está em **1FN** se e somente se todos os seus valores de atributos contêm apenas valores **atômicos** (*simples, indivisíveis*)."

Segunda Forma Normal (2FN)

"Uma **relação/tabela** encontra-se na **2FN** se e somente se estiver em **1FN** e *não contém dependências parciais*, ou seja, todos os atributos não-chave dependem completamente da chave primária, e não apenas de uma parte dela"

- **Dependência Parcial:** ocorre quando uma coluna depende apenas de uma parte de uma chave primária composta.
- Uma relação para estar na 2FN não deve possuir atributo não-chave funcionalmente determinado por parte da chave primária.

Terceira Forma Normal (3FN)

"Uma relação está em **3FN** se e somente se estiver na **2FN** e todos os seus atributos não-chave forem mutuamente independentes, ou seja, **não houver dependências transitivas**."

- **Dependência Transitiva:** ocorre quando uma coluna, além de depender da chave primária de uma tabela, depende de outra coluna ou conjunto de colunas da tabela.

- Uma relação para estar na 3FN não deve ter um atributo não-chave funcionalmente determinado por um outro atributo não-chave.

Dependência Funcional

- Em uma tabela relacional, diz-se que uma coluna Y depende funcionalmente de uma coluna X quando, em todas as linhas da tabela, para cada valor de X que aparece na tabela, o mesmo valor de Y.

X → Y

Lê-se que X determina funcionalmente Y

Exemplo: cod_produto → nome_produto

cod_produto	nome_produto
101	Teclado USB
102	Mouse Óptico
103	Monitor 24"

Dependência Funcional

- A **dependência funcional trivial** fala que um determinante com mais de um atributo pode determinar seus próprios membros quando isolados. Um exemplo:
- [banco, agência] --> banco (DF trivial, pois banco é parte do determinante)
- [banco, agência] --> agência (DF trivial, pois agência é parte do determinante)

- Quando um determinante identifica outro atributo qualquer, temos uma dependência **funcional não trivial**. Um exemplo:
- [banco, agência] --> cidade (DF não trivial, pois cidade não faz parte do determinante)

Forma Normal de Boyce-Codd

- A formal normal de Boyce-Codd (FNBC) foi proposta como uma forma mais simples da 3FN, entretanto, essa forma normal acaba sendo mais rigorosa do que 3FN;

- Um esquema de relação está na FNBC Para toda dependência funcional não trivial X → Y, o conjunto de atributos X deve ser uma superchave da relação.

- **Superchave** = conjunto de atributos que identifica unicamente cada linha da tabela.

- Em outras palavras: nenhum atributo que não seja chave pode determinar outro atributo.